

IES FRANCISCO DE GOYA

GymTracker

Proyecto fin de Ciclo Formativo

Daniel Marín Gómez
Álvaro Gallego Yáñez
David Martín Díaz



Contenido

1 Introducción.....	7
1.1 Objetivos iniciales.....	8
1.2 Tecnologías.....	8
1.2.1 HTML5.....	8
1.2.2 CSS3.....	8
1.2.3 Bootstrap.....	9
1.2.4 JavaScript.....	9
1.2.4.1 Chart.js.....	9
1.2.4.2 Axios.....	9
1.2.5 MySQL y MySQL Workbench.....	10
1.2.6 GitHub.....	10
1.2.7 Java.....	11
1.2.8 Spring Boot.....	11
1.2.8.1 Dependencias para la API de ejercicios.....	11
1.2.8.2 Dependencias para Gymtracker.....	12
1.2.9 Docker.....	13
1.2.10 AWS (instancia linux amazon).....	13
1.2.11 Shell.....	13
2 Análisis de Requisitos.....	14
2.1 Mapa de pantallas.....	15
2.2 Manual de usuario.....	17
2.2.1 Acceso a GymTracker.....	17
2.2.2 Inicio.....	18
2.2.2.1 Inicio Sesión.....	18
2.2.2.2 Crear usuario.....	19
2.2.2.3 Crear Perfil.....	20
2.2.3 Menú.....	20
2.2.4 Nueva rutina.....	22
2.2.4.1 Añadir ejercicio.....	22
2.2.4.2 Añadir Serie.....	23
2.2.5 Mis entrenamientos.....	24
2.2.6 Ejercicios.....	26
2.2.7 Estadísticas.....	27
2.2.8 Clasificación.....	29
2.2.9 Nuestras rutinas.....	30
2.2.10 Calendario.....	31
2.2.11 Perfil.....	31
2.2.11.1 Editar perfil.....	32
2.2.11.2 Cambiar contraseña.....	33

2.2.12 Menú Sidebar.....	34
2.2.13 Error.....	35
2.3 Manual de administrador.....	36
2.3.1 Administrar Usuarios.....	36
2.3.1.1 Cambiar contraseña del usuario.....	37
2.3.1.2 Acceder a la cuenta del usuario.....	37
2.3.1.3 Eliminar usuario.....	37
2.3.1.4 Crear usuario.....	38
3. Adecuación del entorno de trabajo.....	39
3.1 Instalación de MySQL.....	39
3.2 Instalación de Git.....	39
3.3 Clonación del Repositorio.....	39
3.4 Instalación de JDK.....	39
3.4.1 Instalación de JDK en Linux.....	40
3.4.2 Instalación de JDK en Windows.....	40
4.4.2.1 Configurar la variable de entorno JAVA_HOME:.....	40
3.5 Instalación de Maven.....	40
3.5.1 Instalación de Maven en Linux.....	40
3.5.2 Instalación de Maven en Windows.....	40
3.5.2.1 Configurar la variable de entorno MAVEN_HOME.....	41
3.6 Configuración del IDE (STS - Spring Tool Suite).....	41
3.6.1 Instalación de Spring Tool Suite.....	41
3.6.2 Configuración de Lombok.....	41
3.6.3 Importar Proyectos con STS.....	42
3.7 Configuración del IDE (IntelliJ IDEA).....	42
3.7.1 Instalación de IntelliJ IDEA.....	42
3.7.2 Importar Proyectos con IntelliJ IDEA.....	42
4. Arquitectura del proyecto.....	43
4.1 Arquitectura de la API.....	43
4.1.1 Estructura y Funcionalidad.....	44
4.1.2 Recursos.....	44
4.1.3 Tecnologías Utilizadas.....	45
4.2 Arquitectura de la APP.....	46
4.2.1. Configuración.....	47
4.2.2. Controladores (Controllers).....	47
4.2.3. Data Transfer Objects (DTO).....	47
4.2.4. Servicios (Services).....	47
4.2.5. Recursos (Resources).....	48
4.2.6 Dependencias.....	48
4.2.7 Mapeo con ORM.....	48
4.2.8 Flujo de Datos.....	48
4.2.9 Gestión de Seguridad y Sesiones.....	49

4.2.10	Uso de Lombok.....	49
4.2.11	Conclusión.....	49
5.	Desarrollo de la base de datos.....	49
5.1	Arquitectura de la base de datos.....	50
5.1.1	Esquema de la base de datos.....	50
5.1.2	Tablas Principales.....	50
5.1.3	Relación entre Rutinas, Ejercicios y Series.....	50
5.1.4	Relaciones.....	51
5.1.5	Claves Externas y Borrados en Cascada.....	51
5.2	Explicación y justificación de la base de datos.....	51
5.2.1	Integridad de los Datos.....	52
5.2.2	Escalabilidad.....	52
5.2.3	Eficiencia en Consultas.....	52
5.2.4	Consistencia y Mantenibilidad.....	52
6.	Desarrollo del proyecto.....	52
6.1	Desarrollo de la API.....	52
6.1.1	Configuración CORS para el acceso a la API.....	53
6.1.2	Mapeo de la Entidad Ejercicio y Creación de la Tabla en H2.....	54
6.1.3	Repositorio para la Entidad Ejercicio.....	55
6.1.4	Controlador para la Gestión de Ejercicios.....	58
6.1.5	Creación de la Base de Datos H2 con Ejercicios mediante el archivo import.sql...59	
6.2	Desarrollo de la APP.....	60
6.2.1.	Explicación del diseño de la web.....	60
6.2.2.	Implementación de Spring Security.....	61
6.2.2.1	Configuración de Spring Security.....	61
6.2.2.2	Cargar los usuarios de la base de datos a Spring Security.....	65
6.2.2.2	Ajuste de Métodos CRUD en Servicios para Spring Security.....	67
6.2.2.2.1	Cambios en UsuarioService.....	67
6.2.2.2.2	Cambios en PerfilService.....	69
6.2.3	Esquema de Plantillas en Bootstrap para GymTracker.....	71
6.2.4	Explicación de la dinámica general seguida para el MVC.....	72
6.2.4.1	Controlador.....	72
6.2.4.2	Servicio.....	74
6.2.4.2.1	Mapeo de datos de la base de datos a DTO.....	75
6.2.4.3	Plantilla.....	76
7.	Despliegue.....	77
7.1	Despliegue en Aws.....	77
7.1.1	Lanzar instancia.....	77
7.1.2	Conectarse a la instancia.....	80
7.1.3	Preparar instancia para el despliegue.....	81
7.1.4	Ejecutar GymTracker.....	82
7.1.5	IP Elástica.....	84

7.2 Despliegue en local con Docker.....	87
7.2.1 Instalación de docker.....	87
7.2.1.1 Linux:.....	87
7.2.1.2.Para Windows:.....	87
7.2.2 Preparación para el despliegue.....	90
7.2.3 Despliegue.....	95
7.3 Despliegue en local con Spring Tool Suite (STS).....	96
7.3.1 Prerrequisitos.....	96
7.3.1.1 Instalación de Prerrequisitos.....	96
Windows.....	96
7.3.1.1.1 Instalar JDK.....	96
7.3.1.1.2 Instalar Git.....	96
7.3.1.1.3 Instalar Spring Tool Suite (STS).....	97
7.3.1.1.4 Instalar MySql.....	97
Linux.....	100
7.3.1.2.1 Instalar JDK.....	100
7.3.1.2.2 Instalar Git.....	100
7.3.1.2.3 Instalar Spring Tool Suite (STS).....	100
7.3.1.2.4 Instalar MySql.....	101
7.3.2 Clonar el Repositorio desde GitHub.....	102
Windows.....	102
Linux.....	103
7.3.3 Instalar la base de datos.....	103
Windows.....	103
Linux.....	104
7.3.4 Ejecutar la Aplicación.....	104
Windows.....	104
Linux.....	108
7.3.5 Acceder a la Aplicación.....	111
7.4 Despliegue en local desde Github.....	112
7.4.1 Descarga de los archivos JAR.....	112
7.4.1.1 Linux:.....	112
7.4.1.2 Windows:.....	112
7.4.2 Preparación para el despliegue.....	113
7.4.2.1 Windows:.....	114
7.4.2.1 Linux:.....	116
7.4.2.1.1 Clonar repositorio de Github para windows.....	117
7.4.2.1.2 Clonar repositorio de gitHub para linux.....	118
7.4.2.2.1 Volcar base de datos en windows.....	118
7.4.2.2.2 Volcar base de datos en linux.....	119
7.4.3 Despliegue.....	120
7.4.3.1 Despliegue en windows:.....	120

8. Pruebas.....	121
9. Conclusiones.....	123
9.1 Implementación Exitosa de Tecnologías y Herramientas.....	123
9.2 Desafíos y Soluciones.....	124
9.3 Lecciones Aprendidas.....	124
9.4 Impacto y Futuras Mejoras.....	124
10. Bibliografía.....	125
10.1 Enlaces de descarga.....	125
10.1.1 Oracle JDK.....	125
10.1.2 OpenJDK.....	125
10.1.3 Git.....	125
10.1.4 Spring Tool Suite.....	125
10.1.5 MySql.....	125
10.1.6 Lombok.....	125
10.2 Documentación Oficial.....	125
10.2.1 Spring Boot.....	125
10.2.2 Java.....	125
10.2.3 MySql.....	125
10.2.4 JavaScript.....	125
10.2.5 Axios.....	125
10.2.6 Bootstrap.....	126
10.2.7 HTML.....	126
10.2.8 CSS.....	126
10.2.9 AWS.....	126
10.2.10 Docker.....	126
10.2.11 Git.....	126
10.2.12 Lombok.....	126
10.2.13 Apache Tomcat.....	126
10.3 Otras páginas de consulta.....	126
10.3.1 Stack Overflow.....	126
10.3.2 W3Schools.....	126
10.3.3 ChatGPT.....	126
10.3.4 Microsoft designer.....	126
10.4 Páginas importantes en el desarrollo.....	126
10.4.1 Duck DNS.....	127
10.4.2 AWS Academy.....	127
10.4.3 Github.....	127
10.5 Enlaces para el despliegue en Docker.....	127
10.5.1 Tutorial para instalar docker.....	127
10.5.2 Instalar WSL.....	127
10.5.3 Wait-for-it.....	127
10.5.4 Instalar docker.....	127

1 Introducción

El documento presente es la memoria del Proyecto de Fin de Ciclo Formativo de DAW (Desarrollo Aplicaciones Web), Gymtracker es una aplicación web en la que los usuarios tienen un perfil y pueden gestionar sus rutinas de gimnasio, así como seguir un historial de las misma, para tener así conciencia sobre su progresión en el gimnasio. Se puede encontrar como repositorio público en la siguiente dirección.

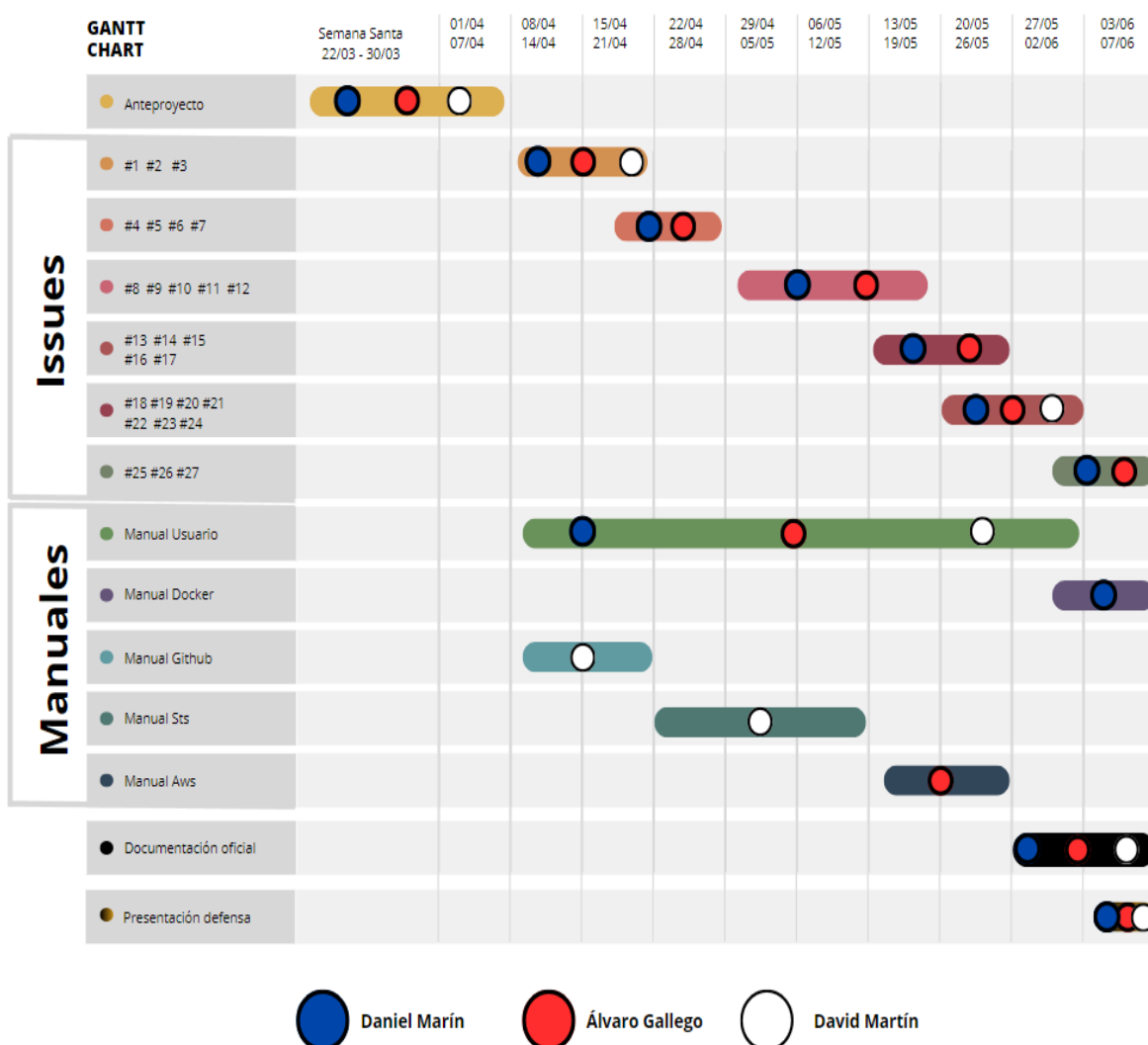
- **Repositorio Github:**

<https://github.com/deiivvv/GymTracker>

Creado por Daniel Marín Gómez, Álvaro Gallego Yáñez y David Martín Díaz.

- **Diagrama Gantt Chart(Reparto de tareas):**

https://www.canva.com/design/DAGHGkLWIGA/oS73m4O6xbt7aY4X8Xp91w/edit?utm_content=DAGHGkLWIGA&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton



1.1 Objetivos iniciales

Como finalidad, el proyecto pretende construir una página web completa, la aplicación gira entorno a la arquitectura de MVC (Modelo Vista Controlador) utilizando como eje principal el framework de Spring boot, usando una temática de gimnasio en la que cada usuario tiene su perfil y sus rutinas las cuáles puede gestionar y guardar para tener por así decirlo un diario de sus entrenamientos.

Los objetivos iniciales que se querían abarcar y superar son:

- Proporcionar a los usuarios una plataforma intuitiva y eficiente para planificar y seguir su progreso en el gimnasio.
- Consolidar conocimientos sobre el framework de Spring Boot así como explorar funcionalidades como lo son la seguridad con spring security.
- Implementar una arquitectura MVC en Gymtracker con Spring Boot, donde los datos se gestionan en servicios que interactúan con la base de datos, utilizando DTOs para mapear y transmitir los datos entre el controlador y las vistas.
- Desarrollar una API separada para gestionar ejercicios, la cual consumiría Gymtracker, ampliando así la funcionalidad del sistema y mejorando la modularidad y escalabilidad del proyecto.

1.2 Tecnologías

Los lenguajes de programación/marcas y tecnologías/entornos usados son:

1.2.1 HTML5

HTML, o "HyperText Markup Language" (Lenguaje de Marcado de Hipertexto), sirve como el esqueleto fundamental de nuestra aplicación web al definir su estructura y contenido. Utilizado principalmente para desarrollar las vistas, HTML organiza la información y elementos visuales de la página web mediante etiquetas que representan diversos elementos como texto, imágenes, enlaces y formularios, facilitando así la presentación de contenido al usuario final en el navegador web.

1.2.2 CSS3

CSS3, o "Cascading Style Sheets" (Hojas de Estilo en Cascada), complementa a HTML al proporcionar el diseño y estilo visual de una aplicación web. Utilizado para definir la apariencia de los elementos HTML, CSS3 permite personalizar aspectos como el color, la

tipografía, el espaciado y el diseño de la página, lo que mejora significativamente la presentación estética y la experiencia del usuario en el navegador web.

1.2.3 Bootstrap

Bootstrap es un marco de trabajo front-end de código abierto que facilita el desarrollo rápido y sensible de sitios web y aplicaciones web. Al aprovechar las capacidades de CSS3 y JavaScript, Bootstrap proporciona una amplia variedad de componentes y estilos predefinidos, como botones, barras de navegación, cuadros modales y diseños de rejilla responsivos. Esto permite a los desarrolladores crear interfaces de usuario atractivas y funcionales de manera eficiente, y nos sirvió como herramienta integral para la maquetación y los estilos principales de la página web.

1.2.4 JavaScript

JavaScript es un lenguaje de programación de alto nivel, interpretado y orientado a objetos, utilizado principalmente para agregar interactividad y dinamismo a las páginas web. En nuestra aplicación Gymtracker, JavaScript fue de gran utilidad para añadir dinamismo a las páginas, permitiendo a los usuarios interactuar con la interfaz de manera fluida y eficiente. Con JavaScript, pudimos manipular el contenido de la página, responder a eventos del usuario, validar formularios, realizar solicitudes a servidores web para cargar datos dinámicamente y crear efectos visuales, lo que contribuyó significativamente a mejorar la experiencia de usuario y la funcionalidad de la aplicación.

1.2.4.1 Chart.js

Chart.js es una biblioteca de JavaScript que facilita la creación de gráficos y diagramas interactivos en aplicaciones web. Proporciona una variedad de tipos de gráficos, incluyendo gráficos de barras, líneas, radiales y más, que pueden ser personalizados y animados. En Gymtracker, utilizamos Chart.js para generar diagramas de barras en la pantalla de estadísticas de la aplicación. Esto nos permite visualizar de manera clara y atractiva el progreso y los datos de los usuarios, mejorando la comprensión y el análisis de su rendimiento en el gimnasio. Gracias a Chart.js, podemos ofrecer una experiencia visualmente enriquecida que ayuda a los usuarios a interpretar sus datos de manera más efectiva.

1.2.4.2 Axios

Axios es una biblioteca de JavaScript utilizada para realizar solicitudes HTTP desde el navegador o desde Node.js. En el contexto de Gymtracker, hemos utilizado Axios para consultar la API de ejercicios externa y para realizar peticiones a controladores que responden con datos en formato JSON en lugar de plantillas HTML completas. Con Axios, pudimos realizar solicitudes HTTP de manera fácil y eficiente, lo que nos permitió obtener y enviar datos entre la aplicación y la API de ejercicios, así como entre la aplicación y los controladores del backend. Esto ha sido fundamental para obtener información dinámica y actualizada sobre ejercicios y rutinas de ejercicio, y para interactuar con la lógica del backend de manera más ágil y flexible.

1.2.5 MySQL y MySQL Workbench

Con MySQL y MySQL Workbench, creamos la base de datos de Gymtracker. MySQL es un sistema de gestión de bases de datos relacional de código abierto ampliamente utilizado, mientras que MySQL Workbench proporciona una interfaz gráfica para diseñar, modelar, administrar y consultar bases de datos MySQL de manera intuitiva. Estas herramientas nos permitieron diseñar y gestionar eficientemente la estructura de la base de datos de Gymtracker, lo que fue fundamental para almacenar y organizar la información de los usuarios, las rutinas de ejercicio y otros datos relevantes para el funcionamiento de la aplicación.

1.2.6 GitHub

GitHub es una plataforma de desarrollo colaborativo que utiliza el sistema de control de versiones Git, permitiendo a los desarrolladores trabajar juntos en proyectos de software de manera eficiente y transparente. En GitHub, los proyectos se alojan en repositorios, donde se almacena el código fuente y se registra el historial de cambios a lo largo del tiempo. Esto facilita la colaboración, ya que múltiples desarrolladores pueden trabajar simultáneamente en el mismo proyecto, realizar contribuciones, revisar y comentar el código de otros, y realizar un seguimiento de los problemas o tareas pendientes a través del sistema de seguimiento de problemas integrado.

Gracias a GitHub, podemos tener un control de versiones efectivo para el desarrollo de Gymtracker. Utilizamos un enfoque basado en ramas, donde cada tarea o issue se asocia con una rama específica. Esto nos permite trabajar de manera organizada y modular, ya que cada miembro del equipo puede trabajar en su propia rama sin interferir con el

trabajo de los demás. Una vez que se completa una tarea, se fusiona la rama correspondiente en la rama principal, lo que facilita la integración continua y garantiza un progreso constante del proyecto. Esta metodología nos brinda un control detallado sobre los cambios realizados en el código, permitiendo una colaboración fluida y eficiente entre los miembros del equipo.

1.2.7 Java

Java es el lenguaje de programación principal utilizado en Gymtracker, ya que la aplicación se desarrolla utilizando el framework Spring Boot, que está basado en Java. Java es un lenguaje de programación de propósito general ampliamente utilizado en el desarrollo de aplicaciones empresariales y web debido a su portabilidad, robustez y seguridad. Spring Boot, por su parte, proporciona un entorno de desarrollo ágil y eficiente para la creación de aplicaciones web Java, facilitando tareas como la configuración, la gestión de dependencias y la implementación de patrones de diseño. Gracias a Java y Spring Boot, podemos desarrollar Gymtracker de manera efectiva, aprovechando las características y funcionalidades que ofrecen para garantizar la fiabilidad, la escalabilidad y el rendimiento de la aplicación.

1.2.8 Spring Boot

Spring Boot es un framework de desarrollo de aplicaciones Java que ofrece un entorno simplificado y eficiente para crear aplicaciones web y servicios RESTful. En Gymtracker, hemos utilizado Spring Boot para desarrollar tanto el backend principal de la aplicación como la API RESTful de ejercicios, la cual está separada del backend principal. Para el backend de Gymtracker, Spring Boot nos proporciona las herramientas necesarias para gestionar la lógica de la aplicación, la conexión a la base de datos y la seguridad a través de Spring Security. Gracias a Spring Boot, pudimos crear una aplicación robusta y segura que cumple con los requerimientos de Gymtracker y garantiza una experiencia de usuario óptima.

1.2.8.1 Dependencias para la API de ejercicios

Las dependencias utilizadas en la API de ejercicios son las siguientes:

- **Spring Boot DevTools:** Facilita el desarrollo y la depuración al proporcionar herramientas para la recarga automática de cambios en la aplicación durante el desarrollo.
- **Spring Web:** Proporciona funcionalidades para el desarrollo de aplicaciones web, como la creación de controladores RESTful y la gestión de solicitudes HTTP.

- **JDBC API:** Ofrece una interfaz de programación para acceder a bases de datos relacionales desde aplicaciones Java, permitiendo la conexión y manipulación de datos de forma eficiente.
- **Spring Data JPA:** Simplifica el acceso y la manipulación de datos en bases de datos relacionales utilizando el framework de persistencia JPA (Java Persistence API), integrado con Spring para una configuración y uso sencillos.
- **H2 Database:** Proporciona una base de datos relacional en memoria que se puede utilizar para pruebas y desarrollo de aplicaciones, ofreciendo un entorno ligero y fácil de configurar para la gestión de datos durante el ciclo de desarrollo.

Utilizamos Spring Data JPA para simplificar los filtrados de la API, permitiendo una fácil manipulación y consulta de los datos de ejercicios establecidos. Además, empleamos H2 Database porque la API solo muestra ejercicios predefinidos y permite únicamente solicitudes GET para mostrar datos, lo que se alinea con la naturaleza de solo lectura de la API, sin permitir la creación, modificación o eliminación de ejercicios.

1.2.8.2 Dependencias para Gymtracker

Las dependencias utilizadas en la aplicación de Gymtracker son las siguientes:

- **Spring Boot DevTools:** Facilita el desarrollo y la depuración al proporcionar herramientas para la recarga automática de cambios en la aplicación durante el desarrollo.
- **Spring Web:** Proporciona funcionalidades para el desarrollo de aplicaciones web, como la creación de controladores RESTful y la gestión de solicitudes HTTP.
- **JDBC API:** Ofrece una interfaz de programación para acceder a bases de datos relacionales desde aplicaciones Java, permitiendo la conexión y manipulación de datos de forma eficiente.
- **Spring Data JPA:** Simplifica el acceso y la manipulación de datos en bases de datos relacionales utilizando el framework de persistencia JPA (Java Persistence API), integrado con Spring para una configuración y uso sencillos.
- **MySQL Driver:** Proporciona conectividad JDBC para interactuar con la base de datos MySQL.
- **Lombok:** Simplifica el desarrollo eliminando la necesidad de escribir código boilerplate en Java.
- **Spring Session:** Gestiona la sesión del usuario en aplicaciones Spring, permitiendo almacenar y compartir información de sesión entre múltiples solicitudes HTTP.
- **Spring Security:** Proporciona una capa de seguridad robusta para aplicaciones Spring, gestionando la autenticación, autorización y otras características de seguridad.
- **Thymeleaf:** Motor de plantillas para la creación de vistas HTML en aplicaciones Spring, permitiendo una integración fácil y flexible con datos del servidor.

Utilizamos Thymeleaf para la creación de plantillas HTML en Gymtracker, lo que nos permite generar vistas dinámicas e integrar fácilmente datos del servidor en la interfaz de usuario. Con Spring Data JPA, en lugar de utilizar métodos predefinidos, empleamos

nuestras propias queries personalizadas para acceder a datos específicos de la base de datos mediante la inyección de `EntityManager`. Spring Security se encarga de garantizar la seguridad de la aplicación, gestionando la autenticación, autorización y otros aspectos de seguridad para proteger los datos y los recursos de manera efectiva.

Spring Session nos ayuda a gestionar los roles de los usuarios y otras características de la sesión en la aplicación, permitiendo un control adecuado sobre los accesos y acciones de los usuarios. Lombok es empleado para simplificar la sintaxis en la creación de DTOs, reduciendo la cantidad de código boilerplate y facilitando la inyección de dependencias a través de la generación automática de constructores y otros métodos.

1.2.9 Docker

Docker es una plataforma de contenerización que permite a los desarrolladores empaquetar aplicaciones y sus dependencias en contenedores portátiles y ligeros. Estos contenedores aseguran que la aplicación se ejecute de manera consistente en diferentes entornos, desde el desarrollo hasta la producción. En Gymtracker, utilizamos Docker para el despliegue de la aplicación. Creamos contenedores a partir de los archivos JAR de Gymtracker para la aplicación principal y la API de ejercicios, y utilizamos Docker Compose para orquestar estos contenedores junto con un contenedor para la base de datos. Esto nos permite desplegar la aplicación de manera eficiente y reproducible, garantizando que todos los componentes funcionen juntos de manera coherente en cualquier entorno.

1.2.10 AWS (instancia linux amazon)

AWS (Amazon Web Services) es una plataforma de servicios en la nube que proporciona una amplia gama de servicios de infraestructura escalables y de bajo costo. En GymTracker, utilizamos una instancia de Amazon Linux en AWS para el despliegue de la aplicación. Configuramos la instancia para alojar nuestra aplicación descargando los archivos JAR necesarios y configurando la base de datos MySQL directamente en la instancia. Esto incluyó la instalación de MySQL, la creación de la base de datos y la importación de los datos necesarios. Este enfoque nos permite aprovechar la escalabilidad y la fiabilidad de la infraestructura de AWS, asegurando un rendimiento óptimo y una alta disponibilidad de la aplicación.

1.2.11 Shell

El shell es una interfaz de línea de comandos que permite a los usuarios interactuar con el sistema operativo mediante la ejecución de comandos. En Linux, Bash (Bourne Again Shell) es uno de los shells más comunes y proporciona un entorno poderoso para la creación y ejecución de scripts.

Tanto en el despliegue en Docker como en AWS, utilizamos scripts en Bash para automatizar y gestionar diversas tareas. En el despliegue con Docker, empleamos el script **wait-for-it** en el entorno Bash, que descargamos de un repositorio de GitHub. Este script asegura que el contenedor de la aplicación GymTracker espere hasta que la base de datos esté lista antes de crearse. Para implementar esto, incluimos el script en el **Dockerfile.app** y lo ejecutamos durante la ejecución de **docker-compose**. En el despliegue en AWS, creamos dos scripts en Bash: uno para levantar los archivos JAR de la aplicación y otro para detenerlos. Estos scripts nos permiten iniciar y detener la aplicación de manera eficiente, facilitando el proceso de administración y mantenimiento en la instancia de Amazon Linux.

2 Análisis de Requisitos

Página de Inicio

- **Login de Usuarios**
 - Formulario para que los usuarios existentes puedan iniciar sesión.
 - Gestión de errores para usuario o contraseña incorrecta con avisos correspondientes.
- **Registro de Nuevos Usuarios**
 - Formulario para que los nuevos usuarios se registren.
 - Validación de nombre de usuario y valores del perfil, con avisos correspondientes en caso de errores.

Menú de Opciones

- Accesible para todos los usuarios.
- Incluye todas las opciones disponibles según el rol del usuario (administrador o usuario regular).

Gestión de Rutinas/Entrenamientos

- **Creación de Rutinas/Entrenamientos**
 - Página para crear nuevas rutinas de entrenamiento.
 - Funcionalidad para guardar las rutinas creadas.
- **Mis Entrenamientos**
 - Página que muestra todas las rutinas realizadas por el usuario.
 - Permite recrear cualquier rutina anterior o comenzar una nueva desde cero.
- **Listado de Ejercicios**
 - Listado de ejercicios posibles con descripciones e imágenes para su correcta ejecución.

Estadísticas y Comparación

- **Página de Estadísticas**
 - Información detallada sobre los entrenamientos del usuario.
- **Pantalla de Clasificación**
 - Comparación de estadísticas del usuario frente a otros usuarios.

Rutinas Predeterminadas

- Página con rutinas de entrenamiento predeterminadas.
- Útil para usuarios que no desean crear una rutina desde cero o que no saben cómo comenzar.

Perfil del Usuario

- **Resumen de Perfil**
 - Página que muestra la información del perfil del usuario.
 - Permite modificar los datos del perfil en cualquier momento, respetando las mismas restricciones que durante la creación del perfil.

Administración (Solo para Administradores)

- **Gestión de Usuarios**
 - Creación y borrado de usuarios.
 - Cambios de rol y de contraseña.
 - Acceso a cuentas de usuarios.

Seguridad

- **Cierre de Sesión**
 - Opción para que los usuarios cierren su sesión de manera segura.

2.1 Mapa de pantallas

Esquema de pantallas y transiciones dentro de la aplicación

<https://www.figma.com/design/l7dGsOrB1fvV1kaDOwWcVj/Pantallas-GymTracker?node-id=0-1&t=HyYpMlcOaltamNlc-0>

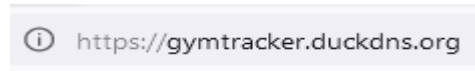
Para continuar les mostramos el manual de usuario y administrador, presentando las distintas funcionalidades del proyecto y el correcto uso de la aplicación.

2.2 Manual de usuario

2.2.1 Acceso a GymTracker

Para acceder a la aplicación web, debes ingresar la URL en tu navegador:

<https://gymtracker.duckdns.org> .



Advertencia: riesgo potencial de seguridad a continuación

Firefox ha detectado una posible amenaza de seguridad y no ha cargado **localhost**. Si visita este sitio, los atacantes podrían intentar robar información como sus contraseñas, correos electrónicos o detalles de su tarjeta de crédito.

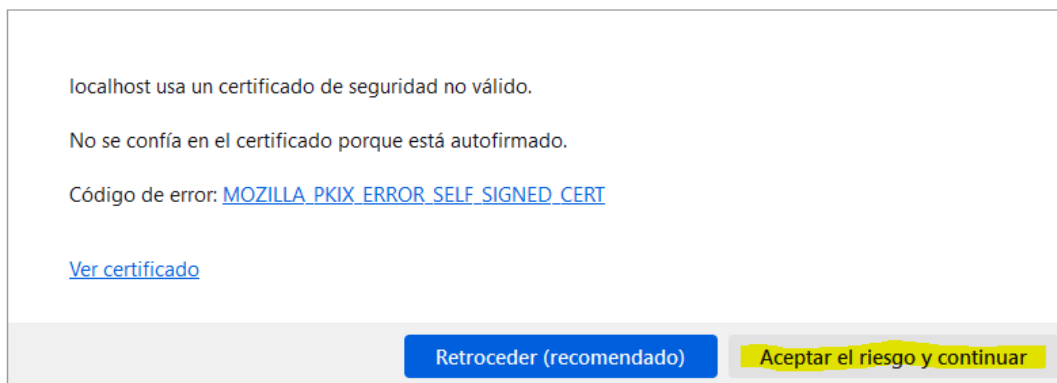
[Más información...](#)

Retroceder (recomendado)

Avanzado...

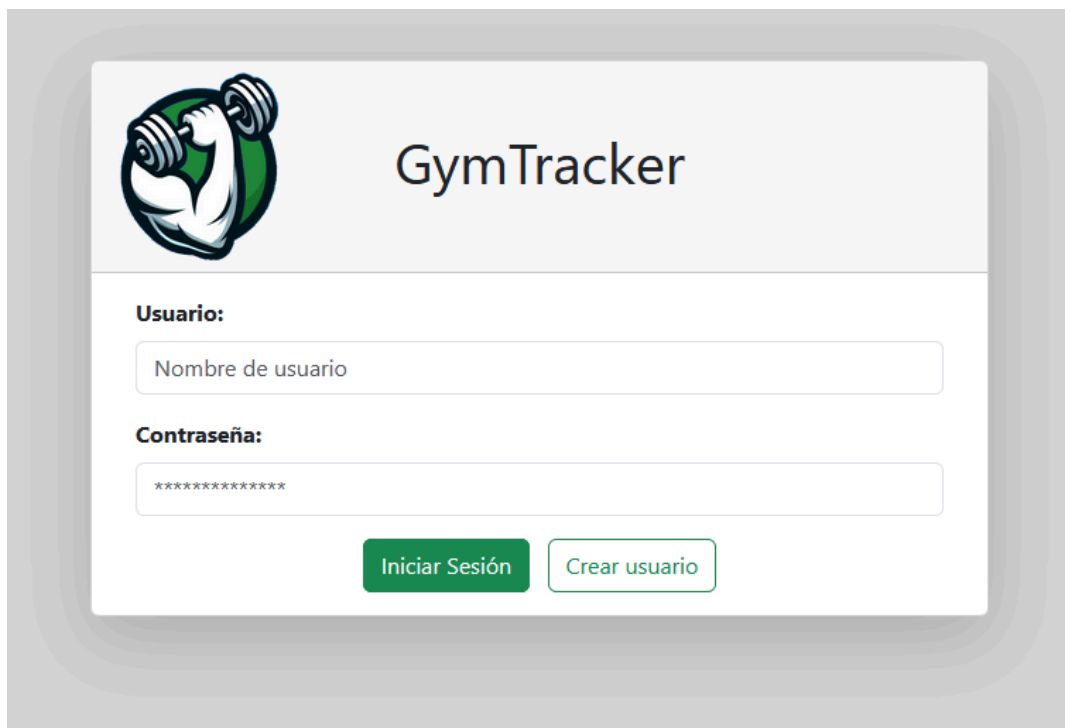
Es posible que aparezca una advertencia de seguridad debido al certificado SSL. Verifica la URL y, si es correcta, selecciona el botón "Avanzado".

Ahora, aparecerá un segundo aviso explicando el motivo. Haz clic en "Aceptar el riesgo y continuar".



2.2.2 Inicio

2.2.2.1 Inicio Sesión

The image shows the GymTracker login interface. At the top left is a logo of a muscular arm holding a green dumbbell. To its right is the text 'GymTracker'. Below the logo, there are two input fields: the first is labeled 'Usuario:' and contains the placeholder text 'Nombre de usuario'; the second is labeled 'Contraseña:' and contains a series of asterisks. At the bottom of the form are two buttons: a green button labeled 'Iniciar Sesión' and a white button with a green border labeled 'Crear usuario'.

La aplicación muestra como pantalla inicial un inicio de sesión.

Tenemos dos opciones.

- En el caso de ya tener una cuenta creada, introducimos nuestro usuario y contraseña para poder acceder y pulsamos en **“Iniciar sesión”**. Pueden darse tres casos:
 - El primer caso idílico sería que las credenciales sean correctas y en ese caso accederemos directamente al menú inicial del que hablaremos más tarde.
 - **Error de usuario y/o contraseña**, veremos un manseje amarillo como aviso indicándonos que el usuario o contraseña que hemos introducido son incorrectos. Es decir que están mal escritos o en su defecto no existen.

Usuario o contraseña incorrectos

- **Error por usuario bloqueado**, puede darse el caso de que por distintos motivos la cuenta con la que estás intentando acceder está bloqueada, es decir que tiene restringido el acceso temporalmente por un administrador. En caso de tener algún problema no dudes en ponerte en contacto a la siguiente dirección administraciongymtracker@gmail.com , alguno de los administradores atenderá tu problema con la mayor brevedad posible.

Usuario bloqueado

2.2.2.2 Crear usuario

- Si no tienes cuenta, pulsa **“Crear usuario”**
Esto nos llevará a un pequeño formulario para rellenar los datos de nuestro usuario.
Nombre y contraseña.
Junto a él un mensaje informativo acerca de la creación del perfil.



Crear Usuario

Nombre:

Contraseña:

Aceptar **Cancelar**

No olvides que no es necesario que completes tu perfil en este momento, pero recuerda actualizarlo más tarde. Pulsa enter para omitirlo.

En el caso de que el nombre usuario elegido ya esté asignado a otra persona nos encontraremos un mensaje de aviso explicando la situación. La diferencia entre mayúsculas y minúsculas no es relevante. Seguirá saltando el mensaje.

Nombre de usuario ya en uso

Sigue probando nombres hasta dar con el que más te guste.
Una vez tengas nombre de usuario pasamos a la creación del perfil. Pulsando **“Aceptar”**. Para saltarte este paso pulsa el enter en tu teclado.

2.2.2.3 Crear Perfil

Completa la información de tu perfil, ante cualquier duda o cambio el perfil es editable

The image shows two sequential screenshots of a web form titled "Crea tu perfil".

Left Screenshot:

- Header: "Crea tu perfil" with a logo of a person lifting weights.
- Field "Edad:" with a value of "0".
- Field "Género:" with the value "Sin especificar".
- Buttons: "Siguiete" (green) and "Volver" (white).
- Footer: "No olvides que no es necesario que completes tu perfil en este momento, pero recuerda actualizarlo más tarde."

Right Screenshot:

- Header: "Crea tu perfil" with the same logo.
- Field "Peso (kg):" with a value of "0.0".
- Field "Altura (cm):" with a value of "0.0".
- Buttons: "Crear" (green) and "Volver" (white).
- Footer: "No olvides que no es necesario que completes tu perfil en este momento, pero recuerda actualizarlo más tarde."

Cualquier campo relleno con información inválida no será aceptado. Se mostrará un mensaje de error.

Ante cualquier cambio se podrá retroceder en el formulario pulsando **"Volver"**.

Una vez rellenado el formulario de usuario y perfil. Pulsaremos **"Crear"**.

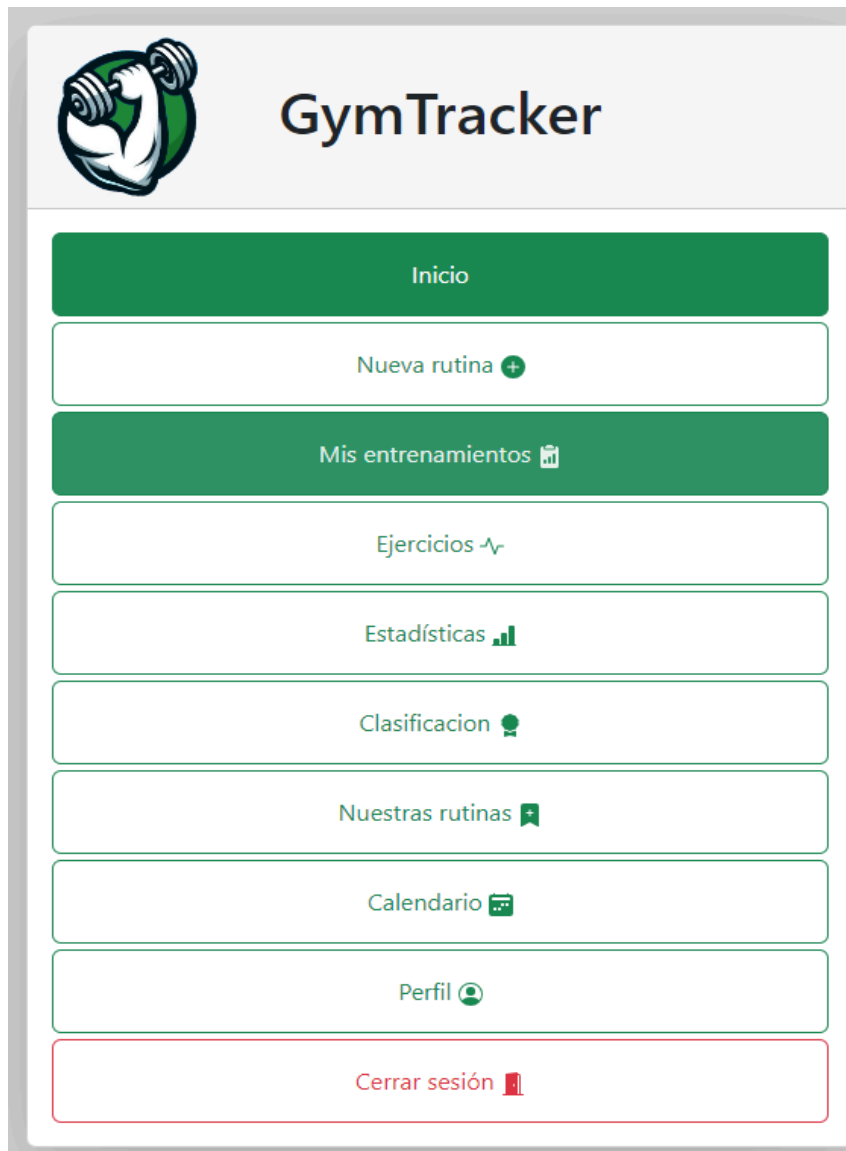
Volveremos a la pantalla inicial **"Inicio de Sesión"**, ahora puedes acceder con el nuevo usuario.

2.2.3 Menú

Una vez hemos accedido a la aplicación. Nos encontraremos con el menú inicial el cual nos indica todas las posibles acciones que podemos realizar desde la aplicación y permite navegar entre ellas. Siempre remarcado en verde la opción en la que te encuentras actualmente.

Siendo estas:

- Inicio
- Nueva rutina
- Mis entrenamientos
- Ejercicios
- Estadísticas
- Clasificación
- Nuestras rutinas
- Calendario
- Perfil
- Cerrar sesión



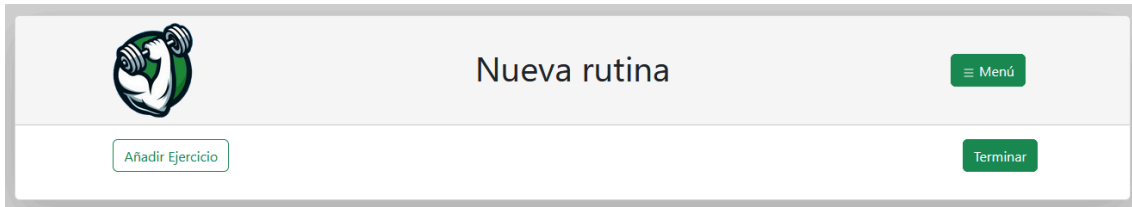
En el caso de tener alguna opción más, ponerse en contacto con la administración al correo indicado previamente (administraciongymtracker@gmail.com) para que le sean retirados los permisos. Pedimos la mayor seriedad posible en cuanto a este tema.

- **Nueva rutina**, te permitirá el acceso a la creación de una nueva rutina propia desde cero.
- **Mis entrenamientos**, mostrará un listado de todos tus entrenamientos que hayas realizado con GymTracker.
- **Ejercicios**, muestra el listado de los ejercicios de los que dispone la aplicación e información básica acerca de cada uno de ellos.
- **Estadísticas**, muestra un resumen de tus estadísticas dentro de la aplicación
- **Clasificación**, muestra una tabla con las estadísticas de todos los usuarios con acceso a la aplicación.
- **Nuestras rutinas**, un pequeño listado de rutinas predeterminadas creadas por nuestros entrenadores para un mejor entrenamiento.
- **Calendario**, un calendario para poder llevar el seguimiento de tus entrenamientos.
- **Perfil**, muestra la información de tu perfil.

- **Cerrar sesión**, cerrar y eliminar tu sesión actual, redirigiéndote a la pantalla inicial de Inicio Sesión.

2.2.4 Nueva rutina

Al acceder a la nueva rutina encontramos una pantalla prácticamente vacía para rellenarla con el entrenamiento que queremos crear.



Dispone de tres botones. “Menú”, “Añadir Ejercicio” y “Terminar”.

- **Menú:** Abrirá el menú sidebar de la aplicación
- **Añadir Ejercicio:** Muestra el conjunto de los ejercicios posibles que puedes introducir a tu rutina. Junto a varios filtros por nombre, músculo que trabaja o equipamiento necesario.
- **Terminar:** Permite finalizar la rutina.

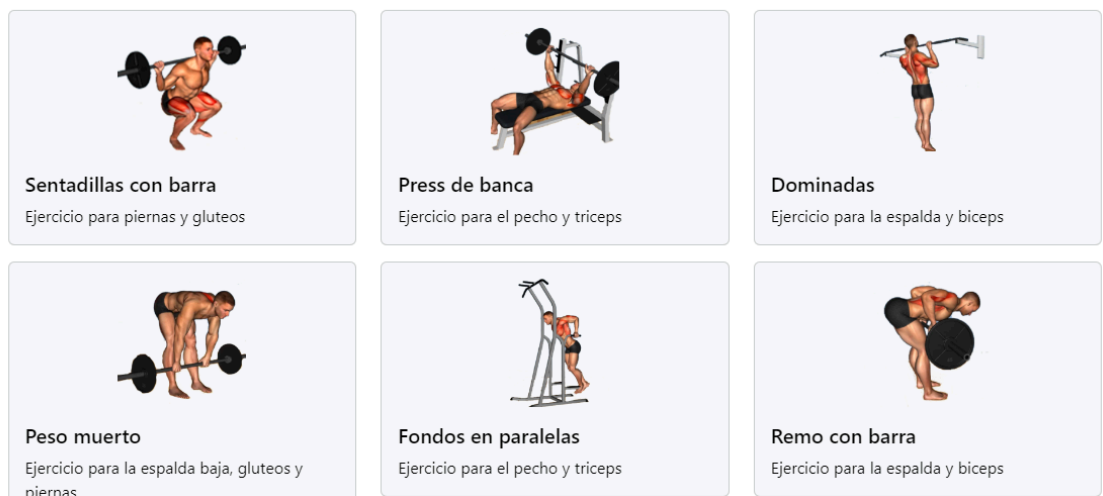
2.2.4.1 Añadir ejercicio

Muestra el listado de ejercicios de la web

Buscar Ejercicios

Cancelar

Listado de ejercicios



Al seleccionar cualquiera de los ejercicios listados este será añadido a nuestra rutina.

Peso(kg)	Repeticiones	Acciones

Puedes añadir tantos ejercicios como quieras e incluso varios iguales, estos solo se guardarán como uno mismo.

2.2.4.2 Añadir Serie

Junto a nuestro ejercicio se encuentran dos botones. Uno verde con un + y otro rojo con el dibujo de una papelera.

El botón verde sirve para crear una **nueva serie**.

Abrirá: una pequeña ventana para rellenar las información acerca de la serie.
Una vez relleno pulsar **“Aceptar”** para añadir la serie.
En el caso de querer cancelar la acción pulsar **“Cancelar”**.

Puedes añadir tantas series como quieras.

 Sentadillas  		
Peso(kg)	Repeticiones	Acciones
20	12	 

Junto a la serie se encuentran dos botones. Uno verde-blanco con el dibujo de un lápiz y otro rojo-blanco con el dibujo de una papelera.

El lápiz permite **modificar los campos de la serie** ya creada.

La papelera **eliminará la serie**.

El botón rojo **eliminará el ejercicio** al completo de la rutina.

- **Terminar**

Al pulsar el botón de “Terminar”, abrirá una ventana para asignarle un nombre a nuestra rutina, este paso es obligatorio.

Crear rutina

Dame el nombre de tu rutina para crearla

Cancelar
Crear

Con dos botones. “Cancelar” y “Crear”

- **Cancelar:** Cerrará la ventana y te permitirá seguir editando tu rutina
- **Crear:** Intentará crear la rutina. En el caso de no introducir nombre o con algún problema en la rutina (rutinas vacías...) Mostrará un mensaje de aviso explicando el problema.

El nombre de la rutina es obligatorio

Debes completar la rutina antes de terminar

Una vez creada la rutina, le redirigirá a la pantalla de “Mis entrenamientos”, entre las cuales ahora se encuentra la rutina que acaba de crear.

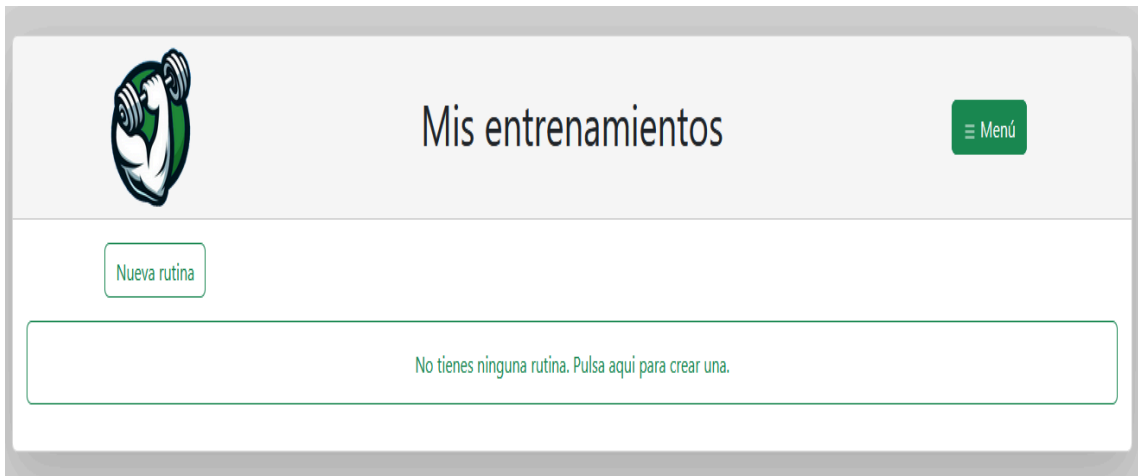
2.2.5 Mis entrenamientos

Muestra un listado de sus entrenamientos dentro de la aplicación

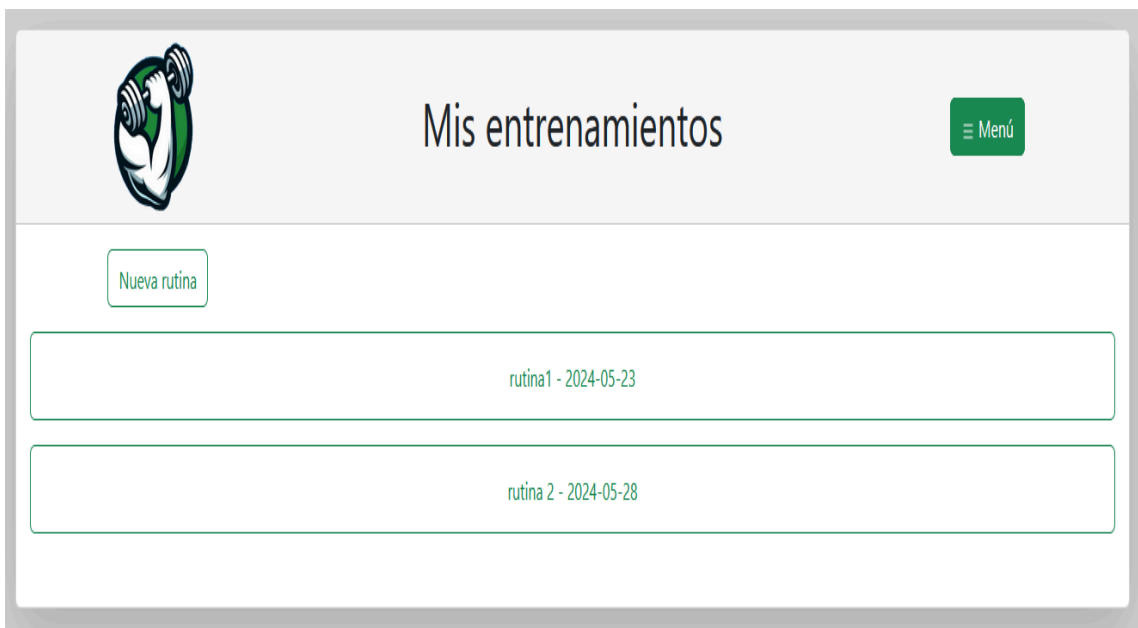
Dispone de dos botones: “Menú” y “Nueva rutina”.

- **Menú:** Abrirá el menú sidebar de la aplicación
- **Nueva rutina:** Redirige a “Nueva rutina” para crear una nueva rutina.

En el caso de no tener entrenamientos registrados en la web. Se mostrará un mensaje que le permitirá crear su primera rutina.



En caso de si tener rutinas registradas se listará en orden de creación ascendente, indicando su nombre y la fecha en la que se creó cada rutina.



Puede clicar encima de cada una de las rutinas para volver a realizarla y se le auto completarán los ejercicios y series automáticamente según los realizados en esa rutina. Puedes repetirla tantas veces como quieras.

Añadir Ejercicio

Terminar



Sentadillas con barra

+

✖

Peso(kg)	Repeticiones	Acciones
20	20	✎ ✖
40	1	✎ ✖



Press de banca

+

✖

Peso(kg)	Repeticiones	Acciones
50	20	✎ ✖
100	1	✎ ✖

2.2.6 Ejercicios

Dispone del botón: “Menú”.

- **Menú:** Abrirá el menú sidebar de la aplicación

Muestra el conjunto de los ejercicios posibles que puedes introducir a tu rutina. Junto a varios filtros por nombre, músculo que trabaja o equipamiento necesario.



2.2.7 Estadísticas

La aplicación cuenta con un botón denominado **“Menú”** al hacer clic en este botón, se abrirá el menú lateral (sidebar) de la aplicación.

La pantalla principal muestra un resumen de tus estadísticas, incluyendo:

- Número total de entrenamientos realizados.

Entrenos
1

- Número de series completadas.

Series
4

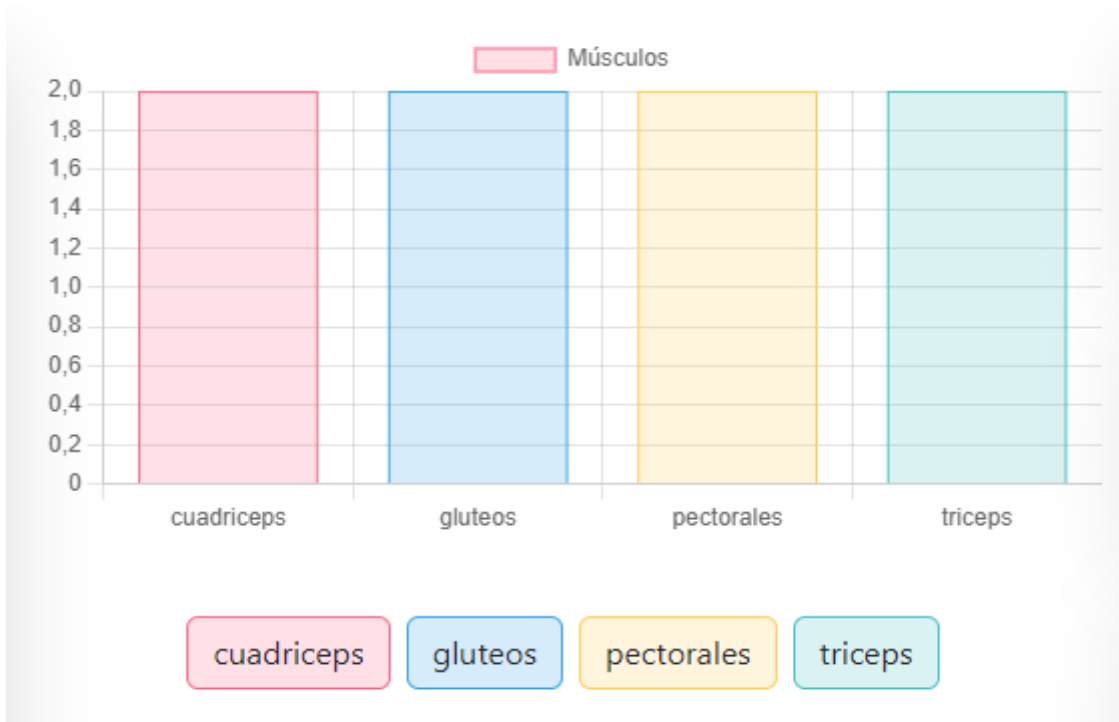
- Volumen total de peso levantado.

Volumen (kg)
1540.0

- Cantidad de ejercicios distintos y ejercicio favorito (que será el que hayas realizado mayor número de veces)



Además, se presenta una gráfica de barras que muestra la cantidad de ejercicios realizados por cada grupo muscular. Puedes interactuar con los distintos botones para ocultar o mostrar cada campo de la gráfica, o incluso todos los campos simultáneamente.



Pantalla completa:



2.2.8 Clasificación

Dispone de dos botones “Menú”, yo.

- **Menú:** Abrirá el menú sidebar de la aplicación
- **Yo:** Encuéntrate en la tabla.

Muestra una tabla paginada con las estadísticas de todos los usuarios con acceso a la aplicación.

Encima de la tabla se encuentra un desplegable que permite ordenarla según la estadística que prefiera.

A su izquierda se encuentra el botón YO, al pulsarlo la tabla se posicionará automáticamente en la ubicación de tu perfil.

Clasificación de usuarios

Entrenos

Nombre	Entrenos	Ejercicios	Series	Volumen (kg)
damago	11	7	14	1684.0
alvaro	0	0	0	0.0
kaka	0	0	0	0.0
w	0	0	0	0.0
david	0	0	0	0.0

1 2 »

2.2.9 Nuestras rutinas

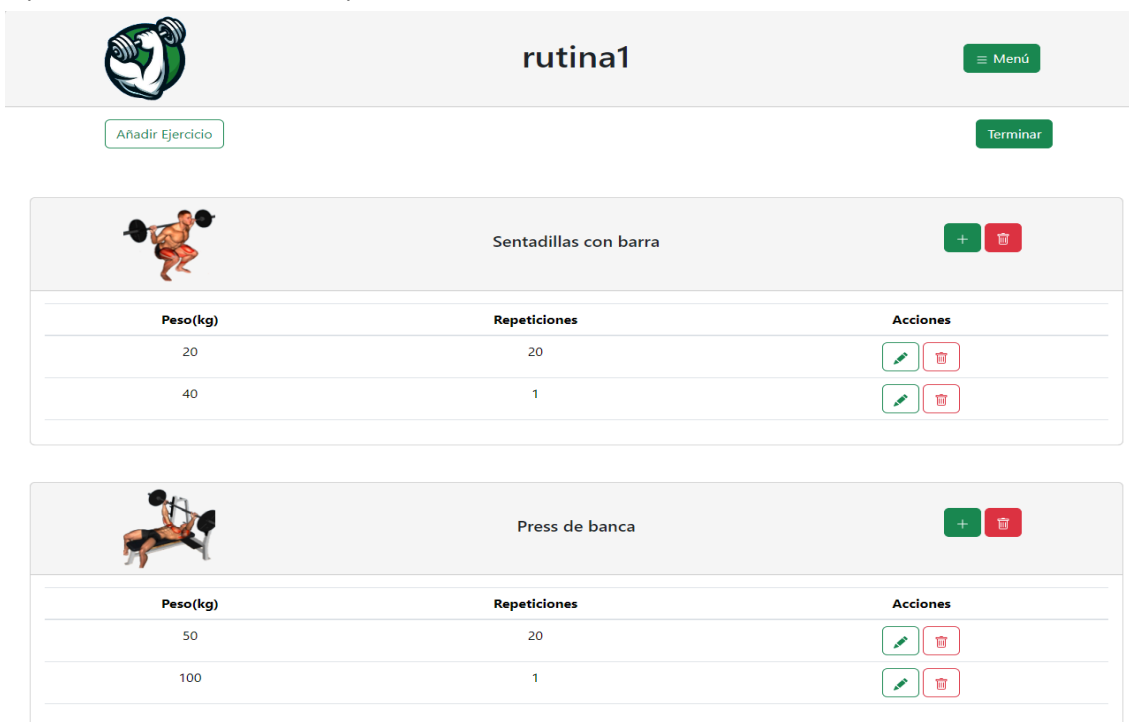
Dispone de dos botones: “Menú” y “Nueva rutina”.

- **Menú:** Abrirá el menú sidebar de la aplicación
- **Nueva rutina:** Redirige a “Nueva rutina” para crear una nueva rutina.

Un pequeño listado de rutinas predeterminadas creadas por nuestros entrenadores para un mejor entrenamiento.



Puede clicar encima de cada una de las rutinas para volver a realizarla y se le auto completarán los ejercicios y series automáticamente según los realizados en esa rutina. Puedes repetirla tantas veces como quieras.



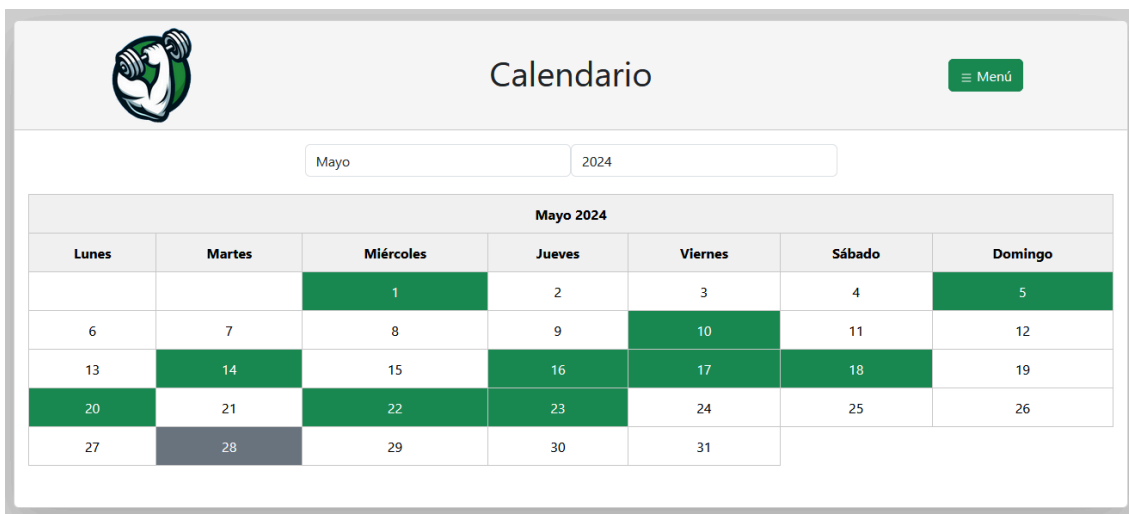
2.2.10 Calendario

Dispone del botón: “Menú”.

- **Menú:** Abrirá el menú sidebar de la aplicación

Tiene acceso a un calendario para poder llevar el seguimiento de tus entrenamientos. mostrandose en verde los días en los que se ha registrado alguna rutina y en gris la fecha actual.

En la parte superior se encuentran el **mes y año** del calendario. Puede navegar tanto como quiera con él.



2.2.11 Perfil

Dispone de tres botones: “Menú”, “Editar perfil” y “Cambiar contraseña”.

- **Menú:** Abrirá el menú sidebar de la aplicación.
- **Editar perfil:** Permite que modifique los campos de su perfil.
- **Cambiar contraseña:** Permite cambiar tu contraseña.

Muestra la información de tu perfil.

Nombre:
damago

Edad:
25

Género:
Masculino

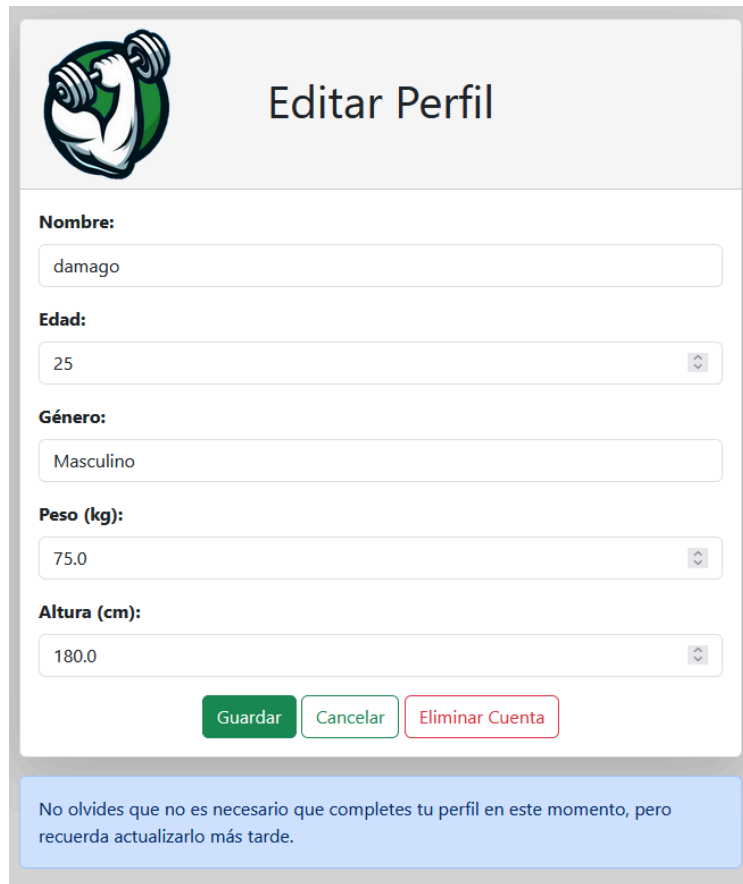
Peso (kg):
75.0

Altura (cm):
180.0

Editar Perfil **Cambiar contraseña**

2.2.11.1 Editar perfil

Habilita los campos para poder modificar la información de su perfil.



Nombre:
damago

Edad:
25

Género:
Masculino

Peso (kg):
75.0

Altura (cm):
180.0

Guardar **Cancelar** **Eliminar Cuenta**

No olvides que no es necesario que completes tu perfil en este momento, pero recuerda actualizarlo más tarde.

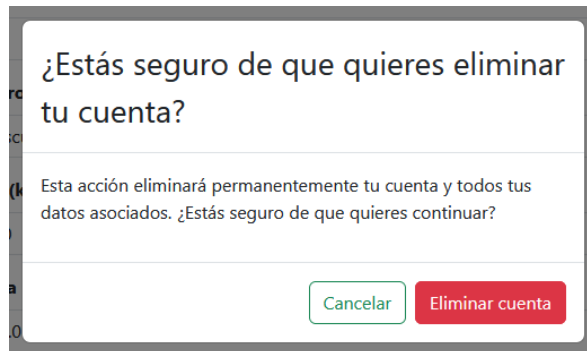
Muestra tres botones “Guardar”, “Cancelar”, “Eliminar Cuenta”, Y un mensaje informativo.

- **Guardar:** Actualiza la información introducida en el perfil. En el caso de encontrar cualquier problema se mostrará un mensaje de aviso explicando la situación. Sigue las mismas restricciones que en la creación de usuario y perfil. Una vez actualizado, volverá al resumen del perfil.

Nombre de usuario ya en uso

- **Cancelar:** No guardar los cambios introducidos y volver al resumen del perfil
- **Eliminar Cuenta:** Muestra una ventana de confirmación con dos botones, “Cancelar” y “Eliminar cuenta”
Si pulsa **Cancelar** volverá a la edición del perfil

Si pulsa **Eliminar cuenta** se borrará el usuario, perfil y toda la información



¿Estás seguro de que quieres eliminar tu cuenta?

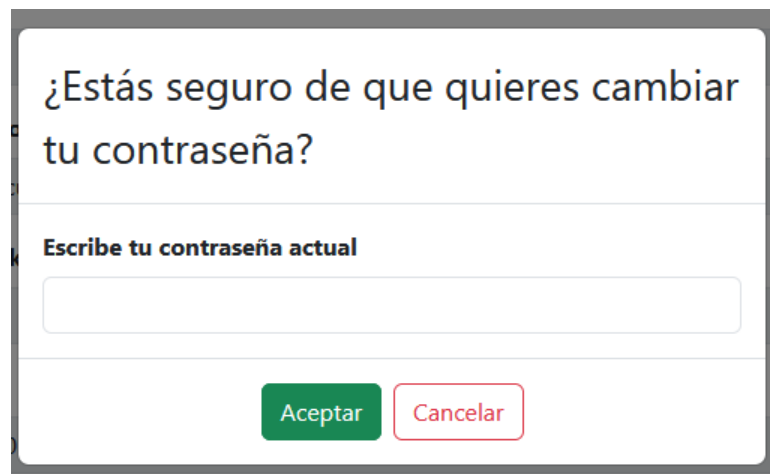
Esta acción eliminará permanentemente tu cuenta y todos tus datos asociados. ¿Estás seguro de que quieres continuar?

relacionada con el usuario. Se redirigirá a la pantalla de inicio “Inicio Sesión”.

2.2.11.2 Cambiar contraseña

Permite cambiar su contraseña.

- Se mostrará una ventana en la que deberá introducir su contraseña actual.
- Dispondrá de dos botones “Aceptar” y “Cancelar”.



¿Estás seguro de que quieres cambiar tu contraseña?

Escribe tu contraseña actual

- **Aceptar:** Comprobará si la contraseña introducida es correcta.
 - Si lo es se cerrará la ventana y se abrirá una nueva para introducir la nueva contraseña.
 - Dispondrá de dos botones “Aceptar” y “Cancelar”
 - **Aceptar** se mantendrá deshabilitado hasta que la contraseña y la repetición de la contraseña sean iguales. Si clicas en él se cambiará su contraseña. Seguirá en el usuario y en resumen del perfil.
 - **Cancelar**, cerrará la ventana y se mantendrá en el resumen del perfil.

Si no es correcta se mostrará un mensaje de aviso.

- **Cancelar:** Cerrará la ventana y se mantendrá en el resumen del perfil.

Crea tu nueva contraseña

Nueva contraseña

Repite la nueva contraseña

Aceptar

Cancelar

2.2.12 Menú Sidebar

Indica todas las posibles acciones que podemos realizar desde la aplicación y permite navegar entre ellas. Siempre remarcado en verde la opción en la que te encuentras actualmente.

Siendo estás:

- Inicio
- Nueva rutina
- Mis entrenamientos
- Ejercicios
- Nuestras rutinas
- Calendario
- Perfil
- Cerrar sesión

En el caso de tener alguna opción más, ponerse en contacto con la administración al correo indicado previamente (administraciongymtracker@gmail.com) para que le sean retirados los permisos. Pedimos la

mayor seriedad posible en cuanto a este tema.



GymTracker



Inicio

Nueva rutina

Mis entrenamientos

Ejercicios

Estadísticas

Clasificación

Nuestras rutinas

Calendario

Perfil

Cerrar sesión

2.2.13 Error

Si se ha encontrado una pantalla de error

Dispondrá de un botón arriba a la derecha “Inicio”, el cual le llevará a la pantalla principal, Menú.

Dentro de esta pantalla podrá recopilar información acerca de los sucedido.

Arriba en el centro encontrará el mensaje de error y a su izquierda el código al que pertenece este error.

Posibles errores

404 -> la página solicitada por el usuario no se encontró en el servidor.

400 -> el servidor no puede procesar o reconocer la solicitud.

500 -> el servidor encuentra una condición inesperada que le impide cumplir con la solicitud.

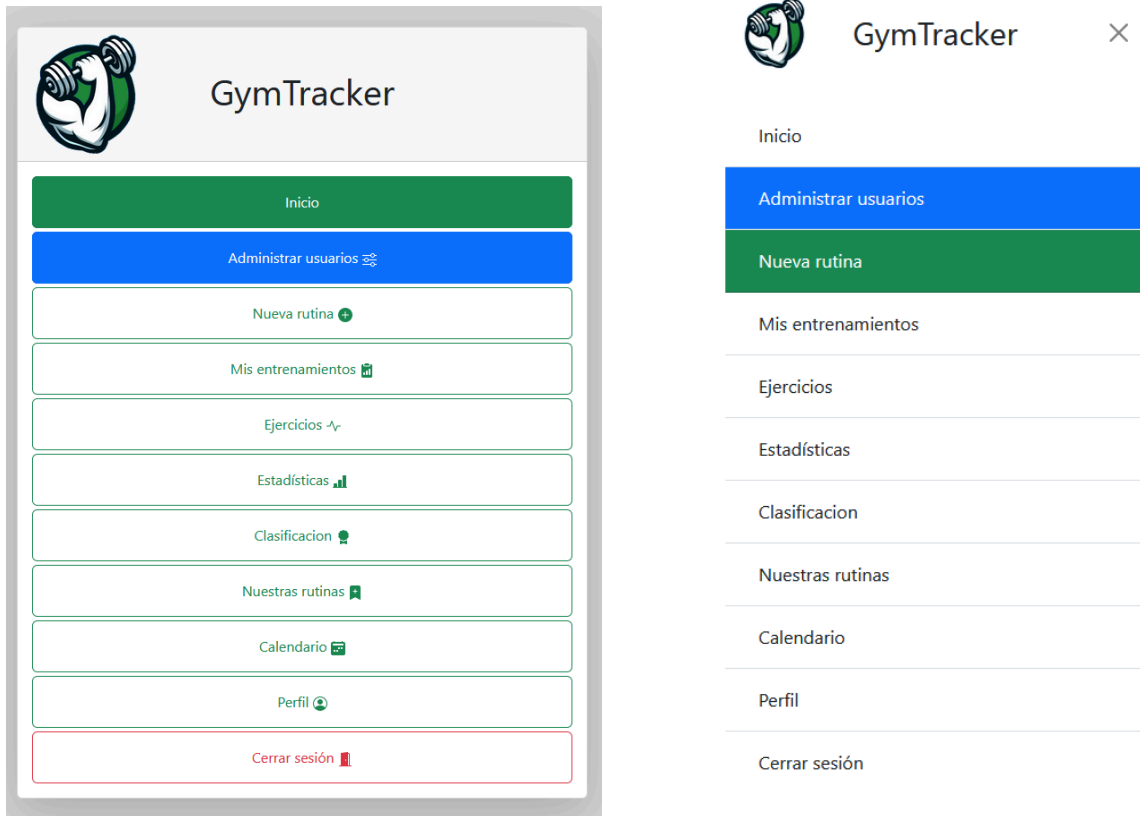
Si el problema persiste no dude en ponerse en contacto con nosotros.



2.3 Manual de administrador

En el caso de los administradores la gran mayoría de funcionalidades serán las mismas que un usuario normal salvo excepciones.

El menú inicial y sidebar de los administradores mostrará un campo más en color azul “Administrar usuarios”.



2.3.1 Administrar Usuarios

Dispone de dos botones: “Menú”, “Crear usuario”..

- **Menú:** Abrirá el menú sidebar de la aplicación.
- **Crear usuarios:** Permite la creación de nuevos usuarios.

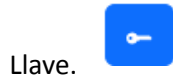
Id	Usuario	Rol	Acciones
-1	usuario_rutinas	bloqueado	[iconos]
1	damago	usuario	[iconos]
2	alvaro	administrador	[iconos]
7	4	bloqueado	[iconos]
8	q	usuario	[iconos]

Muestra una tabla paginada de todos los usuarios de la aplicación

La tabla puede ser filtrada por tipo de rol de los usuarios “Administrador”, “Usuario”, “Bloqueado” o “Todos los usuarios”.

Cada fila de la tabla muestra un usuario y junto con él su id, nombre de usuario, rol y tres botones de acción: Uno azul con una llave, uno verde con una puerta u otro rojo con una papelera.

2.3.1.1 Cambiar contraseña del usuario



- Permite cambiar la contraseña del usuario sin necesidad de saber la que tenía previamente
 - Dispondrá de dos botones “Aceptar” y “Cancelar”
 - **Aceptar** se mantendrá deshabilitado hasta que la contraseña y la repetición de la contraseña sean iguales. Si clicas en él se cambiará su contraseña.
 - **Cancelar**, cerrará la ventana, sin cambios

2.3.1.2 Acceder a la cuenta del usuario



- Dará acceso a la cuenta del usuario seleccionado

2.3.1.3 Eliminar usuario



- Muestra una ventana de confirmación con dos botones, “Cancelar” y “Eliminar cuenta”
 - Dispondrá de dos botones “Cancelar ” y “Eliminar cuenta”
 - Si pulsa **Cancelar** volverá a la edición del perfil
 - Si pulsa **Eliminar cuenta** se borrará el usuario, perfil y toda la información relacionada con el usuario.

¿Estás seguro de que quieres eliminar tu cuenta?

Esta acción eliminará permanentemente tu cuenta y todos tus datos asociados. ¿Estás seguro de que quieres continuar?

Cancelar Eliminar cuenta

En el campo rol de cada usuario al pulsarlo se le abrirá una ventana con los posibles roles que se le pueden asignar al usuario.

Rol

usuario

Cambiar rol

administrador usuario bloqueado

Volver

De forma adicional puede encontrarse símbolo gris de una persona, indicativo del perfil en que está el administrador en este momento.



1	damago	administrador		
---	--------	---------------	--	--

2.3.1.4 Crear usuario

- Permite crear un usuario siguiendo las mismas condiciones que la creación de usuario en la pantalla inicial. Con la diferencia que los administradores pueden elegir el rol que tendrá el usuario a crear.
- Dispondrá de dos botones “Aceptar” y “Cancelar”

Crear Usuario

Nombre:

Nombre de usuario

Contraseña:

Rol usuario:

Usuario

Cancelar Aceptar

Nombre de usuario ya en uso

- **Cancelar**, cerrará la ventana, el usuario no será creado.

3. Adecuación del entorno de trabajo

Las instalaciones necesarias para llevar a cabo el desarrollo de GymTracker son mínimas, principalmente el IDE de trabajo (STS en Linux e IntelliJ en Windows), MySQL y la clonación del repositorio remoto de GitHub.

A continuación se detallan las configuraciones necesarias para ambos entornos:

3.1 Instalación de MySQL

- Instalar MySQL Server tanto en linux, como en windows según las instrucciones proporcionadas en la documentación oficial de MySQL: [MySQL Community Downloads](#).
- O en Linux simplemente con los comando:
 - **sudo apt update**
 - **sudo apt install mysql-server**

3.2 Instalación de Git

- En linux:
 - **sudo apt update** (actualizar paquetes)
 - **sudo apt install git** (instalar git)
 - **git --version** (verificar instalación)
- En windows:
 - Descarga la última versión del instalador de Git desde el sitio oficial: [Git for Windows](#).
 - Sigue las instrucciones del instalador. Durante la instalación, puedes elegir las opciones predeterminadas o personalizar la instalación según tus preferencias.
 - Verificar la instalación: **git --version**

3.3 Clonación del Repositorio

- Clonar el repositorio de GymTracker desde GitHub:
 - **git clone https://github.com/deiivvv/GymTracker.git**

3.4 Instalación de JDK

Para el desarrollo de GymTracker, necesitaremos una JDK de Java versión 17 o superior. A continuación se detallan los pasos para instalar JDK en Linux y Windows.

3.4.1 Instalación de JDK en Linux

- Actualizar el índice de paquetes:
 - **sudo apt update**
- Instalar OpenJDK 17:
 - **sudo apt install openjdk-17-jdk**
- Verificar la instalación:
 - **java -version**

Deberías ver una salida que indique que Java 17 está instalado.

3.4.2 Instalación de JDK en Windows

- Descargar el JDK de Oracle: [Oracle JDK 17 Downloads](#)
- Ejecutar el instalador: sigue las instrucciones del instalador para instalar el JDK.

4.4.2.1 Configurar la variable de entorno JAVA_HOME:

- Abre el Panel de Control y ve a "**Sistema y Seguridad**" > "**Sistema**" > "**Configuración avanzada del sistema**".
- En la pestaña "**Avanzado**", haz clic en "**Variables de entorno**".
- En "**Variables del sistema**", haz clic en "**Nueva**" e introduce:
 - Nombre de la variable: **JAVA_HOME**
 - Valor de la variable: **Ruta al directorio de instalación del JDK** (por ejemplo, C:\Program Files\Java\jdk-17)
- Busca la variable Path en "**Variables del sistema**" y haz clic en "**Editar**". Añade una nueva entrada con la ruta **%JAVA_HOME%\bin**.
- Verificar la instalación:
 - **java -version**

3.5 Instalación de Maven

3.5.1 Instalación de Maven en Linux

- Actualizar el índice de paquetes:
 - **sudo apt update**
- Instalar Maven:
 - **sudo apt install maven**
- Verificar la instalación:
 - **mvn -version**

Deberías ver una salida que indique la versión de Maven instalada.

3.5.2 Instalación de Maven en Windows

- Descarga la última versión binaria zip de Maven desde el sitio oficial: Apache Maven Downloads.

- Extrae el contenido del archivo zip descargado en una ubicación de preferencia (por ejemplo, C:\Program Files\Maven).

3.5.2.1 Configurar la variable de entorno MAVEN_HOME

- Abre el Panel de Control y ve a "**Sistema y Seguridad**" > "**Sistema**" > "**Configuración avanzada del sistema**".
- En la pestaña "**Avanzado**", haz clic en "**Variables de entorno**".
- En "**Variables del sistema**", haz clic en "**Nueva**" e introduce:
 - Nombre de la variable: **MAVEN_HOME**
 - Valor de la variable: **Ruta al directorio de Maven** (por ejemplo, C:\Program Files\Maven\apache-maven-3.8.6).
- Busca la variable Path en "**Variables del sistema**" y haz clic en "**Editar**". Añade una nueva entrada con la ruta **%MAVEN_HOME%\bin**.
- Verificar la instalación:
 - Abre una nueva ventana de Command Prompt (CMD) y ejecuta:
 - **cmd**
 - **mvn -version**

Deberías ver una salida que indique la versión de Maven instalada.

3.6 Configuración del IDE (STS - Spring Tool Suite)

3.6.1 Instalación de Spring Tool Suite

- Descargar e instalar Spring Tool Suite (STS) desde el sitio oficial: [Spring Tool Suite](#).
- Descomprimir el archivo descargado en una ubicación de preferencia en tu sistema.

3.6.2 Configuración de Lombok

- Descargar el archivo JAR de Lombok desde su sitio oficial: [Project Lombok](#).
- Ejecutar el archivo JAR descargado: `java -jar lombok.jar`.
- Se abrirá un instalador, sigue las instrucciones para integrar Lombok con Spring Tool Suite.

3.6.3 Importar Proyectos con STS

Para importar los proyectos GymTracker-AppWeb y Api-Ejercicios en Spring Tool Suite (STS), sigue estos pasos:

1. Abre Spring Tool Suite.
2. Ve a **"File"** (Archivo) en la barra de menú.
3. Selecciona **"Import"** (Importar) en el menú desplegable.
4. En el cuadro de diálogo de importación, selecciona **"Maven"** > **"Existing Maven Projects"** (Proyectos Maven Existentes).
5. Haz clic en **"Next"** (Siguiente).
6. En el campo **"Root Directory"** (Directorio Raíz), navega hasta la carpeta clonada del repositorio de Git donde se encuentra el proyecto GymTracker.
7. Dentro de la carpeta clonada, busca y selecciona la carpeta **"GymTracker-AppWeb"** y luego repite el proceso para **"Api-Ejercicios"**.
8. Haz clic en **"Finish"** (Finalizar) para importar el proyecto en Spring Tool Suite.

3.7 Configuración del IDE (IntelliJ IDEA)

3.7.1 Instalación de IntelliJ IDEA

- Ve al sitio oficial de JetBrains y descarga IntelliJ IDEA: [JetBrains IntelliJ IDEA](#).

3.7.2 Importar Proyectos con IntelliJ IDEA

Para importar los proyectos GymTracker-AppWeb y Api-Ejercicios en IntelliJ IDEA, sigue estos pasos:

1. Abre IntelliJ IDEA.
2. En la pantalla de bienvenida, selecciona **"Open"** (Abrir).
3. Navega hasta la carpeta clonada del repositorio de Git donde se encuentra el proyecto GymTracker.
4. Selecciona la carpeta **"GymTracker-AppWeb"** y haz clic en **"OK"** para abrir el proyecto.
5. Espera a que IntelliJ IDEA indexe el proyecto y descargue las dependencias necesarias.
6. Para importar otro proyecto, repite los pasos:
 - Ve a **"File"** (Archivo) en la barra de menú.
 - Selecciona **"New"** > **"Project from Existing Sources..."** (Nuevo > Proyecto desde fuentes existentes).
 - Navega hasta la carpeta clonada del repositorio de Git y selecciona la carpeta **"Api-Ejercicios"**.
 - Haz clic en **"OK"** para abrir el proyecto.

7. IntelliJ IDEA te pedirá que configures el proyecto como un proyecto Maven. Acepta las configuraciones predeterminadas y haz clic en "Next" (Siguiente) y luego en "Finish" (Finalizar).

4. Arquitectura del proyecto

El proyecto GymTracker se divide en tres partes principales para asegurar un mayor control y aislamiento de cada componente, lo que facilita su gestión, mantenimiento y evolución. Estas partes son:

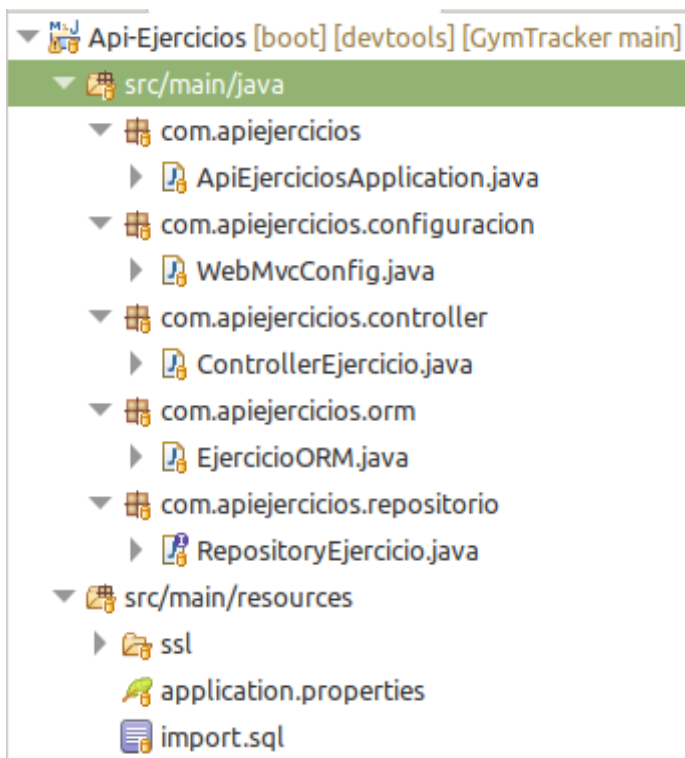
- **Base de datos**
- **Api de ejercicios**
- **App de GymTracker**

Esta arquitectura modular no solo simplifica el desarrollo y la implementación de nuevas funcionalidades, sino que también promueve una mayor reutilización del código y una mejor organización del proyecto.

A continuación la explicación de la arquitectura de la api de ejercicios y la app de GymTracker:

4.1 Arquitectura de la API

Nuestra API de ejercicios está organizada de manera estructurada para facilitar su desarrollo y mantenimiento. A continuación, explicamos la arquitectura y los componentes principales de nuestro proyecto:



4.1.1 Estructura y Funcionalidad

1. **ApiEjerciciosApplication.java:**
 - Es el punto de entrada de nuestra aplicación Spring Boot. Aquí se configura y se inicia la aplicación.
2. **Configuración (configuracion/WebMvcConfig.java):**
 - En este paquete y archivo configuramos las políticas de acceso a la API. Aquí determinamos quién puede acceder a los distintos endpoints, configuramos CORS (Cross-Origin Resource Sharing) y otras configuraciones de seguridad y acceso.
3. **Controlador (controller/ControllerEjercicio.java):**
 - Este controlador se encarga de manejar las solicitudes HTTP. Define los endpoints de la API y utiliza el repositorio para consultar y devolver los datos. En nuestro caso, solo permitimos solicitudes GET para mantener la API en un estado de solo lectura.
4. **ORM (orm/EjercicioORM.java):**
 - Utilizamos Spring Data JPA para definir nuestras entidades de base de datos con anotaciones como @Id y @Entity. Esta clase representa la estructura de la tabla de ejercicios en la base de datos H2 y facilita la manipulación y consulta de los datos.
5. **Repositorio (repositorio/RepositoryEjercicio.java):**
 - Aquí definimos el repositorio que interactúa con la base de datos H2. Spring Data JPA nos simplifica la consulta y filtrado de datos predefinidos de ejercicios. Este repositorio es utilizado por el controlador para obtener los datos necesarios para las respuestas de la API.

4.1.2 Recursos

1. **application.properties:**
 - Contiene las configuraciones de la aplicación, incluyendo detalles de la conexión a la base de datos H2, configuraciones de servidor y otros parámetros importantes.

2. **import.sql:**
 - Este archivo se utiliza para importar datos predefinidos a la base de datos H2 al inicio de la aplicación. Contiene los ejercicios que la API mostrará.
3. **SSL (ssl/certificate.pem, certificate-request.csr, keystore.p12, private-key.pem):**
 - Aquí almacenamos los certificados SSL y las llaves privadas necesarias para asegurar nuestra API. Estos archivos aseguran que las comunicaciones con la API sean seguras y encriptadas con https.

4.1.3 Tecnologías Utilizadas

- **Spring Data JPA:** Simplifica la manipulación y consulta de datos en la base de datos.
- **H2 Database:** Utilizamos esta base de datos en memoria porque nuestra API solo necesita mostrar datos predefinidos y no permite modificaciones.
- **Spring Boot:** Facilita la creación de aplicaciones stand-alone, de producción listas para ejecutarse.
- **Spring Boot DevTools:** Facilita el desarrollo y la depuración al proporcionar herramientas para la recarga automática de cambios en la aplicación durante el desarrollo.
- **Spring Web:** Proporciona funcionalidades para el desarrollo de aplicaciones web, como la creación de controladores RESTful y la gestión de solicitudes HTTP.

En resumen, hemos diseñado esta API para que sea fácil de mantener y de escalar, con una arquitectura clara y tecnologías que aseguran un rendimiento óptimo y seguridad en el acceso a los datos.

4.2 Arquitectura de la APP

Nuestra aplicación GymTracker está diseñada siguiendo una arquitectura MVC (Modelo-Vista-Controlador) para asegurar una organización clara y una separación de responsabilidades eficiente. A continuación, detallamos la estructura y los componentes principales de nuestro proyecto:



4.2.1. Configuración

En la carpeta **Configuracion**, gestionamos la configuración de la aplicación. Incluye archivos como:

Configuración para poder pillar la conexión a la base de datos con variables de entorno como se hizo para Docker:

- **AppConfig.java**: Configuración general de la aplicación.
- **DataSourceConfig.java**: Configuración de la fuente de datos para la base de datos.

Configuración de seguridad:

- **SecurityConfig.java**: Configuración de seguridad utilizando Spring Security.

4.2.2. Controladores (Controllers)

Los controladores en la carpeta **Controller** gestionan las solicitudes HTTP y dirigen las respuestas adecuadas. Algunos de los controladores son:

- **AdminController.java**: Gestión de la interfaz y funcionalidades del administrador.
- **CalendarioController.java**: Gestión del calendario de entrenamientos.
- **EjerciciosController.java**: Gestión de los ejercicios disponibles.
- **EstadisticasControler.java**: Gestión de las estadísticas de los usuarios.
- **LoginController.java**: Gestión del inicio de sesión de los usuarios.

4.2.3. Data Transfer Objects (DTO)

En la carpeta **Dto**, definimos objetos que se utilizan para transferir datos entre la capa de servicio y los controladores. Algunos ejemplos son:

- **EstadisticasUsuarioDTO.java**
- **MisEntrenamientosDTO.java**
- **UsuarioDTO.java**

4.2.4. Servicios (Services)

La carpeta **Service** contiene la lógica de negocio de la aplicación. Aquí es donde interactuamos con la base de datos a través de nuestros servicios. Algunos servicios incluyen:

- **CalendarioService.java**
- **ClasificacionService.java**
- **UsuarioService.java**

4.2.5. Recursos (Resources)

La carpeta **resources** contiene archivos de configuración, recursos estáticos y plantillas Thymeleaf. Incluye:

- **application.properties:** Configuración general de la aplicación.
- **static:** Archivos CSS, imágenes y JavaScript.
- **templates:** Plantillas HTML utilizadas por Thymeleaf.
- **ssl (ssl/certificate.pem, certificate-request.csr, keystore.p12, private-key.pem):**
 - Aquí almacenamos los certificados SSL y las llaves privadas necesarias para asegurar nuestra app. Estos archivos aseguran que las comunicaciones con la app sean seguras y encriptadas con https.

4.2.6 Dependencias

Utilizamos varias dependencias para desarrollar la aplicación:

- **Spring Boot DevTools:** Facilita el desarrollo y la depuración.
- **Spring Web:** Proporciona funcionalidades para desarrollar aplicaciones web.
- **JDBC API:** Permite el acceso a bases de datos relacionales.
- **Spring Data JPA:** Simplifica el acceso y manipulación de datos.
- **MySQL Driver:** Proporciona conectividad JDBC para MySQL.
- **Lombok:** Simplifica el desarrollo eliminando código boilerplate.
- **Spring Session:** Gestiona las sesiones de usuario.
- **Spring Security:** Proporciona una capa de seguridad robusta.
- **Thymeleaf:** Motor de plantillas para generar vistas HTML dinámicas.

4.2.7 Mapeo con ORM

En el archivo **orm.xml**, se realiza el mapeo de los datos recuperados desde la base de datos para formar objetos del tipo de DTO en los servicios. Este archivo configura cómo los datos deben ser transformados y utilizados dentro de la aplicación, asegurando que los datos se estructuren correctamente al ser transferidos entre la base de datos y los objetos Java.

4.2.8 Flujo de Datos

En nuestra arquitectura MVC, los datos fluyen desde la base de datos a través de los servicios, donde se mapean a DTOs (Data Transfer Objects). Los controladores reciben estos DTOs y los pasan a las vistas. Utilizamos Thymeleaf para integrar fácilmente los datos del servidor en la interfaz de usuario.

Además, los controladores también responden a peticiones Axios para recuperar datos concretos y hacer peticiones a la API de ejercicios, permitiendo una comunicación y una experiencia de usuario más dinámica.

4.2.9 Gestión de Seguridad y Sesiones

- **Spring Security:** Gestiona la autenticación y autorización de usuarios, protegiendo los datos y recursos.
- **Spring Session:** Permite gestionar los roles de usuarios y otras características de la sesión, asegurando un control adecuado sobre los accesos y acciones de los usuarios.

4.2.10 Uso de Lombok

Lombok nos ayuda a reducir el código repetitivo y simplificar la sintaxis en la creación de DTOs. Esto facilita la inyección de dependencias y la generación automática de constructores y otros métodos.

4.2.11 Conclusión

Nuestra aplicación GymTracker está cuidadosamente estructurada para facilitar el desarrollo y el mantenimiento, asegurando una separación clara de responsabilidades y una interacción fluida entre las diferentes capas de la aplicación. Utilizamos una combinación de tecnologías y dependencias robustas para proporcionar una experiencia de usuario segura y eficiente.

5. Desarrollo de la base de datos

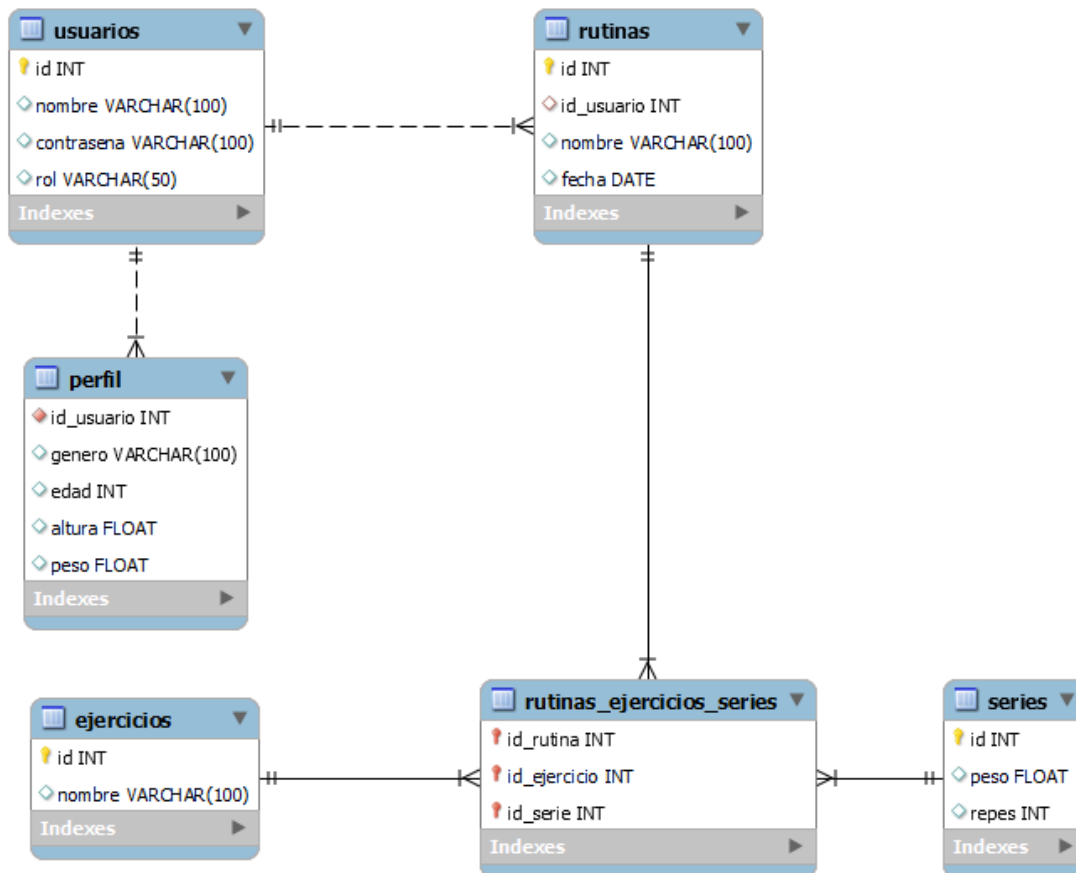
En este punto se documenta todo sobre la base de datos de GymTracker, la cuál se compone de las siguientes 6 tablas: *usuarios*, *perfil*, *rutinas*, *rutinas_ejercicios_series*, *ejercicios*, *series*.

```
mysql> use gymtracker;
Database changed
mysql> show tables;
+-----+
| Tables_in_gymtracker |
+-----+
| ejercicios            |
| perfil                |
| rutinas                |
| rutinas_ejercicios_series |
| series                |
| usuarios              |
+-----+
6 rows in set (0.01 sec)
```

A continuación se explica la base de datos más a fondo:

5.1 Arquitectura de la base de datos

5.1.1 Esquema de la base de datos



5.1.2 Tablas Principales

- **Usuarios:** Almacena información sobre los usuarios del sistema, como su nombre, contraseña y rol.
- **Perfil:** Contiene detalles adicionales sobre los usuarios, como género, edad, altura y peso. La clave externa se relaciona con la tabla “Usuarios”.
- **Rutinas:** Registra las rutinas de entrenamiento, incluyendo el nombre, la fecha y la clave externa que se relaciona con la tabla “Usuarios”.
- **Ejercicios:** Almacena los diferentes ejercicios disponibles.
- **Series:** Guarda información sobre las series de ejercicios, como el peso y el número de repeticiones.

5.1.3 Relación entre Rutinas, Ejercicios y Series

- La relación entre las tablas “Rutinas”, “Ejercicios” y “Series” se establece a través de la tabla intermedia “rutinas_ejercicios_series”. Esta tabla actúa como un puente entre las rutinas, los ejercicios y las series.

- Cada fila en “**rutinas_ejercicios_series**” representa una combinación específica de rutina, ejercicio y serie.

5.1.4 Relaciones

- **Usuarios** → **Perfil**: Relación 1:1 (un usuario tiene un perfil).
- **Usuarios** → **Rutinas**: Relación 1:m (un usuario puede tener varias rutinas).
- **Rutinas** → **Ejercicios**: Relación m:m (una rutina puede incluir varios ejercicios y un ejercicio puede estar en varias rutinas).
- **Rutinas** → **Series**: Relación m:m (una rutina puede tener múltiples series y una serie puede estar en varias rutinas).

5.1.5 Claves Externas y Borrados en Cascada

- **Clave Externa en “Rutinas” (id_usuario)**: Esta clave se relaciona con la tabla “Usuarios”. Si se elimina un usuario, todas sus rutinas asociadas también se eliminarán automáticamente (borrado en cascada).
- **Clave Externa en “Rutinas_ejercicios_series” (id_rutina)**: Se relaciona con la tabla “Rutinas”. Si se elimina una rutina, todas las filas asociadas en “rutinas_ejercicios_series” también se eliminarán (borrado en cascada).
- **Clave Externa en “Rutinas_ejercicios_series” (id_ejercicio)**: Se relaciona con la tabla “Ejercicios”. Si se elimina un ejercicio, las filas correspondientes en “rutinas_ejercicios_series” también se eliminarán (borrado en cascada).
- **Clave Externa en “Rutinas_ejercicios_series” (id_serie)**: Se relaciona con la tabla “Series”. Si se elimina una serie, las filas asociadas en “rutinas_ejercicios_series” también se eliminarán (borrado en cascada).

En resumen, la base de datos permite gestionar usuarios, sus perfiles, las rutinas de entrenamiento que crean, los ejercicios asociados a alguna rutina y las series asociadas a cada ejercicio en una rutina específica. Las relaciones entre las tablas están diseñadas para mantener la integridad referencial y facilitar la gestión de datos.

5.2 Explicación y justificación de la base de datos

La elección y diseño de la base de datos para GymTracker se basó en el objetivo principal de proporcionar a cada usuario su perfil personalizado y la capacidad de gestionar sus rutinas de entrenamiento, junto con consideraciones clave sobre integridad, escalabilidad y eficiencia en consultas.

En esta sección, profundizaremos en la justificación de la elección de esta estructura de base de datos para el sistema GymTracker. Aquí hay algunas consideraciones importantes:

5.2.1 Integridad de los Datos

- La relación entre usuarios, rutinas, ejercicios y series se establece de manera coherente mediante claves externas. Esto garantiza que los datos estén bien estructurados y que no haya inconsistencias.
- El borrado en cascada asegura que, al eliminar un usuario, todas las entidades relacionadas se eliminen automáticamente sin dejar registros huérfanos.

5.2.2 Escalabilidad

- El diseño m:m entre rutinas y ejercicios permite flexibilidad. Los usuarios pueden crear rutinas personalizadas con cualquier combinación de ejercicios.
- La tabla intermedia “rutinas_ejercicios_series” facilita la expansión futura. Si se agregan más ejercicios o se modifican las series, la estructura sigue siendo válida.

5.2.3 Eficiencia en Consultas

- Las relaciones 1:m entre usuarios y rutinas, así como entre rutinas y ejercicios, permiten consultas eficientes. Por ejemplo, buscar todas las rutinas de un usuario específico es rápido.
- Las claves externas optimizan las búsquedas y las uniones entre tablas.

5.2.4 Consistencia y Mantenibilidad

- La estructura bien definida facilita el mantenimiento y la corrección de errores.
- Las relaciones claras ayudan a los desarrolladores a comprender la lógica de la base de datos.

En resumen, la base de datos “GymTracker” se diseñó cuidadosamente para satisfacer las necesidades de los usuarios, garantizar la integridad de los datos y permitir futuras expansiones

6. Desarrollo del proyecto

En este apartado se detalla el proceso de construcción de nuestra aplicación GymTracker. Comenzamos por establecer la base de datos y desarrollar la API de ejercicios. La idea central consiste en que los ejercicios se almacenen en la API, mientras que en la base de datos solo se guardan aquellos que pertenecen a una rutina específica dentro de GymTracker. A continuación, se describe el desarrollo de la API de ejercicios y la aplicación de GymTracker:

6.1 Desarrollo de la API

El desarrollo de la API se ha diseñado para tener los ejercicios de manera separada, lo que permite actualizarlos fácilmente en el futuro por nosotros, los desarrolladores. Si se desean añadir más ejercicios o modificar los existentes, podemos hacerlo directamente desde el archivo import.sql de la API. Además, la API está pensada únicamente para listar los ejercicios, es decir, realizar peticiones GET filtradas según el nombre del ejercicio, el músculo que trabaja y el equipamiento utilizado.

Hemos optado por utilizar una base de datos H2, ya que no requiere persistencia. Esto es ideal para nuestra implementación, dado que los datos se modificarán directamente en el archivo import.sql por nosotros, los desarrolladores, en lugar de en la base de datos misma. A continuación una breve explicación de las partes de API:

6.1.1 Configuración CORS para el acceso a la API

```
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**") // Permitir acceso a todos los endpoints
            .allowedOrigins("https://localhost",
                           "http://localhost:8080",
                           "http://gymtracker.duckdns.org:8080",
                           "http://gymtracker.duckdns.org",
                           "https://gymtracker.duckdns.org") // Permitir solicitudes desde estos orígenes
            .allowedMethods("GET", "POST", "PUT", "DELETE") // Permitir estos métodos HTTP
            .allowCredentials(true) // Permitir credenciales (si se están usando)
            .maxAge(3600); // Tiempo de vida del preflight request en segundos
    }
}
```

WebMvcConfig es una configuración específica para Spring Framework en una aplicación web. Explicación:

1. **@Configuration:** Esta anotación marca la clase como una configuración de Spring. Indica que contiene definiciones de beans o configuraciones personalizadas.
2. **WebMvcConfigurer:** Esta interfaz proporciona métodos para personalizar la configuración de Spring MVC (Model-View-Controller). En este caso, estamos implementando esta interfaz para personalizar la configuración CORS (Cross-Origin Resource Sharing).
3. **addCorsMappings(CorsRegistry registry):** Este método se llama durante la inicialización de la aplicación y configura las políticas CORS. Aquí están los detalles:
 - **registry.addMapping("/**"):** Define un mapeo global para todos los endpoints de la API.
 - **.allowedOrigins(...):** Especifica los orígenes permitidos para las solicitudes CORS. En este caso, se permiten solicitudes desde varios dominios (localhost y gymtracker.duckdns.org).
 - **.allowedMethods(...):** Define los métodos HTTP permitidos para las solicitudes CORS (GET, POST, PUT, DELETE).
 - **.allowCredentials(true):** Permite el envío de credenciales (como cookies o encabezados de autenticación) en las solicitudes CORS.

- **.maxAge(3600):** Establece el tiempo de vida (en segundos) del preflight request (una solicitud de verificación previa) para evitar solicitudes CORS frecuentes.

En resumen este archivo nos sirve para configurar quién tiene acceso a nuestra API.

6.1.2 Mapeo de la Entidad Ejercicio y Creación de la Tabla en H2

En esta sección, explicaremos cómo se realiza el mapeo de un ejercicio utilizando ORM (Object-Relational Mapping) y cómo se crea la tabla correspondiente en una base de datos H2.

La clase EjercicioORM es una entidad que representa un ejercicio en el contexto de un ORM utilizando Jakarta Persistence (JPA). Esta clase define los atributos y comportamientos de un ejercicio y los mapea a una tabla en la base de datos.

A continuación, se explica cada parte de la clase:

```
package com.apiejercicios.orm;

import java.util.Objects;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

- **Paquete e importaciones:** La clase está en el paquete **com.apiejercicios.orm**. Se importan las clases necesarias para definir una entidad JPA y para la generación automática de valores para el identificador (ID).

```
@Entity
public class EjercicioORM {
```

- **@Entity:** Esta anotación indica que la clase es una entidad JPA. Cada instancia de esta clase se corresponderá con una fila en una tabla de la base de datos.

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String nombre;
private String musculo;
private String descripcion;
private String equipamiento;

```

- **Atributos:**
 - **@Id:** Marca el campo **id** como la clave primaria de la entidad.
 - **@GeneratedValue(strategy = GenerationType.IDENTITY):** Indica que el valor del **id** será generado automáticamente por la base de datos usando una estrategia de identidad.
 - Los otros atributos (**nombre, musculo, descripcion, equipamiento**) representan las columnas de la tabla y las propiedades del ejercicio.
- **Getters y Setters:** Métodos públicos para acceder y modificar los valores de los atributos privados.
- **Métodos hashCode y equals:** Definen la igualdad entre dos instancias de **EjercicioORM**. Dos instancias son iguales si tienen el mismo **id**.

6.1.3 Repositorio para la Entidad Ejercicio

En esta sección, explicaremos la implementación del repositorio para la entidad **EjercicioORM** utilizando Spring Data JPA. El repositorio es una interfaz que facilita la interacción con la base de datos, proporcionando métodos para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) y consultas personalizadas.

La interfaz **RepositoryEjercicio** extiende **CrudRepository**, lo que le proporciona métodos básicos de CRUD. Además, define métodos personalizados para buscar ejercicios basados en varios criterios. Recordemos que la API está pensada únicamente para listar los ejercicios, es decir, realizar peticiones GET filtradas según el nombre del ejercicio, el músculo que trabaja y el equipamiento utilizado. Por esta razón, el repositorio define varios métodos específicos para estos filtros.

A continuación, se describe cada método del repositorio:

```
package com.apiejercicios.repositorio;

import java.util.List;

import com.apiejercicios.orm.EjercicioORM;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;

public interface RepositoryEjercicio extends CrudRepository<EjercicioORM, Long> {
```

- **Paquete e importaciones:** La interfaz está en el paquete **com.apiejercicios.repositorio**. Se importan las clases necesarias para manejar la entidad **EjercicioORM** y para definir el repositorio.

```
List<EjercicioORM> findByNombreContaining(
    @Param("nombre") String nombre);
```

- **findByNombreContaining:** Este método busca ejercicios cuyo nombre contenga la cadena proporcionada. Utiliza la anotación **@Param** para vincular el parámetro **nombre**. Este método es útil para listar ejercicios filtrados por el nombre del ejercicio, alineándose con la funcionalidad de la API de filtrar los ejercicios por nombre.

```
List<EjercicioORM> findByMusculoContaining(
    @Param("musculo") String musculo);
```

- **findByMusculoContaining:** Este método busca ejercicios cuyo músculo contenga la cadena proporcionada. Este método permite listar ejercicios filtrados por el músculo que trabaja, cumpliendo con la funcionalidad de la API de proporcionar filtros basados en el músculo trabajado.

```
List<EjercicioORM> findByEquipamientoContaining(
    @Param("equipamiento") String equipamiento);
```

- **findByEquipamientoContaining:** Este método busca ejercicios cuyo equipamiento contenga la cadena proporcionada. Este método facilita la lista de ejercicios filtrados por el equipamiento utilizado, otra funcionalidad clave de la API.

```
List<EjercicioORM> findByMusculoContainingAndEquipamientoContaining(
    @Param("musculo") String musculo,
    @Param("equipamiento") String equipamiento);
```

- **findByMusculoContainingAndEquipamientoContaining:** Este método busca ejercicios que coincidan con ambas cadenas proporcionadas: músculo y equipamiento. Permite realizar filtrados combinados según músculo y equipamiento, ampliando las posibilidades de filtrado de la API.

```
List<EjercicioORM> findByNombreContainingAndMusculoContaining(
    @Param("nombre") String nombre,
    @Param("musculo") String musculo);
```

- **findByNombreContainingAndMusculoContaining:** Este método busca ejercicios que coincidan con ambas cadenas proporcionadas: nombre y músculo. Ayuda a listar ejercicios filtrados por nombre y músculo trabajado, lo que permite una búsqueda más precisa en la API.

```
List<EjercicioORM> findByNombreContainingAndEquipamientoContaining(
    @Param("nombre") String nombre,
    @Param("equipamiento") String equipamiento);
```

- **findByNombreContainingAndEquipamientoContaining:** Este método busca ejercicios que coincidan con ambas cadenas proporcionadas: nombre y equipamiento. Facilita la búsqueda de ejercicios filtrados por nombre y equipamiento utilizado, proporcionando mayor flexibilidad en los filtros de la API.

```
List<EjercicioORM> findByNombreContainingAndEquipamientoContainingAndMusculoContaining(
    @Param("nombre") String nombre,
    @Param("equipamiento") String equipamiento,
    @Param("musculo") String musculo);
```

- **findByNombreContainingAndEquipamientoContainingAndMusculoContaining:** Este método busca ejercicios que coincidan con todas las cadenas proporcionadas: nombre, equipamiento y músculo. Permite listar ejercicios filtrados por todos los criterios a la vez: nombre, equipamiento y músculo trabajado, ofreciendo una búsqueda muy específica y detallada en la API.

Estas consultas personalizadas permiten filtrar los ejercicios de manera flexible según los criterios especificados, aprovechando el poder de Spring Data JPA para generar las consultas necesarias a partir de los nombres de los métodos. Esto facilita la implementación y el mantenimiento del código, asegurando que las consultas sean intuitivas y fáciles de entender, y se alineen perfectamente con el objetivo de la API de listar ejercicios filtrados según diversos parámetros(nombre,músculo y equipamiento)

6.1.4 Controlador para la Gestión de Ejercicios

En esta sección, presentaremos el controlador **ControllerEjercicio**, encargado de gestionar las peticiones relacionadas con los ejercicios en la API. Este controlador utiliza los métodos del repositorio **RepositoryEjercicio** para realizar operaciones de búsqueda de ejercicios basadas en diferentes criterios.

El objetivo de este controlador es permitir la búsqueda de ejercicios filtrando por nombre, músculo y equipamiento en todas las combinaciones posibles, es decir, pasando todos los parámetros, alguno de ellos o ninguno. Esto se alinea con la funcionalidad de la API de proporcionar una lista filtrada de ejercicios según diversos criterios de búsqueda.

Explicación del código:

```
@RestController
public class ControllerEjercicio {

    @Autowired
    private RepositoryEjercicio ejerciciosRepository;

    @GetMapping("/ejercicios/buscar")
    public ResponseEntity<Iterable<EjercicioORM>> buscarEjercicios(@RequestParam(required = false) String nombre,
        @RequestParam(required = false) String musculo, @RequestParam(required = false) String equipamiento) {

        if (nombre != null) {
            if (musculo != null && equipamiento != null) {
                return findByNombreAndEquipamientoAndMusculo(nombre, equipamiento, musculo);
            } else if (musculo != null) {
                return findByNombreAndMusculo(nombre, musculo);
            } else if (equipamiento != null) {
                return findByNombreAndEquipamiento(nombre, equipamiento);
            } else {
                return findByNombre(nombre);
            }
        } else if (musculo != null) {
            if (equipamiento != null) {
                return findByMusculoAndEquipamiento(musculo, equipamiento);
            } else {
                return findByMusculo(musculo);
            }
        } else if (equipamiento != null) {
            return findByEquipamiento(equipamiento);
        } else {
            return null;
        }
    }
}
```

Este endpoint **/ejercicios/buscar** permite buscar ejercicios filtrando por nombre, músculo y equipamiento en todas las combinaciones posibles. Los parámetros de búsqueda son opcionales (**@RequestParam(required = false)**) para permitir consultas flexibles.

Cada método del controlador, como **findByName**, **findByMusculo** y **findByEquipamiento**, realiza un filtrado específico utilizando los métodos correspondientes del repositorio **RepositoryEjercicio**. Esto garantiza una búsqueda flexible y personalizable de ejercicios según los criterios proporcionados, permitiendo que la API responda a una variedad de necesidades de búsqueda de los usuarios de manera eficiente y precisa.

6.1.5 Creación de la Base de Datos H2 con Ejercicios mediante el archivo **import.sql**

El archivo **import.sql** es fundamental para la inicialización de la base de datos H2 cuando se levanta la API. Este archivo contiene una serie de sentencias SQL que insertan datos en la tabla **ejercicioORM**, creando así una base de datos poblada con ejercicios.

El propósito de este archivo es proporcionar una forma rápida y sencilla de tener datos de ejemplo en la base de datos H2 al iniciar la aplicación. Esto es útil para propósitos de desarrollo y pruebas, ya que asegura que la API tenga datos disponibles para ser consultados y manipulados.

Las sentencias SQL en el archivo **import.sql** consisten en inserciones de datos que representan ejercicios con sus respectivos nombres, músculos trabajados, descripciones y equipamientos utilizados. Cada inserción utiliza la sintaxis **INSERT INTO** seguida del nombre de la tabla (**ejercicioORM**) y los valores de las columnas (**nombre, musculo, descripcion, equipamiento**) correspondientes a cada ejercicio.

A continuación se muestra un fragmento del contenido del archivo **import.sql**:

```
INSERT INTO ejercicioORM (nombre, musculo, descripcion, equipamiento) VALUES ('sentadillas con barra', 'cuadriceps, gluteos', 'Ejercicio para piernas y gluteos', 'pesas');
INSERT INTO ejercicioORM (nombre, musculo, descripcion, equipamiento) VALUES ('press de banca', 'pectorales, triceps', 'Ejercicio para el pecho y triceps', 'pesas');
INSERT INTO ejercicioORM (nombre, musculo, descripcion, equipamiento) VALUES ('dominadas', 'espalda, biceps', 'Ejercicio para la espalda y biceps', 'peso corporal');
-- Más sentencias de inserción...
```

Cada una de estas sentencias **INSERT INTO** añade un nuevo ejercicio a la base de datos con su nombre, músculos trabajados, descripción y equipamiento utilizado.

El archivo **import.sql** garantiza que al iniciar la API, la base de datos H2 está configurada con ejercicios disponibles.

6.2 Desarrollo de la APP

El desarrollo de la aplicación GymTracker se ha llevado a cabo siguiendo una arquitectura MVC (Modelo-Vista-Controlador) meticulosamente diseñada. Esta estructura proporciona una organización clara y una separación de responsabilidades eficiente, lo que facilita tanto el desarrollo inicial como el mantenimiento continuo del proyecto. En esta sección, explicaremos en detalle el proceso de desarrollo de la aplicación, las decisiones de diseño clave y los desafíos enfrentados durante el desarrollo.

6.2.1. Explicación del diseño de la web

GymTracker es una aplicación web diseñada para ayudar a los usuarios a gestionar sus rutinas de ejercicios, seguimiento de entrenamientos y estadísticas personales de manera eficiente y visualmente atractiva. A continuación se detalla la explicación del diseño general de la web, incluyendo la paleta de colores y la teoría del color utilizada.

Diseño General de la Aplicación

Interfaz de Usuario

El diseño de GymTracker se caracteriza por una interfaz limpia y moderna que promueve una experiencia de usuario intuitiva y eficiente. Los elementos clave del diseño incluyen:

1. **Navegación Clara y Estructurada:** La aplicación utiliza un menú lateral accesible desde todas las páginas principales, facilitando la navegación entre las diferentes secciones como inicio, administración de usuarios, nuevas rutinas, entrenamientos, ejercicios, estadísticas, clasificación, calendario y perfil del usuario.
2. **Consistencia Visual:** Los componentes de la interfaz, como botones, formularios, tablas y gráficos, mantienen un diseño coherente en toda la aplicación, utilizando una tipografía legible y un espaciado adecuado para mejorar la claridad y la usabilidad.
3. **Resaltado de Acciones Importantes:** Las acciones críticas y positivas están claramente destacadas mediante el uso de colores específicos, asegurando que los usuarios puedan identificar y realizar acciones rápidamente.
4. **Visualización de Datos:** La aplicación presenta datos de entrenamiento y progreso de manera visual, utilizando gráficos y tablas que permiten a los usuarios interpretar rápidamente su rendimiento y progreso.
5. **Compatibilidad y Responsividad:** GymTracker está diseñada para ser responsiva, garantizando una experiencia de usuario óptima tanto en dispositivos de escritorio como móviles.

Paleta de Colores y Teoría del Color

1. **Colores Principales:**
 - **Verde (#198754):** Es el color predominante utilizado en los botones de acción positiva como "Iniciar Sesión", "Crear usuario", "Nueva Rutina" y "Guardar". El

verde simboliza crecimiento, salud y energía, atributos fundamentales para una aplicación de fitness.

- **Rojo (#dc3545):** Utilizado para acciones críticas como "Cerrar sesión" y "Eliminar cuenta". El rojo se asocia con la alerta y la urgencia, destacando las acciones que requieren precaución.
- **Azul (#007bff):** Empleado en roles administrativos y en botones relacionados con la gestión de usuarios, simboliza confianza, seguridad y profesionalismo.
- **Amarillo(#ffc107):** Permite remarcar los avisos y alertas producidos por acciones que puedan provocar problemas dentro de la aplicación.
- **Gris (#6c757d):** Utilizado para elementos secundarios y de texto, proporciona un contraste neutro que equilibra el diseño y ayuda a resaltar los elementos más importantes.

2. Teoría del Color:

- **Contraste y Legibilidad:** El diseño asegura un alto contraste entre el texto y los fondos para mejorar la legibilidad. Se utilizan colores vibrantes como el verde y el rojo para destacar acciones importantes y guiar visualmente al usuario.
- **Consistencia:** La aplicación mantiene una consistencia en el uso de colores, lo que crea una experiencia de usuario fluida. Los usuarios pueden identificar fácilmente las diferentes secciones y acciones gracias a la coherencia cromática.
- **Emoción y Psicología del Color:** Los colores seleccionados buscan evocar emociones específicas. El verde transmite una sensación de bienestar y motivación, el rojo comunica urgencia y atención, mientras que el azul proporciona un sentido de profesionalidad y confianza.

En resumen, el diseño general de GymTracker es funcional, accesible y visualmente atractivo, utilizando una paleta de colores y teoría del color que mejoran la experiencia del usuario y refuerzan los objetivos de la aplicación.

6.2.2. Implementación de Spring Security

Uno de los desafíos más significativos que enfrentamos en este proyecto fue la integración de Spring Security en nuestra aplicación. Inicialmente, la seguridad de la aplicación se basaba en un sistema de redirecciones, lo cual requería una adaptación cuidadosa para alinearse con las características avanzadas y robustas que ofrece Spring Security. A continuación, detallamos los pasos que seguimos para lograr esta implementación:

6.2.2.1 Configuración de Spring Security

Para implementar Spring Security en nuestra aplicación, primero configuramos una clase de seguridad con las anotaciones y métodos necesarios para gestionar la autenticación y la

autorización. A continuación, explicamos paso a paso cada sección del código:

```
package com.dad.gymtracker.Configuracion;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.Authen
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigur
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

Explicación de los imports

- **org.springframework.context.annotation.Bean:** Permite definir métodos de configuración que serán gestionados por el contenedor de Spring.
- **org.springframework.context.annotation.Configuration:** Indica que la clase contiene configuraciones de Spring.
- **org.springframework.security.authentication.AuthenticationManager:** Gestiona la autenticación.
- **org.springframework.security.config.annotation.authentication.configuration.Authen ticationConfiguration:** Proporciona configuración para la autenticación.
- **org.springframework.security.config.annotation.web.builders.HttpSecurity:** Construye la configuración de seguridad HTTP.
- **org.springframework.security.config.annotation.web.configuration.EnableWebSecuri ty:** Habilita la configuración de seguridad web.
- **org.springframework.security.config.annotation.web.configurers.AbstractHttpConfig urer:** Permite personalizar la configuración HTTP.
- **org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder:** Implementa la codificación de contraseñas con BCrypt.
- **org.springframework.security.crypto.password.PasswordEncoder:** Interfaz para codificar contraseñas.
- **org.springframework.security.provisioning.InMemoryUserDetailsManager:** Gestiona los detalles de usuario en memoria.
- **org.springframework.security.web.SecurityFilterChain:** Define una cadena de filtros de seguridad.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

Explicación de las anotaciones

- **@Configuration:** Indica que esta clase es una fuente de definiciones de beans de Spring.
- **@EnableWebSecurity:** Habilita la configuración de seguridad web para la aplicación.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests((authorize) -> authorize
            .requestMatchers("/", "/usuario/**", "/css/**", "/images/**", "/fragments/**", "/js/**").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(login -> login
            .loginPage("/")
            .loginProcessingUrl("/login")
            .usernameParameter("nombre")
            .passwordParameter("contrasena")
            .successForwardUrl("/menu")
            .failureUrl("/?error=true")
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/?logout=true")
            .permitAll()
        )
        .csrf(AbstractHttpConfigurer::disable)
        .exceptionHandling(exceptionHandling -> exceptionHandling
            .accessDeniedPage("/403"));
    return http.build();
}
```

Explicación del método securityFilterChain

- **SecurityFilterChain securityFilterChain(HttpSecurity http):** Define la configuración de la cadena de filtros de seguridad.
- **authorizeHttpRequests:** Configura las autorizaciones de solicitudes HTTP.
 - **requestMatchers("/", "/usuario/**", "/css/**", "/images/**", "/fragments/**", "/js/**").permitAll():** Permite el acceso a las rutas especificadas sin autenticación.
 - **anyRequest().authenticated():** Requiere autenticación para cualquier otra solicitud.
- **formLogin:** Configura la autenticación basada en formularios.
 - **loginPage("/"):** Especifica la URL de la página de inicio de sesión.
 - **loginProcessingUrl("/login"):** Especifica la URL para procesar el inicio de sesión.
 - **usernameParameter("nombre"):** Define el parámetro del nombre de usuario.
 - **passwordParameter("contrasena"):** Define el parámetro de la contraseña.
 - **successForwardUrl("/menu"):** Redirige a esta URL en caso de inicio de sesión exitoso.
 - **failureUrl("/?error=true"):** Redirige a esta URL en caso de error en el inicio de sesión.
- **logout:** Configura la gestión de la sesión de cierre de sesión.
 - **logoutUrl("/logout"):** Especifica la URL para cerrar sesión.
 - **logoutSuccessUrl("/?logout=true"):** Redirige a esta URL después de cerrar sesión exitosamente.

- **csrf(AbstractHttpConfigurer::disable):** Deshabilita la protección CSRF para simplificar el desarrollo.
- **exceptionHandling:** Configura la gestión de excepciones.
 - **accessDeniedPage("/403"):** Especifica la URL de la página de acceso denegado.

```
@Bean
public InMemoryUserDetailsManager userDetailsService() {
    return new InMemoryUserDetailsManager();
}
```

Explicación del método userDetailsService

- **InMemoryUserDetailsManager userDetailsService():** Crea un gestor de usuarios en memoria. En nuestra configuración se utiliza para cargar los usuarios desde la base de datos gracias a un componente **DataLoader** personalizado.

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration) throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}
```

Explicación del método authenticationManager

- **AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration):** Proporciona el **AuthenticationManager** necesario para gestionar la autenticación en la aplicación.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Explicación del método passwordEncoder

- **PasswordEncoder passwordEncoder():** Define un codificador de contraseñas utilizando BCrypt, lo que proporciona una seguridad robusta al almacenar las contraseñas de los usuarios.

Esta configuración de seguridad en Spring Security asegura que nuestra aplicación gestione correctamente la autenticación y la autorización de usuarios, proporcionando una capa de seguridad fundamental para proteger los datos y recursos de la aplicación.

6.2.2.2 Cargar los usuarios de la base de datos a Spring Security

Para cargar los usuarios de la base de datos, se utilizó un componente **DataLoader** que carga los usuarios de la base de datos a Spring Security. A continuación, se presenta la explicación del código:

Declaraciones del paquete y las importaciones:

```
package com.dad.gymtracker.Loader;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.stereotype.Component;

import com.dad.gymtracker.Dto.UsuarioDTO;
import com.dad.gymtracker.Service.UsuarioService;

import java.util.List;
```

Estas líneas declaran el paquete y las importaciones necesarias para el funcionamiento del código. Ustedes importaron clases de Spring y las propias del proyecto.

Definición de la clase DataLoader:

```
@Component
public class DataLoader implements CommandLineRunner {
```

La anotación **@Component** indica que esta clase es un componente de Spring, lo que permite que Spring la detecte y la gestione automáticamente. La clase implementa **CommandLineRunner**, lo que significa que el método **run** se ejecutará después de que la aplicación Spring se haya iniciado.

Inyección de dependencias:

```

private final UsuarioService usuarioService;
private final InMemoryUserDetailsManager inMemoryUserDetailsManager;
private final PasswordEncoder passwordEncoder;

@Autowired
public DataLoader(UsuarioService usuarioService, InMemoryUserDetailsManager inMemoryUserDetailsManager, PasswordEncoder passwordEncoder) {
    this.usuarioService = usuarioService;
    this.inMemoryUserDetailsManager = inMemoryUserDetailsManager;
    this.passwordEncoder = passwordEncoder;
}

```

Ustedes inyectaron tres dependencias mediante el constructor:

- **UsuarioService:** Servicio que proporciona acceso a los datos de los usuarios.
- **InMemoryUserDetailsManager:** Clase que maneja los detalles de los usuarios en memoria.
- **PasswordEncoder:** Codificador de contraseñas para asegurar que las contraseñas sean almacenadas de manera segura.

Método run:

```

@Override
public void run(String... args) throws Exception {
    List<UsuarioDTO> usuarioDTOList = usuarioService.buscarAllUsuarios();
    for (UsuarioDTO usuarioDTO : usuarioDTOList) {
        UserDetails user = User.withUsername(usuarioDTO.getNombre())
            .password(passwordEncoder.encode(usuarioDTO.getContraseña()))
            .roles(usuarioDTO.getRol().toUpperCase())
            .build();
        inMemoryUserDetailsManager.createUser(user);
    }
}

```

- **run(String... args):** Este método se ejecuta al inicio de la aplicación.
- **List<UsuarioDTO> usuarioDTOList = usuarioService.buscarAllUsuarios();** obtener una lista de todos los usuarios de la base de datos utilizando el servicio **UsuarioService**.
- **Recorren la lista de usuarios y por cada UsuarioDTO:**
 - Crean un objeto **UserDetails** utilizando el nombre, la contraseña (codificada) y el rol del usuario.
 - Añaden el usuario a **InMemoryUserDetailsManager**.

6.2.2.2 Ajuste de Métodos CRUD en Servicios para Spring Security

En esta sección, explicamos los cambios realizados en los métodos CRUD en **UsuarioService** y **PerfilService** para adaptarlos a Spring Security. Estos son los métodos cambiados en cada servicio:

6.2.2.2.1 Cambios en UsuarioService

Método crearUsuario:

```
@Transactional
public void crearUsuario(UsuarioDTO usuarioDTO, PerfilDTO perfilUsuarioDTO) {
    try {
        String encodedPassword = passwordEncoder.encode(usuarioDTO.getContraseña());

        String sqlInsertUsuario = "INSERT INTO usuarios (nombre, contraseña, rol)" +
            " VALUES (?, ?, ?);";

        String rol = "usuario";
        if (usuarioDTO.getRol() != null) {
            rol = usuarioDTO.getRol();
        }

        entityManager.createNativeQuery(sqlInsertUsuario)
            .setParameter(1, usuarioDTO.getNombre())
            .setParameter(2, encodedPassword)
            .setParameter(3, rol)
            .executeUpdate();

        String sqlInsertPerfil = "INSERT INTO perfil (altura, edad, peso, genero, id_usuario)" +
            " VALUES (?, ?, ?, ?, (SELECT id FROM usuarios" +
            " WHERE id =(select LAST_INSERT_ID())));";

        entityManager.createNativeQuery(sqlInsertPerfil)
            .setParameter(1, perfilUsuarioDTO.getAltura())
            .setParameter(2, perfilUsuarioDTO.getEdad())
            .setParameter(3, perfilUsuarioDTO.getPeso())
            .setParameter(4, perfilUsuarioDTO.getGenero())
            .executeUpdate();

        // Crear UserDetails para el nuevo usuario (Spring Security)
        UserDetails user = User.withUsername(usuarioDTO.getNombre())
            .password(encodedPassword)
            .roles(rol.toUpperCase())
            .build();

        // Agregar el nuevo usuario al InMemoryUserDetailsManager
        inMemoryUserDetailsManager.createUser(user);
    } catch (Exception e) {
        log.error("Error al crear el usuario: {}", e.getMessage());
    }
}
```

- **Codificación de la contraseña:** Codificamos la contraseña del usuario utilizando **PasswordEncoder**.
- **Inserción del usuario en la base de datos:** Insertamos el usuario en la tabla **usuarios** con la contraseña codificada y el rol correspondiente.
- **Inserción del perfil del usuario:** Insertamos el perfil del usuario en la tabla **perfil**, asociándolo con el ID del usuario recién creado.

- **Creación de UserDetails:** Creamos un objeto **UserDetails** con el nombre de usuario, la contraseña codificada y el rol.
- **Agregar a InMemoryUserDetailsManager:** Añadimos el usuario a **InMemoryUserDetailsManager** para que Spring Security lo utilice.

Método cambiarContrasena:

```
@Transactional
public void cambiarContrasena(int id, String contrasena) {
    try {
        String encodedPassword = passwordEncoder.encode(contrasena);
        // Actualizar la contraseña en la base de datos
        String sqlCambiarContrasena = "UPDATE usuarios SET contrasena = ? WHERE id = ?";
        entityManager.createNativeQuery(sqlCambiarContrasena)
            .setParameter(1, encodedPassword)
            .setParameter(2, id)
            .executeUpdate();

        // Recuperar el nombre de usuario basado en el ID
        String sqlObtenerNombre = "SELECT nombre, rol FROM usuarios WHERE id = ?";
        Object[] result = (Object[]) entityManager.createNativeQuery(sqlObtenerNombre)
            .setParameter(1, id)
            .getSingleResult();

        String nombreUsuario = (String) result[0];
        String rolUsuario = (String) result[1];

        // Actualizar la contraseña en el InMemoryUserDetailsManager
        UserDetails user = User.withUsername(nombreUsuario)
            .password(encodedPassword)
            .roles(rolUsuario.toUpperCase())
            .build();

        inMemoryUserDetailsManager.updateUser(user);
    } catch (Exception e) {
        log.error("Error al cambiar la contraseña: {}", e.getMessage());
    }
}
```

- **Codificación de la contraseña:** Codificamos la nueva contraseña utilizando **PasswordEncoder**.
- **Actualización de la contraseña en la base de datos:** Actualizamos la contraseña del usuario en la tabla **usuarios**.
- **Recuperación del nombre de usuario y rol:** Obtenemos el nombre de usuario y el rol correspondiente a partir del ID del usuario.
- **Actualización en InMemoryUserDetailsManager:** Creamos un nuevo objeto **UserDetails** con la nueva contraseña codificada y actualizamos en **InMemoryUserDetailsManager**.

6.2.2.2 Cambios en PerfilService

Método editarUsuario:

```
@Transactional
public void editarUsuario(PerfilDTO perfilDTO) {
    try {
        // Actualizar el perfil y el nombre de usuario en la base de datos
        String sql = "UPDATE perfil AS p "
            + "JOIN usuarios AS u ON p.id_usuario = u.id "
            + "SET p.edad = ?, p.genero = ?, p.peso = ?, p.altura = ?, u.nombre = ? "
            + "WHERE p.id_usuario = ?";

        entityManager.createNativeQuery(sql)
            .setParameter(1, perfilDTO.getEdad())
            .setParameter(2, perfilDTO.getGenero())
            .setParameter(3, perfilDTO.getPeso())
            .setParameter(4, perfilDTO.getAltura())
            .setParameter(5, perfilDTO.getNombre())
            .setParameter(6, perfilDTO.getId())
            .executeUpdate();

        // Recuperar el nombre de usuario y el rol basado en el ID
        String sqlObtenerDatos = "SELECT u.nombre, u.contrasena, u.rol FROM usuarios AS u WHERE u.id = ?";
        Object[] resultado = (Object[]) entityManager.createNativeQuery(sqlObtenerDatos)
            .setParameter(1, perfilDTO.getId())
            .getSingleResult();

        String nombreUsuarioAnterior = (String) resultado[0];
        String contrasenaAnterior = (String) resultado[1];
        String rolUsuario = (String) resultado[2];

        // Eliminar el usuario antiguo del InMemoryUserDetailsManager
        inMemoryUserDetailsManager.deleteUser(nombreUsuarioAnterior);

        // Construir un nuevo UserDetails con el nuevo nombre de usuario y la contraseña existente
        UserDetails updatedUser = User.withUsername(perfilDTO.getNombre())
            .password(contrasenaAnterior) // Mantener la contraseña existente
            .roles(rolUsuario.toUpperCase()) // Mantener los roles existentes
            .build();

        // Agregar el usuario actualizado al InMemoryUserDetailsManager
        inMemoryUserDetailsManager.createUser(updatedUser);
    } catch (Exception e) {
        log.error("Error al editar el usuario: {}", e.getMessage());
    }
}
```

- **Actualización en la base de datos:** Actualizamos el perfil del usuario y el nombre de usuario en las tablas **perfil** y **usuarios**.
- **Recuperación de datos del usuario:** Obtenemos el nombre de usuario, la contraseña y el rol del usuario.
- **Eliminación del usuario antiguo:** Eliminamos el usuario antiguo de **InMemoryUserDetailsManager**.
- **Creación de UserDetails actualizado:** Creamos un nuevo **UserDetails** con el nuevo nombre de usuario y la contraseña existente.
- **Agregar el usuario actualizado:** Añadimos el usuario actualizado a **InMemoryUserDetailsManager**.

Método borrarUsuario:

```

@Transactional
public void borrarUsuario(int idUsuario) {
    try {
        // Borrar series asociadas al usuario
        String deleteSeriesSql = "DELETE FROM series " +
            "WHERE id IN (" +
            "    SELECT rs.id_serie " +
            "    FROM rutinas_ejercicios_series rs " +
            "    JOIN rutinas r ON rs.id_rutina = r.id " +
            "    WHERE r.id_usuario = ? )";
        entityManager.createNativeQuery(deleteSeriesSql)
            .setParameter(1, idUsuario)
            .executeUpdate();

        // Recuperar el nombre de usuario basado en el ID
        String sqlObtenerNombre = "SELECT nombre FROM usuarios WHERE id = ?";
        String nombreUsuario = (String) entityManager.createNativeQuery(sqlObtenerNombre)
            .setParameter(1, idUsuario)
            .getSingleResult();

        // Eliminar al usuario de la base de datos
        String deleteUsuarioSql = "DELETE FROM usuarios WHERE id = ?";
        entityManager.createNativeQuery(deleteUsuarioSql)
            .setParameter(1, idUsuario)
            .executeUpdate();

        // Eliminar al usuario del InMemoryUserDetailsManager
        inMemoryUserDetailsManager.deleteUser(nombreUsuario);
    } catch (Exception e) {
        log.error("Error al borrar el usuario: {}", e.getMessage());
    }
}

```

- **Eliminación de series asociadas:** Eliminamos las series asociadas al usuario de las tablas correspondientes.
- **Recuperación del nombre de usuario:** Obtenemos el nombre de usuario basado en el ID.
- **Eliminación del usuario de la base de datos:** Eliminamos el usuario de la tabla usuarios.
- **Eliminación del usuario de InMemoryUserDetailsManager:** Eliminamos el usuario de InMemoryUserDetailsManager.

Estos cambios mejoran la seguridad al codificar las contraseñas, asegurando que no se almacenen en texto plano. Además, la integración con `InMemoryUserDetailsManager` facilita la gestión centralizada de usuarios dentro de Spring Security, lo que simplifica el mantenimiento y la actualización de los detalles de usuario. En conjunto, estos ajustes permiten manejar usuarios y contraseñas de manera segura y eficiente, alineándose con las mejores prácticas de Spring Security.

6.2.3 Esquema de Plantillas en Bootstrap para GymTracker

Para mantener una apariencia consistente y una funcionalidad óptima en toda la aplicación GymTracker, hemos diseñado nuestras plantillas utilizando Bootstrap. Esto asegura que la identidad de la aplicación se mantenga uniforme, brindando una experiencia fluida a los usuarios. Aquí está cómo hemos implementado estas plantillas:

Elementos Comunes de las Plantillas

- 1. HTML Básico y Metadatos**
 - Declaramos DOCTYPE y el lenguaje (`<html lang="es">`).
 - Configuramos la codificación (`<meta charset="UTF-8">`) y el viewport para la responsividad (`<meta name="viewport" content="width=device-width, initial-scale=1.0">`).
 - Cada página tiene un título específico y un favicon para la identidad de la marca.
- 2. Inclusión de Fragmentos y Estilos**
 - Utilizamos Thymeleaf para incluir fragmentos comunes, como Bootstrap y el sidebar (`<div th:replace="{fragments/bootstrap.html}"></div>` y `<div th:replace="{fragments/sidebar.html}"></div>`).
 - Incluimos archivos CSS específicos para cada sección (`<link rel="stylesheet" href="/css/styles_inicioSesion.css">`).
- 3. Estructura del Contenido**
 - Organizamos el contenido dentro de contenedores (`<div class="container mt-5">`) y filas (`<div class="row">`).
 - Utilizamos clases de Bootstrap para el diseño responsivo y la alineación de elementos, como **d-flex**, **justify-content-center**, **align-items-center**, **col-md-6**, **shadow-lg**.
- 4. Encabezado de la Tarjeta**
 - Cada plantilla incluye un encabezado con el logo de GymTracker y el título de la sección actual (`<h1 class="text-center">`).
 - Todas las secciones tienen un botón de menú para el sidebar (`<button class="btn btn-success" data-bs-toggle="offcanvas" data-bs-target="#idSidebar">`).
- 5. Cuerpo de la Tarjeta**
 - Colocamos formularios, listas de rutinas, estadísticas y otros contenidos específicos dentro del cuerpo de la tarjeta (`<div class="card-body">`).
- 6. Mensajes de Alerta y Errores**
 - Utilizamos alertas de Bootstrap para mostrar mensajes de error o advertencia (`<div class="alert alert-warning mt-3" th:if="{mensajeErrorWarning}">`).
- 7. JavaScript y Thymeleaf**
 - Incluimos scripts de JavaScript para funcionalidades dinámicas, propios para cada sección (`<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>` y `<script src="/js/estadisticas.js"></script>`).

- Empleamos sintaxis de Thymeleaf para la inclusión dinámica de datos y valores (**th:if**, **th:each**, **th:text**, etc.).

Explicación de la Implementación

Hemos estructurado las plantillas de GymTracker siguiendo este esquema común para garantizar una experiencia de usuario coherente y mantener la identidad visual de la aplicación. Al utilizar Bootstrap y Thymeleaf, podemos centralizar y reutilizar componentes comunes, lo que simplifica el mantenimiento y la actualización de la aplicación.

La inclusión de fragmentos y estilos comunes asegura que todas las páginas tengan un aspecto uniforme y profesional. Además, la utilización de Thymeleaf nos permite generar dinámicamente contenido y valores, mientras que Bootstrap nos proporciona una estructura sólida y responsiva para el diseño.

En resumen, al seguir estos principios de diseño y desarrollo, hemos creado una aplicación que no solo es funcional, sino que también presenta una apariencia consistente y atractiva en todas sus páginas, lo que mejora la usabilidad y la experiencia del usuario.

6.2.4 Explicación de la dinámica general seguida para el MVC

En esta sección, exploramos nuestra dinámica de desarrollo para las pantallas de GymTracker, centrándonos en la pantalla "Mis Entrenamientos". Explicamos cómo aplicamos el patrón Modelo-Vista-Controlador (MVC) para organizar el código, asegurando una clara separación de responsabilidades. A través de este ejemplo, demostraremos cómo el MVC facilita la gestión eficiente de la lógica de negocio, la presentación de datos y la interacción del usuario. A continuación, se detalla el código.

6.2.4.1 Controlador

En el controlador "MisEntrenamientosController", se invoca el servicio "rutinaService" para recuperar los entrenamientos asociados al usuario. Luego, estos datos se agregan al modelo, lo que permite pasarlos a la plantilla correspondiente. Finalmente, la plantilla se muestra al usuario, proporcionando así la interfaz visual con la lista de entrenamientos.

```
MisEntrenamientosController.java x RutinaService.java listar.html

1 package com.dad.gymtracker.Controller;
2
3 > import ...
13
14 @Controller
15 @AllArgsConstructor
16 @RequestMapping("/mis-entrenamientos")
17 public class MisEntrenamientosController {
18
19     2 usages
20     private final String RUTATEMPLES= "misEntrenamientos/";
21
22     private final RutinaService rutinaService;
23
24     @GetMapping
25     public String misEntrenamientos(Model model, HttpSession session) {
26         List<MisEntrenamientosDTO> listaEntrenamientos = rutinaService.buscarEntrenamientosById((int) session.getAttribute("idUsuario"));
27         model.addAttribute("rolUsuario", session.getAttribute("rolUsuario"));
28         if(!listaEntrenamientos.isEmpty()) {
29             model.addAttribute("listaEntrenamientos", listaEntrenamientos);
30         }
31         return RUTATEMPLES + "listar";
32     }
33 }
```

1. **Declaración de la Ruta y el Servicio:** El controlador está anotado con **@Controller** y **@RequestMapping("/mis-entrenamientos")**, lo que significa que manejará las solicitudes HTTP dirigidas a la ruta `/mis-entrenamientos`. Además, se inyecta el servicio **RutinaService** que se utilizará para recuperar los entrenamientos del usuario.

2. **Método misEntrenamientos:** Este método se anota con **@GetMapping**, lo que indica que responderá a las solicitudes HTTP GET en la ruta `"/mis-entrenamientos"`. Toma dos parámetros: un objeto **Model** y un objeto **HttpSession**. El **Model** se utiliza para agregar atributos que se enviarán a la plantilla. La **HttpSession** se utiliza para obtener el ID del usuario actualmente autenticado.
3. **Recuperación de los Entrenamientos:** Dentro del método **misEntrenamientos**, se invoca el método **buscarEntrenamientosById** del servicio **rutinaService**, pasando el ID de usuario obtenido de la sesión. Esto devuelve una lista de entrenamientos asociados al usuario.
4. **Adición de Atributos al Modelo:** Se agrega al modelo el atributo `"rolUsuario"`, que se obtiene de la sesión. Esto permitirá que la plantilla acceda al rol del usuario actual.
5. **Verificación y Adición de la Lista de Entrenamientos al Modelo:** Se verifica si la lista de entrenamientos no está vacía. Si contiene datos, se agrega al modelo con el nombre `"listaEntrenamientos"`. Esto asegura que solo se envíe a la plantilla una lista válida de entrenamientos.
6. **Retorno de la Ruta de la Plantilla:** Finalmente, se retorna la ruta de la plantilla que se utilizará para mostrar los entrenamientos. La ruta está construida concatenando la constante **RUTATEMPLES** con el nombre de la plantilla `"listar"`.

En resumen, este controlador se encarga de recuperar los entrenamientos del usuario, agregarlos al modelo y mostrarlos en la plantilla correspondiente, manteniendo así la lógica de negocio separada de la presentación de datos.

6.2.4.2 Servicio

```
@Slf4j
@Service
public class RutinaService {

    9 usages
    @PersistenceContext
    private EntityManager entityManager;

    2 usages  Daniel Marin Gomez
    public List<MisEntrenamientosDTO> buscarEntrenamientosById(int id) {
        String sqlBuscarMisEntrenamientos = "SELECT * FROM rutinas WHERE id_usuario = ? ORDER BY id";

        List<MisEntrenamientosDTO> resultado = entityManager.createNativeQuery(sqlBuscarMisEntrenamientos, MisEntrenamientosDTO.class)
            .setParameter(1, id)
            .getResultList();

        return resultado.stream()
            .map(t -> MisEntrenamientosDTO.builder()
                .id(t.getId())
                .nombre(t.getNombre())
                .fecha(t.getFecha())
                .build())
            .collect(Collectors.toList());
    }
}
```

En este servicio, nos encargamos de interactuar con la base de datos para recuperar los entrenamientos del usuario. Lo que hacemos es:

1. **Anotaciones y Declaraciones:** Hemos anotado el servicio con **@Service**, indicando que es un componente de servicio de Spring. También hemos declarado **@PersistenceContext** para inyectar el **EntityManager**, que utilizaremos para interactuar con la base de datos.
2. **Método buscarEntrenamientosById:** Este método recibe como parámetro el ID del usuario y devuelve una lista de entrenamientos asociados a ese usuario. Construimos una consulta SQL para seleccionar todos los entrenamientos del usuario especificado, ordenados por ID.
3. **Ejecución de la Consulta:** Ejecutamos la consulta SQL utilizando el **EntityManager**. Los resultados se obtienen como una lista de objetos **MisEntrenamientosDTO**.
4. **Mapeo de Resultados:** Mapeamos los resultados obtenidos a objetos **MisEntrenamientosDTO**. Esto lo hacemos utilizando **stream()** y **map()**, donde construimos nuevos objetos **MisEntrenamientosDTO** con los datos obtenidos de la consulta.
5. **Retorno de la Lista Mapeada:** Finalmente, retornamos la lista mapeada, que contiene los entrenamientos del usuario en el formato deseado.

Con este enfoque, conseguimos una clara separación de responsabilidades entre el controlador y el acceso a datos, siguiendo el principio de MVC (Modelo-Vista-Controlador). Esto nos permite mantener un código organizado y modular, facilitando su mantenimiento y escalabilidad.

6.2.4.2.1 Mapeo de datos de la base de datos a DTO

Para poder mapear los datos de la base de datos al objeto DTO **MisEntrenamientoDTO** utilizamos el fichero **orm.xml** en la carpeta **resources**, definimos el siguiente esquema para mapear **MisEntrenamientosDTO**:

```
<entity class="com.dad.gymtracker.Dto.MisEntrenamientosDTO" name="MisEntrenamientosMapping">
  <attributes>
    <basic name="nombre"/>
    <basic name="fecha"/>
  </attributes>
</entity>
```

Este esquema se realiza para definir cómo los datos recogidos de la base de datos deben ser mapeados a un objeto DTO llamado **MisEntrenamientosDTO**. En este caso, especificamos que el atributo **nombre** se mapea a la columna correspondiente en la base de datos con el mismo nombre, y lo mismo para el atributo **fecha**. Este mapeo facilita la manipulación de datos entre la base de datos y los objetos DTO en la aplicación, asegurando una integración fluida y coherente de la capa de acceso a datos con la lógica de la aplicación.

6.2.4.3 Plantilla

En esta plantilla se utilizan los datos añadidos al modelo gracias a la sintaxis de Thymeleaf, lo que nos permite ejemplificar el patrón Modelo-Vista-Controlador (MVC) mencionado anteriormente. A continuación, explicaremos cómo interactúa la plantilla de la pantalla de "Mis Entrenamientos" con los datos añadidos al modelo por el controlador:

```
<div class="row">
  <div class="col-12 order-md-1 mb-3">
    <div class="list-group text-center" th:unless="{listaEntrenamientos}">
      <a href="/nueva-rutina" class="btn btn-outline-success p-3">No tienes ninguna rutina.
        Pulsa aquí para crear una.</a>
    </div>
    <div class="list-group text-center" th:each="entrenamiento:{listaEntrenamientos}">
      <a th:href="@{/mis-entrenamientos/mostrar(id={entrenamiento.id})}" class="btn btn-outline-success p-3 mb-3"
        th:text="{entrenamiento.nombre + ' - ' + entrenamiento.fecha}"></a>
    </div>
  </div>
</div>
```

Explicación:

- **th:unless="{listaEntrenamientos}"**: Esta directiva de Thymeleaf verifica si la lista de entrenamientos está vacía. Si es así, muestra un mensaje indicando que no hay rutinas y proporciona un enlace para crear una nueva rutina.
- **th:each="entrenamiento:{listaEntrenamientos}"**: Esta directiva de Thymeleaf itera sobre la lista de entrenamientos. Por cada entrenamiento en la lista, crea un enlace para mostrar los detalles del entrenamiento.
- **th:href="@{/mis-entrenamientos/mostrar(id={entrenamiento.id})}"**: Esta expresión de Thymeleaf se utiliza para establecer dinámicamente el atributo **href** del enlace. Utiliza la sintaxis **{}** para acceder al **id** de cada entrenamiento en la lista. El enlace generado llevará al controlador de **MisEntrenamientosController** y pasará el **id** del entrenamiento como parámetro en la URL.
- **th:text="{entrenamiento.nombre + ' - ' + entrenamiento.fecha}"**: Esta expresión de Thymeleaf se utiliza para establecer dinámicamente el texto del enlace. Concatena el nombre y la fecha del entrenamiento para proporcionar una descripción concisa del mismo.

En resumen, esta plantilla HTML interactúa con el modelo al mostrar dinámicamente la lista de entrenamientos disponibles. Cada entrenamiento se presenta como un enlace que contiene el nombre y la fecha del mismo, y al hacer clic en uno de estos enlaces, se accede a la pantalla de nueva rutina. Además, si la lista de entrenamientos está vacía, se proporciona un enlace directo para crear una nueva rutina. Esta interacción permite al usuario navegar fácilmente

entre los entrenamientos existentes y crear uno nuevo con los datos del entrenamiento seleccionado.

7. Despliegue

En este apartado se abordará el proceso de despliegue de la aplicación web, Gymtracker, utilizando diversas plataformas y herramientas para asegurar su disponibilidad y correcto funcionamiento en diferentes entornos. En primer lugar, se detallará el despliegue en Amazon Web Services (AWS), una de las plataformas en la nube más robustas y ampliamente utilizadas. A continuación, se describirá el uso de Docker para la creación de contenedores que faciliten la portabilidad y escalabilidad de la aplicación. También se incluirá la implementación a través de Spring Tools Suite (STS), un entorno de desarrollo integrado especializado en aplicaciones Spring, y finalmente, se explicará el despliegue local, proporcionando una guía para ejecutar la aplicación en un entorno de desarrollo personal. Cada método será presentado con sus respectivas configuraciones, ventajas y consideraciones clave, permitiendo una visión completa y detallada de las opciones de despliegue disponibles.

7.1 Despliegue en Aws

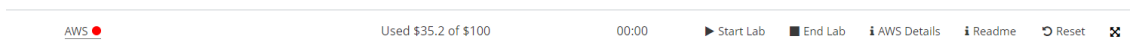
7.1.1 Lanzar instancia

Para el despliegue de en una instancia de aws

Accedemos a la cuenta de aws en nuestro caso será **awsacademy**

<https://awsacademy.instructure.com/>

Una vez dentro encendemos el laboratorio de aws



StarLab y esperamos a que el indicador de AWS cambie de rojo/amarillo a verde



Pulsamos en AWS y entramos a la “Página de inicio de la Consola”

Entre los distintos servicios que nos ofrece buscamos el que dice EC2, en la categoría de informática.



Informática

EC2

Vamos a lanzar una instancia nueva, le damos al botón naranja “Lanzar la instancia”.

Lanzar la instancia

Para comenzar, lance una instancia de Amazon EC2, que es un servidor virtual en la nube.

Lanzar la instancia ▼

Migrar un servidor ↗

Nota: Sus instancias se lanzarán en la región EE.UU. Este (Norte de Virginia)

Ponemos un nombre a la instancia.

Nombre y etiquetas Información

Nombre

GymTracker

Agregar etiquetas adicionales

Y seleccionamos la imagen que queremos que tenga nuestra instancia, en nuestro caso será **Amazon Linux..**

▼ Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon) Información

Una AMI es una plantilla que contiene la configuración de software (sistema operativo, servidor de aplicaciones y aplicaciones) necesaria para lanzar la instancia. Busque o examine las AMI si no ve lo que busca a continuación.

Q Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Recientes Inicio rápido

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux

Buscar más AMI

Inclusión de AMI de AWS, Marketplace y la comunidad

▼ Tipo de instancia Información | Obtener asesoramiento

Tipo de instancia

t2.micro Apto para la capa gratuita

Familia: t2 1 vCPU 1 GiB Memoria Generación actual: true

Bajo demanda Windows base precios: 0.0162 USD por hora

Bajo demanda SUSE base precios: 0.0116 USD por hora

Bajo demanda RHEL base precios: 0.0716 USD por hora

Bajo demanda Linux base precios: 0.0116 USD por hora

Se aplican costos adicionales a las AMI con software preinstalado

Todas las generaciones

Comparar tipos de instancias

Generamos un par de claves nuevas

▼ **Par de claves (inicio de sesión)** [Información](#)

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - *obligatorio*

[Crear un nuevo par de claves](#)

Ponemos nombre a la clave y nos aseguramos de crearlas en formato **RSA** y **.pem**

Crear par de claves ✕

Nombre del par de claves
Con los pares de claves es posible conectarse a la instancia de forma segura.

El nombre puede incluir hasta 255 caracteres ASCII. No puede incluir espacios al principio ni al final.

Tipo de par de claves

☒ **RSA**
Par de claves pública y privada cifradas mediante RSA

☐ **ED25519**
Par de claves privadas y públicas cifradas ED25519

Formato de archivo de clave privada

☒ **.pem**
Para usar con OpenSSH

☐ **.ppk**
Para usar con PuTTY

⚠ Cuando se le solicite, almacene la clave privada en un lugar seguro y accesible del equipo. **Lo necesitará más adelante para conectarse a la instancia.** [Más información](#)

[Cancelar](#) [Crear par de claves](#)

Permitimos el tráfico **SSH | HTTP | HTTPS**

▼ **Configuraciones de red** [Información](#) [Editar](#)

Red [Información](#)
vpc-0156ec04ea749c386

Subred [Información](#)
Sin preferencias (subred predeterminada en cualquier zona de disponibilidad)

Asignar automáticamente la IP pública [Información](#)

Habilitar

Se aplican cargos adicionales cuando no se cumplen los límites del [nivel gratuito](#)

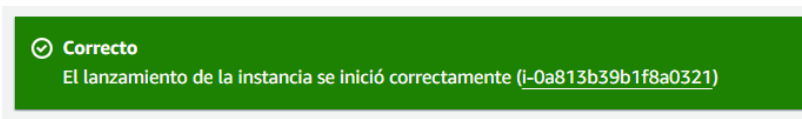
Firewall (grupos de seguridad) [Información](#)
Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☒ **Crear grupo de seguridad** ☐ Seleccionar un grupo de seguridad existente

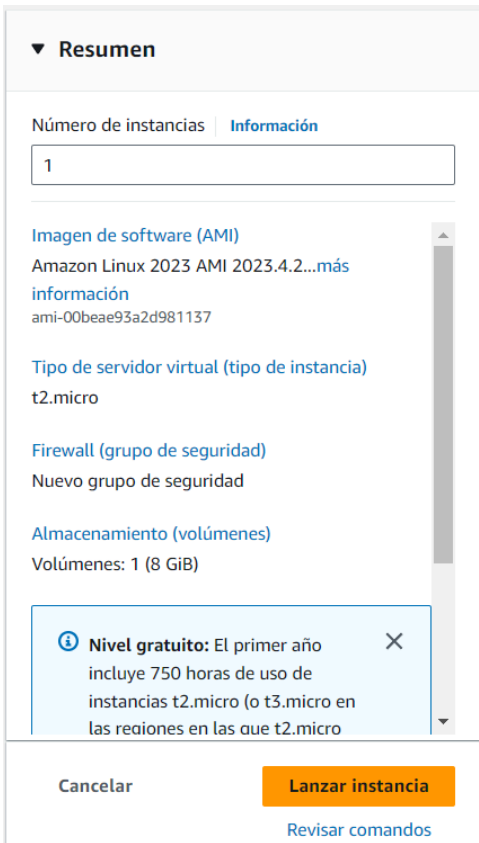
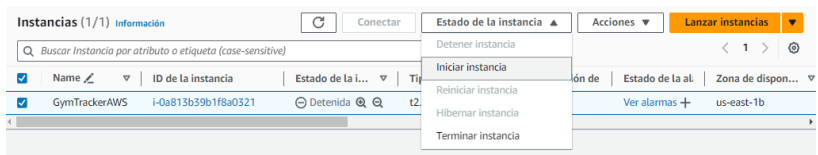
Crearemos un nuevo grupo de seguridad denominado **"launch-wizard-7"** con las siguientes reglas:

☒ **Permitir el tráfico de SSH desde**
Ayuda a establecer conexión con la instancia

Y lanzamos la instancia.

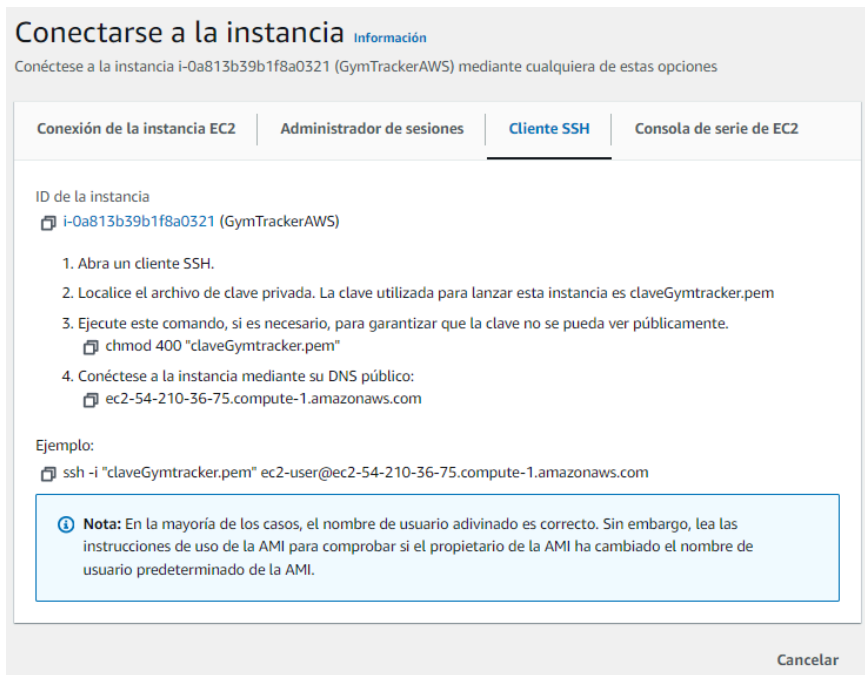


La instancia se pondrá en ejecución automáticamente. Si no lo hiciere seleccionamos la instancia, y marcamos en estado de instancia la opción de “Iniciar instancia”.



7.1.2 Conectarse a la instancia

Para conectarnos a la instancia por SSH seguiremos los pasos que nos indica.



Nos situamos en la ubicación en la que hemos guardado la clave previamente

y nos aseguramos de darle los permisos necesarios.

```
tfg@tfg-VirtualBox:~/Escritorio$ chmod 400 "claveGymtracker.pem"
```

Nos conectamos a la instancia mediante ssh con el comando

```
ssh -i "nombre_de_la_clave" usuario@dns_de_la_instancia
```

```
tfg@tfg-VirtualBox:~/Escritorio$ ssh -i "claveGymtracker.pem" ec2-user@ec2-54-210-36-75.compute-1.amazonaws.com  
The authenticity of host 'ec2-54-210-36-75.compute-1.amazonaws.com (54.210.36.75)' can't be established.  
ED25519 key fingerprint is SHA256:R0hgNgj8Fv68ODVYEsIa8dekE5cfA1RL05BGMbZChuU.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-54-210-36-75.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
```

The terminal window displays the successful execution of an SSH command from a local machine named tfg at ip-172-31-19-47 to a remote Amazon EC2 instance. The user is identified as ec2-user. After confirming the ED25519 key fingerprint, an ASCII art logo representing the AWS Linux distribution is shown. To the right of the logo, the system version 'Amazon Linux 2023' and the official URL '<https://aws.amazon.com/linux/amazon-linux-2023>' are displayed. The prompt returns to '~/\$' indicating the session is active.

```
#  
~\####          Amazon Linux 2023  
~~ \#####\  
~~   \|###|  
~~    \|#/  
~~~~ V ~ ' .->  
      /_____  
     /_./_.  
    /__/_/___  
   /m/'_____
```

[ec2-user@ip-172-31-19-47 ~]\$

7.1.3 Preparar instancia para el despliegue

Una vez que hemos comprobado que la instancia funciona correctamente la preparamos para el despliegue de la aplicación web.

Descargamos los jars de la aplicación desde github. en el siguiente enlace en la carpeta aws / Jars

<https://github.com/deiivvv/GymTracker>

Para copiar los archivos usamos el comando

```
scp -i nombreClave.pem nombreJar.jar usuario@dns_de_la_instancia:ubicacionEnLaInstancia
```

```
tfg@tfg-VirtualBox:~/Escritorio/GymTracker/aws$ scp -i ./Claves/claveGymtracker.pem ./Jars/GymTracker-0.0.1-SNAPSHOT.jar ./Jars/Api-Ejercicios-0.0.1-SNAPSHOT.jar ec2-user@ec2-54-210-36-75.compute-1.amazonaws.com:/home/ec2-user/GymTracker-0.0.1-SNAPSHOT.jar
GymTracker-0.0.1-SNAPSHOT.jar          100% 53MB 12.4MB/s  00:04
Api-Ejercicios-0.0.1-SNAPSHOT.jar     100% 45MB 15.5MB/s  00:02
```

Podemos comprobar que se copiaron los jars correctamente

```
[ec2-user@ip-172-31-19-47 ~]$ ls
Api-Ejercicios-0.0.1-SNAPSHOT.jar  GymTracker-0.0.1-SNAPSHOT.jar
[ec2-user@ip-172-31-19-47 ~]$
```

Nos aseguramos que tenemos Java en la instancia con el comando:

```
java -version
```

En el caso de no tenerlo o tenerlo en una versión del jdk por debajo de la 17 usamos el comando

```
sudo yum install java
```

Una vez completada la instalación de java se recomienda volver a tirar el `java -version` para comprobar que se ha instalado correctamente.

Lo mismo hacemos con la base de datos que usaremos, **mariadb**

sudo yum install mariadb105-server

```
[ec2-user@ip-172-31-19-47 ~]$ sudo yum install mariadb105-server
```

Una vez completa la instalación nos aseguramos de que mariadb esté iniciado

sudo systemctl start mariadb

o

sudo systemctl restart mariadb

```
[ec2-user@ip-172-31-19-47 ~]$ sudo systemctl restart mariadb
```

Accedemos a mariadb

sudo mysql

```
[ec2-user@ip-172-31-19-47 ~]$ sudo mysql
Welcome to the MariaDB monitor.  Command
```

Y volcamos todo el contenido del archivo **gymtracker.sql** que se encuentra en el github en la carpeta **bbdd**

```
MariaDB [(none)]> DROP DATABASE IF EXISTS gymtracker;
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [(none)]> CREATE DATABASE gymtracker;
Query OK, 1 row affected (0.000 sec)

-- -u alvaro -p alvaro
MariaDB [(none)]> GRANT ALL PRIVILEGES ON gymtracker.* TO 'alvaro'@'localhost';
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> DROP USER IF EXISTS 'dad'@'localhost';
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [(none)]> CREATE USER 'dad'@'localhost' IDENTIFIED BY 'padre';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON gymtracker.* TO 'dad'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> USE gymtracker;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> CREATE TABLE usuarios (
  id INT AUTO_INCREMENT,
  nombre VARCHAR(100) UNIQUE,
  contraseña VARCHAR(100),
  rol VARCHAR(50),
  PRIMARY KEY (id)
);
Query OK, 0 rows affected (0.001 sec)
```

```
Code Blame 201 lines (161 loc) · 6.28 KB

1 DROP DATABASE IF EXISTS gymtracker;
2 CREATE DATABASE gymtracker;
3
4 DROP USER IF EXISTS 'dad'@'localhost';
5 CREATE USER 'dad'@'localhost' IDENTIFIED BY 'padre';
6 GRANT ALL PRIVILEGES ON gymtracker.* TO 'dad'@'localhost';
7
8 USE gymtracker;
9
10 CREATE TABLE usuarios (
11   id INT AUTO_INCREMENT,
12   nombre VARCHAR(100) UNIQUE,
13   contraseña VARCHAR(100),
14   rol VARCHAR(50),
15   PRIMARY KEY (id)
16 );
17
```

Para salir de mariadb ponemos **exit** en la

consola.

7.1.4 Ejecutar GymTracker

Creamos los dos scripts necesarios para ejecutar los jar.

O los descargamos desde el github en la carpeta **aws / Scripts** y los copiamos en la instancia según los pasos previos.

sudo vim gymtracker.sh

```
[ec2-user@ip-172-31-19-47 ~]$ sudo vim gymtracker.sh
```

Le añadimos el siguiente contenido:

#!/bin/bash

nohup java -jar GymTracker-0.0.1-SNAPSHOT.jar > first.log 2>&1 &

nohup java -jar Api-Ejercicios-0.0.1-SNAPSHOT.jar > second.log 2>&1 &

```
#!/bin/bash
nohup java -jar GymTracker-0.0.1-SNAPSHOT.jar > first.log 2>&1 &
nohup java -jar Api-Ejercicios-0.0.1-SNAPSHOT.jar > second.log 2>&1 &
```

sudo vim stop_gymtracker.sh

```
[ec2-user@ip-172-31-19-47 ~]$ sudo vim stop gymtracker.sh
```

Le añadimos el siguiente contenido:

```
#!/bin/bash
kill -f 'java -jar GymTracker-0.0.1-SNAPSHOT.jar'
kill -f 'java -jar Api-Ejercicios-0.0.1-SNAPSHOT.jar'
```

```
#!/bin/bash
kill -f 'java -jar GymTracker-0.0.1-SNAPSHOT.jar'
kill -f 'java -jar Api-Ejercicios-0.0.1-SNAPSHOT.jar'
```

gymtracker.sh -> levantará los dos jars en segundo plano y creará los archivos first.log y second.log donde se almacenará la información que devuelva la consola.

stop_gymtracker.sh -> finaliza la ejecución de los jars

Para ejecutar los scripts usamos el comando con permisos de administrador, **SUDO**,
sudo bash ./nombre_script.sh

sudo bash ./gymtracker.sh

```
[ec2-user@ip-172-31-19-47 ~]$ bash ./gymtracker.sh
```

sudo bash ./stop_gymtracker.sh

```
[ec2-user@ip-172-31-19-47 ~]$ bash ./stop gymtracker.sh
```

Después de ejecutar el primer script y levantar la aplicación.

Para comprobar que se han levantado correctamente, usamos el comando:

ps -ef | grep java

Busca todos los procesos en ejecución y filtrarlos por “java” .

```
[ec2-user@ip-172-31-19-47 ~]$ ps -ef | grep java
ec2-user  27930      1 45 15:44 pts/0    00:00:02 java -jar GymTracker-0.0.1-SNAPSHOT.jar
ec2-user  27931      1 45 15:44 pts/0    00:00:02 java -jar Api-Ejercicios-0.0.1-SNAPSHOT.jar
ec2-user  27959    2408  0 15:44 pts/0    00:00:00 grep --color=auto java
```

Ahora puedes acceder desde cualquiera de tus navegadores con la url

https://dns_ipv4_publica_dela_instancia

algo así como

```
https://ec2-18-208-158-106.compute-1.amazonaws.com
```

el dns público de la instancia lo podrás encontrar en el apartado de Detalles de tu instancia en aws

i-0a813b39b1f8a0321 (GymTrackerAWS)		
Detalles	Estado y alarmas Novedad	Monitoreo Seguridad Redes Almacenamiento Etiquetas
▼ Resumen de instancia Información		
ID de la instancia i-0a813b39b1f8a0321 (GymTrackerAWS)	Dirección IPv4 pública 18.208.158.106 dirección abierta	Direcciones IPv4 privadas 172.31.19.47
Dirección IPv6 -	Estado de la instancia En ejecución	DNS de IPv4 pública ec2-18-208-158-106.compute-1.amazonaws.com dirección abierta
Tipo de nombre de anfitrión Nombre de IP: ip-172-31-19-47.ec2.internal	Nombre DNS de IP privada (solo IPv4) ip-172-31-19-47.ec2.internal	Direcciones IP elásticas -
Responder al nombre DNS de recurso privado IPv4 (A)	Tipo de instancia t2.micro	

Para finalizar la ejecución de la aplicación web ejecutamos el comando script stop_gymtracker

bash ./stop_gymtracker.sh

```
ec2-user@ip-172-31-19-47 ~]$ bash ./stop_gymtracker.sh
```

7.1.5 IP Elástica

La dirección IPv4 de la instancia cambia cada vez que se inicia la instancia.

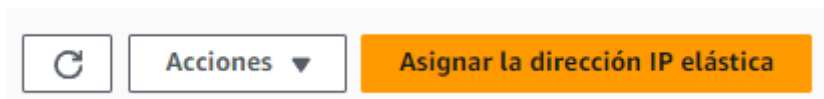
Para esto vamos a generar un ip elástica para que nuestra instancia mantenga siempre la misma ip y el mismo dns de aws.

Volvemos al panel de AWS, en el apartado de **Red y Seguridad**.

Entramos en **Direcciones IP elásticas**

- ▼ Red y seguridad
- Security Groups
- Direcciones IP elásticas**
- Grupos de ubicación
- Pares de claves
- Interfaces de red

Pulsamos el botón naranja Asignar la dirección IP elástica”



Mantenemos la configuración como está y le damos a asignar

Configuraciones de la dirección IP elástica [Información](#)

Grupo de borde de red [Información](#)

Grupo de direcciones IPv4 públicas

- ☒ Grupo de direcciones IPv4 de Amazon
 - ☐ Dirección IPv4 pública que utiliza en la cuenta de AWS con BYOIP. (opción deshabilitada porque no se encontraron grupos) [Más información](#)
 - ☐ Conjunto de direcciones IPv4 propiedad del cliente creado a partir de la red local para su uso con un Outpost. (opción deshabilitada porque no se encontraron grupos propiedad del cliente) [Más información](#)

Direcciones IP estáticas globales

AWS Global Accelerator puede proporcionar direcciones IP estáticas globales que se anuncian en todo el mundo mediante difusión por proximidad desde ubicaciones de borde de AWS. Esto puede ayudar a mejorar la disponibilidad y la latencia del tráfico de usuarios mediante el uso de la red global de Amazon. [Más información](#)

Crear acelerador

Etiquetas: *opcional*

Las etiquetas son marcas que se asignan a un recurso de AWS. Cada etiqueta consta de una clave y un valor opcional. Puede utilizarlas para buscar y filtrar los recursos, o para realizar un seguimiento de sus costos de AWS.

No hay etiquetas asociadas a este recurso.

Agregar nueva etiqueta

Puede agregar hasta 50 etiqueta más

Cancelar

Asignar

Una vez generada marcamos la dirección ip y entre las acciones le damos a “Asociar la dirección IP elástica”

Direcciones IP elásticas (1/1) [🔄](#)

☒	Name	Dirección IPv4 asig...	Tipo
☒	-	54.156.245.113	IP pública

Acciones [▲](#)

Asignar la dirección IP elástica

Ver los detalles

Liberar direcciones IP elásticas

Asociar la dirección IP elástica

Desasociar la dirección IP elástica

Actualizar DNS inverso

Activar transferencias

Desactivar transferencias

Aceptar transferencias

1 > ⓘ
Registro DNS i

Marcamos la instancia que hemos creado y le damos a “Asociar”

Asociar la dirección IP elásticaInformación
Elegir la instancia o la interfaz de red que se desea asociar a esta dirección IP elástica (54.156.245.113)

Dirección IP elástica: 54.156.245.113

Tipo de recurso
Elija el tipo de recurso al que desea asociar la dirección IP elástica.
☒ Instancia
☐ Interfaz de red

⚠ Si asocia una dirección IP elástica a una instancia que ya tiene asociada una dirección IP elástica, la dirección IP elástica asociada anteriormente se desasociará, pero la dirección seguirá asignada a la cuenta. [Más información](#)

Si no se especifica ninguna dirección IP privada, la dirección IP elástica se asociará a la dirección IP privada principal.

Instancia

Dirección IP privada
La dirección IP privada a la que desea asociar la dirección IP elástica.

Nueva asociación
Especifique si la dirección IP elástica se puede volver a asociar a un recurso diferente en el caso de que ya exista otra asociación.
☐ Permitir que se vuelva a asociar esta dirección IP elástica

Ya hemos asignado una dirección ip elástica a nuestra instancia. Reinicia la instancia para asegurarte de que esta se asocia correctamente.

No olvides apagar el laboratorio cuando termines.

7.2 Despliegue en local con Docker



7.2.1 Instalación de docker

7.2.1.1 Linux:

Para instalar docker en linux abrimos una terminal y ejecutamos el siguiente comando:

sudo apt install docker.io

Y para poder usar docker sin sudo metemos nuestro usuario en el grupo de docker con este comando:

sudo usermod -aG docker NuestroUsuario

Y posteriormente reiniciamos nuestra máquina para que se aplique el cambio de grupo a nuestro usuario:

sudo reboot

7.2.1.2. Para Windows:

Para instalar docker en windows nos ayudamos de este tutorial:

<https://youtu.be/ZO4KWQfUBBc?si=ii53FktLvmKTIXQ8>

Siguiendo el tutorial vamos a descargar docker con wsl (Windows Subsystems for Linux).

Para ello nos ayudaremos de las documentaciones oficiales:

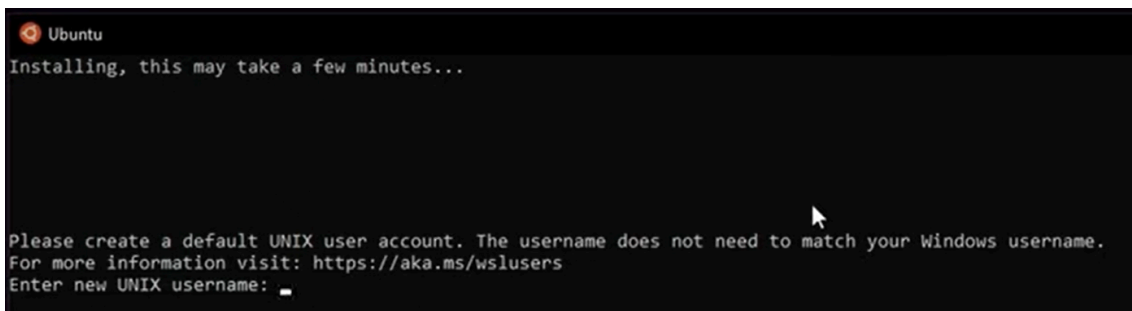
- documentación oficial de docker:
<https://docs.docker.com/desktop/install/windows-install/>
- documentación oficial de microsoft:
[Install WSL | Microsoft Learn](#)

Comenzaremos por instalar WSL con el siguiente comando en una consola powershell:

wsl --install

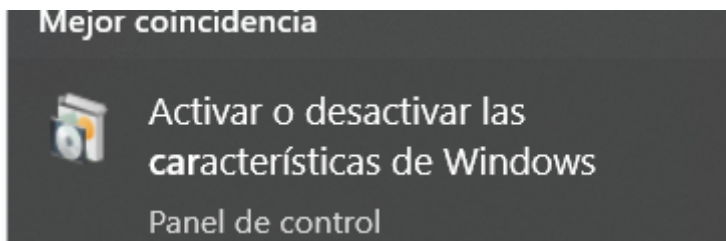
```
PS C:\Users\damag> wsl --install
```

Reiniciamos y se nos abre la ventana de que se está instalando wsl y luego nos pide usuario y contraseña:

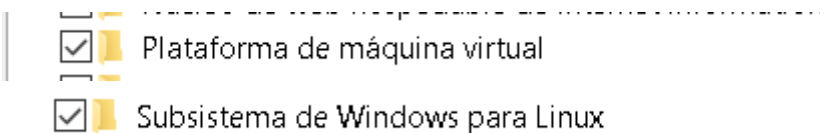


Rellenamos y ya tenemos wsl en nuestro windows.

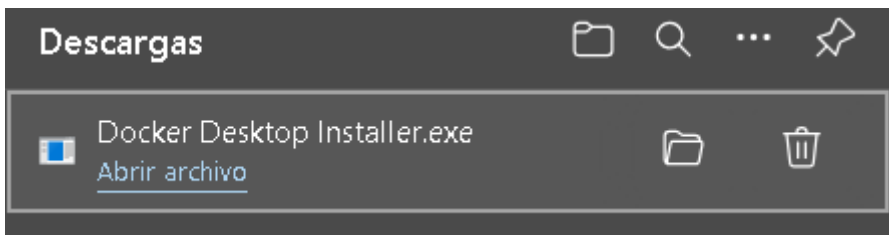
Después de esto seguimos los pasos de la documentación oficial y configuramos la virtualización. Para ello buscamos **features o características**:



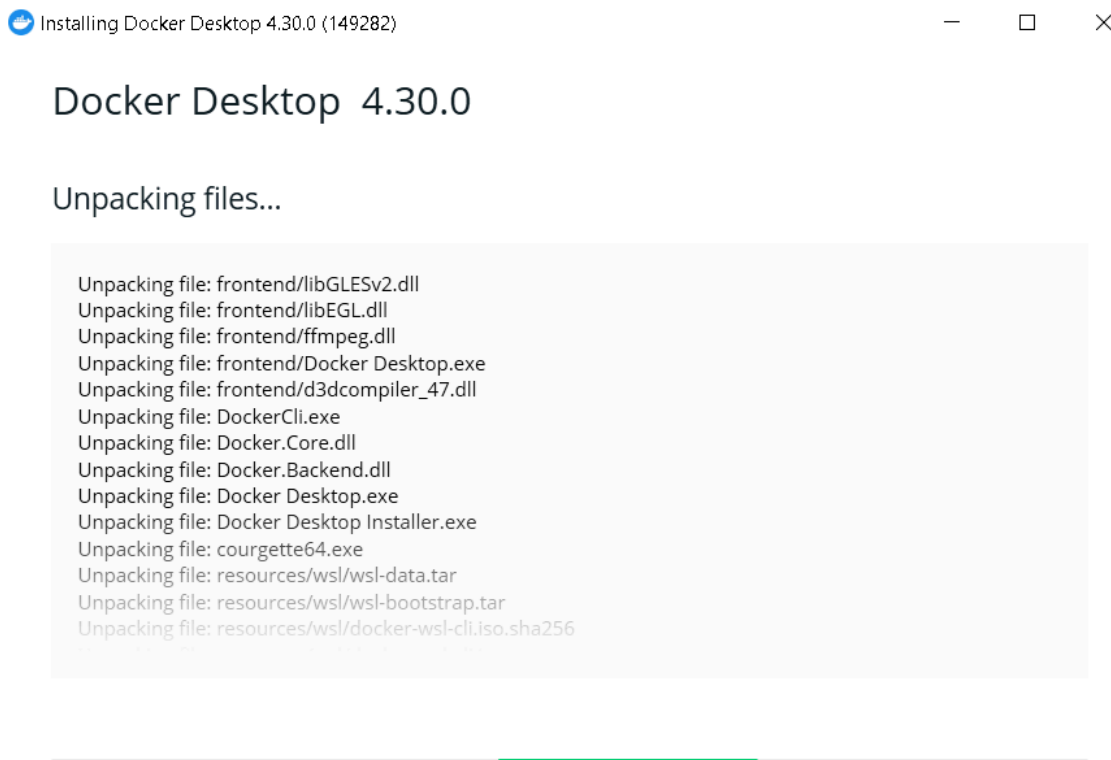
Abrimos y buscamos las opciones de **virtual machine platform o plataforma de máquina virtual y Windows Subsystems for Linux o Subsistema de Windows para Linux** y activamos las opciones si no están activadas:



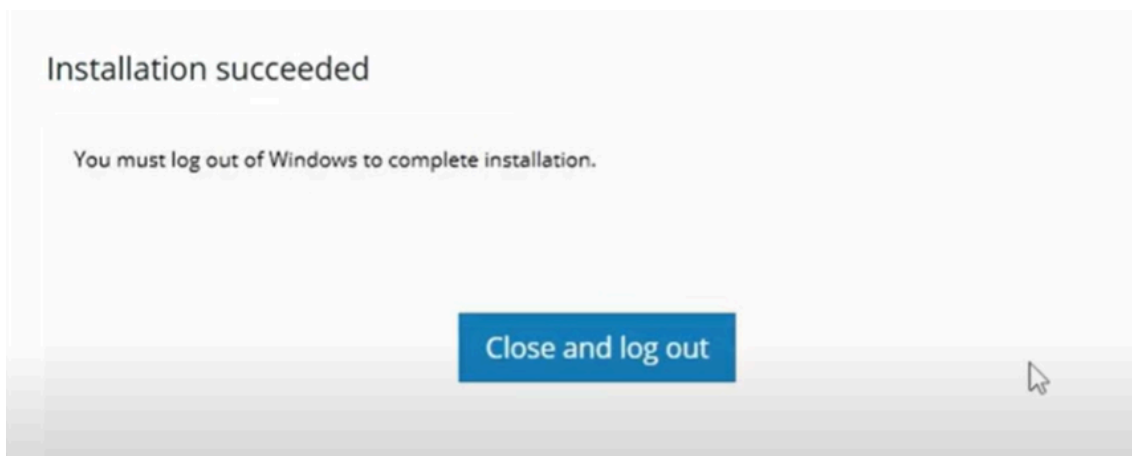
Después de esto ya podremos instalar docker desktop desde la página oficial descargamos el instalador:



Y lo ejecutamos:



Nos saldrá esto al terminar:



Pulsamos en **Close and log out** y se reiniciará la máquina una vez se reinicie nos muestra los términos de docker desktop los aceptamos y ya tendremos docker desktop y podremos ejecutar comandos de docker en la terminal.

Para comprobarlo podemos poner el siguiente comando para ver la versión de docker que tenemos:

docker --version

```
C:\Users\damag>docker --version
Docker version 26.1.1, build 4cf5afa
```

7.2.2 Preparación para el despliegue

Para preparar el despliegue de nuestra aplicación en docker comenzamos por crear una carpeta para docker en la que pusimos lo siguiente:

- Un readme con comandos de interés de docker a la hora de desplegar llamado **readme_comandos_docker.md**.

Ejecutar Docker Compose para Generar o Iniciar los Contenedores

```
docker-compose -p contenedor-gymtracker up --build
```

Este comando genera los contenedores de GymTracker y los inicia. Si es la primera vez que se ejecuta este comando o si se realizaron cambios en la configuración, se reconstruirán las imágenes de Docker antes de iniciar los contenedores.

Parar Contenedores

Para detener los contenedores de GymTracker, presiona Ctrl + C en la terminal donde se están ejecutando.

Ver Contenedores

```
docker ps -a
```

Este comando muestra una lista de todos los contenedores, incluidos los que están detenidos.

Ver Volúmenes

```
docker volume ls
```

Este comando muestra una lista de todos los volúmenes Docker en el sistema.

Acceder a la Base de Datos del Contenedor

Para acceder a la base de datos del contenedor de la base de datos de GymTracker, primero accede al shell del contenedor y luego ingresa a MySQL:

```
docker exec -it contenedor-gymtracker_database_1 /bin/bash
mysql -u root -p
```

La contraseña predeterminada es "root_password".

Acceder a los Logs de los Contenedores de la app y la api

Puedes acceder a los logs de los contenedores de la aplicación y la API de GymTracker de la siguiente manera:

```
docker logs contenedor-gymtracker_app_1
docker logs contenedor-gymtracker_api_1
```

Borrar Contenedores de GymTracker

```
docker rm contenedor-gymtracker_database_1 contenedor-gymtracker_app_1 contenedor-gymtracker_api_1
```

Este comando elimina los contenedores específicos de GymTracker.

Borrar Volumen de GymTracker

```
docker volume rm contenedor-gymtracker_db_data
```

Este comando elimina el volumen llamado "contenedor-gymtracker_db_data", utilizado por la base de datos de GymTracker.

- Los jars, el jar de la api de ejercicios y el jar de la app de GymTracker.

- **Los Dockerfile** para cada jar para crear una imagen a partir de la cuál crear el contenedor para la api y para la app.
 - **Dockerfile.api:**

```

1  # Dockerfile para la API
2  FROM openjdk:17-jdk-slim
3  COPY Api-Ejercicios-0.0.1-SNAPSHOT.jar /api/api.jar
4  WORKDIR /api
5  CMD ["java", "-jar", "api.jar"]

```

FROM openjdk:17-jdk-slim:

Esta línea especifica la imagen base que se utilizará para construir la nueva imagen. En este caso, se utiliza una imagen de OpenJDK 17 en su versión "slim", que es una versión más ligera y optimizada.

COPY Api-Ejercicios-0.0.1-SNAPSHOT.jar /api/api.jar:

Esta línea copia el archivo Api-Ejercicios-0.0.1-SNAPSHOT.jar desde el sistema de archivos del host (donde se encuentra el Dockerfile) a la ruta /api/api.jar dentro de la imagen Docker. Este archivo es el ejecutable de la API.

WORKDIR /api:

Esta línea establece el directorio de trabajo dentro del contenedor en /api. Esto significa que cualquier comando que se ejecute después de esta línea se ejecutará dentro de este directorio.

CMD ["java", "-jar", "api.jar"]:

Esta línea especifica el comando que se ejecutará cuando se inicie un contenedor basado en esta imagen. En este caso, se ejecuta el comando `java -jar api.jar` para iniciar la API.

- **Dockerfile.app:**

```

1  # Dockerfile para la aplicación
2  FROM openjdk:17-jdk-slim
3
4  # Copia el archivo JAR de tu aplicación
5  COPY GymTracker-0.0.1-SNAPSHOT.jar /app/app.jar
6
7  # Copia el script wait-for-it
8  COPY wait-for-it.sh /app/wait-for-it.sh
9
10 # Para que el script wait-for-it.sh tenga permisos de ejecución
11 RUN chmod +x /app/wait-for-it.sh
12
13 # Establecer el directorio de trabajo
14 WORKDIR /app
15
16 # Comando para iniciar la aplicación, esperando a que la base de datos esté lista
17 CMD ["/wait-for-it.sh", "database:3306", "--", "java", "-jar", "app.jar"]

```

Este Dockerfile crea una imagen Docker para la aplicación Java GymTracker, asegurándose de que la aplicación espere hasta que la base de datos esté lista antes de iniciarse.

FROM openjdk:17-jdk-slim:

Usa una imagen base ligera de OpenJDK 17 para ejecutar la aplicación Java.

COPY GymTracker-0.0.1-SNAPSHOT.jar /app/app.jar:

Copia el archivo JAR de la aplicación al contenedor.

COPY wait-for-it.sh /app/wait-for-it.sh:

Copia el script wait-for-it.sh al contenedor para esperar a que la base de datos esté lista.

RUN chmod +x /app/wait-for-it.sh:

Hace que el script sea ejecutable.

WORKDIR /app:

Establece /app como el directorio de trabajo dentro del contenedor.

CMD ["/wait-for-it.sh", "database:3306", "--", "java", "-jar", "app.jar"]:

Usa wait-for-it.sh para esperar a que la base de datos esté disponible en database:3306 antes de iniciar la aplicación con java -jar app.jar.

Uso del script wait-for-it.sh

El script wait-for-it.sh asegura que la aplicación GymTracker no intente conectarse a la base de datos antes de que esta esté lista, evitando errores de conexión. Esto nos sirvió para que el Docker Compose pudiera ejecutar el contenedor de la base de datos y la aplicación simultáneamente.

- **Script wait-for-it:**

El script `wait-for-it` lo obtuvimos de GitHub, y puedes encontrarlo en el siguiente enlace: <https://github.com/vishnubob/wait-for-it>. Este script nos ayudó a solucionar el error que ocurría al hacer el `docker-compose up`, donde la aplicación no se iniciaba correctamente porque dependía de que la base de datos estuviera disponible primero. Utilizando `wait-for-it`, pudimos hacer que nuestra aplicación esperará hasta que la base de datos estuviera lista antes de intentar iniciarse, asegurando así un inicio ordenado y sin errores.

- Archivo para la inicialización de la base de datos **init.sql**.
- **docker-compose.yml** :

Este archivo docker-compose.yml lo hemos creado para desplegar la aplicación Gymtracker. Configura tres contenedores: una base de datos MySQL, la API de ejercicios y la aplicación principal. A continuación, una explicación breve de cada sección:

Versión:

```
version: '3.8'
```

- Especificamos la versión del formato de **docker-compose**.

Servicios

Base de datos (database)

```
services:
  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: gymtracker
      MYSQL_USER: dad
      MYSQL_PASSWORD: padre
    ports:
      - "3307:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - app-network
```

- Usamos MySQL 8.0 con configuraciones de entorno para la base de datos.
- Mapeamos el puerto 3306 del contenedor al 3307 del host.
- Definimos volúmenes para la persistencia de datos y la inicialización.
- Conectamos a la red app-network.

API (api)

```
api:
  build:
    context: .
    dockerfile: Dockerfile.api
  ports:
    - "8081:8081"
  networks:
    - app-network
```

- Construimos la imagen usando Dockerfile.api.
- Mapeamos el puerto 8081 del contenedor al 8081 del host.

- Conectamos a la red app-network.

Aplicación (app)

```
app:
  build:
    context: .
    dockerfile: Dockerfile.app
  depends_on:
    - database
    - api
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://database:3306/gymtracker
    SPRING_DATASOURCE_USERNAME: dad
    SPRING_DATASOURCE_PASSWORD: padre
    SERVER_PORT: 443
  ports:
    - "443:443"
  networks:
    - app-network
```

- Construimos la imagen usando Dockerfile.app.
- Establecemos dependencias para que la aplicación espere a que database y api estén listas.
- Configuramos variables de entorno para la conexión a la base de datos.
- Mapeamos el puerto 443 del contenedor al 443 del host.
- Conectamos a la red app-network.

Redes y Volúmenes

```
networks:
  app-network:
    driver: bridge
volumes:
  db_data:
```

- Definimos una red app-network para la comunicación entre servicios.
- Definimos un volumen db_data para la persistencia de datos de la base de datos.

- Con este docker-compose.yml, aseguramos que la aplicación GymTracker espere a que la base de datos y la API estén listas antes de iniciar, previniendo errores de dependencias.

Una vez hemos hecho la carpeta ya está todo listo para el despliegue.

7.2.3 Despliegue

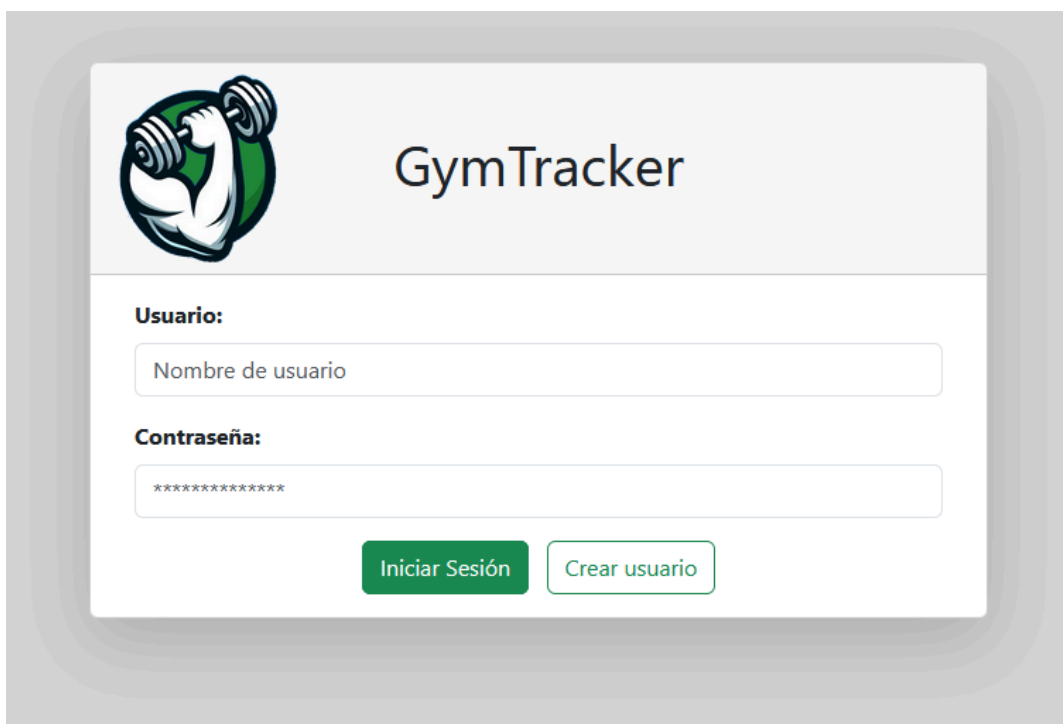
Tanto en linux como windows:

1. Entrar en el github de GymTracker:
[deiivvv/GymTracker: TFG de Alvaro Gallego, Daniel Marin y David Martin \(github.com\)](https://github.com/deiivvv/GymTracker)
2. Localiza el archivo ZIP llamado despliegue_docker.zip en el repositorio y descargalo.
3. Descomprime el archivo ZIP en la ubicación deseada.
4. Abre la carpeta descomprimida y verás el archivo readme_comandos_docker.md en el que tienes los comandos para despliegue y mantenimiento del mismo.
5. Abre una terminal en la carpeta y ejecuta el siguiente comando:

docker-compose -p contenedor-gymtracker up --build

Y ya tendrás GymTracker listo para acceder en tu navegador desde la siguiente dirección:

https://localhost



7.3 Despliegue en local con Spring Tool Suite (STS)

7.3.1 Prerrequisitos

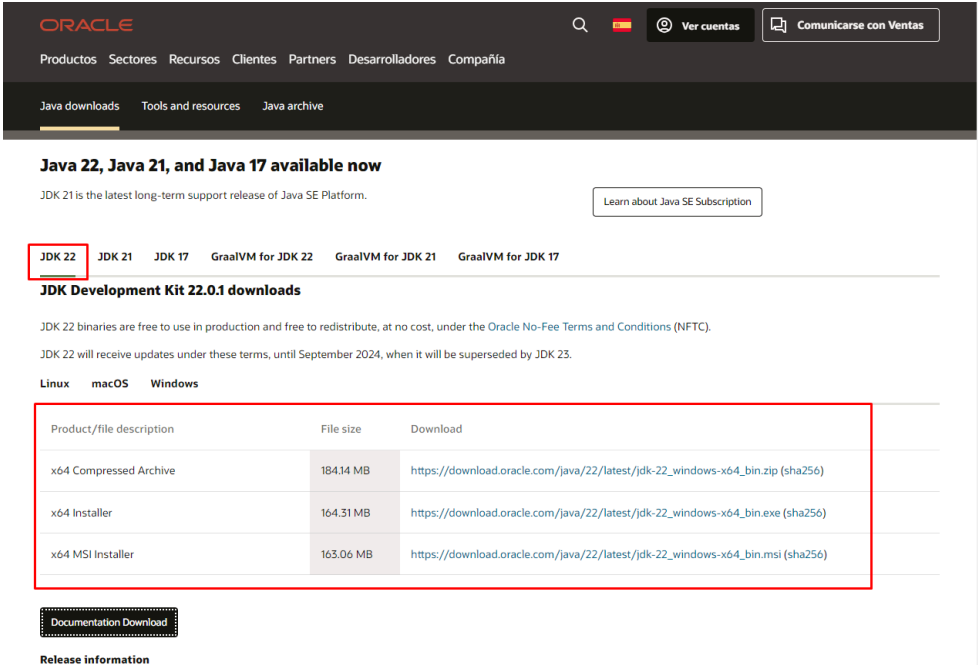
1. **Java Development Kit (JDK):** Asegúrate de tener instalado JDK (Java Development Kit) 17 o superior.
2. **Git:** Necesitas Git para clonar el repositorio desde GitHub.
3. **Spring Tool Suite (STS):** Un IDE recomendado para el desarrollo de aplicaciones Spring.
4. **MySQL:** MySQL es un sistema de gestión de bases de datos relacional de código abierto.

7.3.1.1 Instalación de Prerrequisitos

Windows

7.3.1.1.1 Instalar JDK

Descarga el JDK desde [Oracle JDK](#) o [OpenJDK](#).
Sigue las instrucciones del instalador.



ORACLE

Productos Sectores Recursos Clientes Partners Desarrolladores Compañía

Java downloads Tools and resources Java archive

Java 22, Java 21, and Java 17 available now

JDK 21 is the latest long-term support release of Java SE Platform. [Learn about Java SE Subscription](#)

JDK 22 JDK 21 JDK 17 GraalVM for JDK 22 GraalVM for JDK 21 GraalVM for JDK 17

JDK Development Kit 22.0.1 downloads

JDK 22 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).
JDK 22 will receive updates under these terms, until September 2024, when it will be superseded by JDK 23.

Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	184.14 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.zip (sha256)
x64 Installer	164.31 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.06 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.msi (sha256)

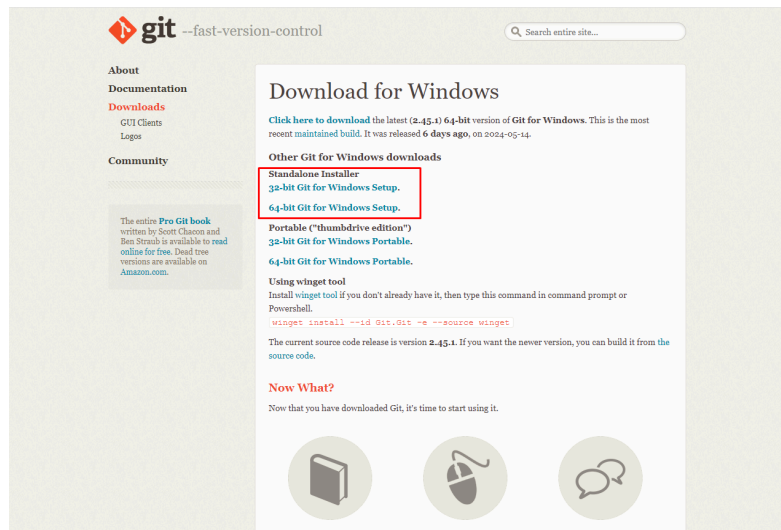
[Documentation Download](#)

Release information

7.3.1.1.2 Instalar Git

Descarga Git desde [Git](#).

Sigue las instrucciones del instalador.



7.3.1.1.3 Instalar Spring Tool Suite (STS)

Descarga STS desde [Spring Tool Suite](#).

Sigue las instrucciones del instalador.

Spring Tools 4 for Eclipse

The all-new Spring Tool Suite 4. Free. Open source.

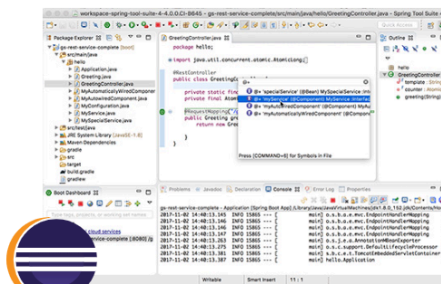
4.22.1 - LINUX X86_64

4.22.1 - LINUX ARM_64

4.22.1 - MACOS X86_64

4.22.1 - MACOS ARM_64

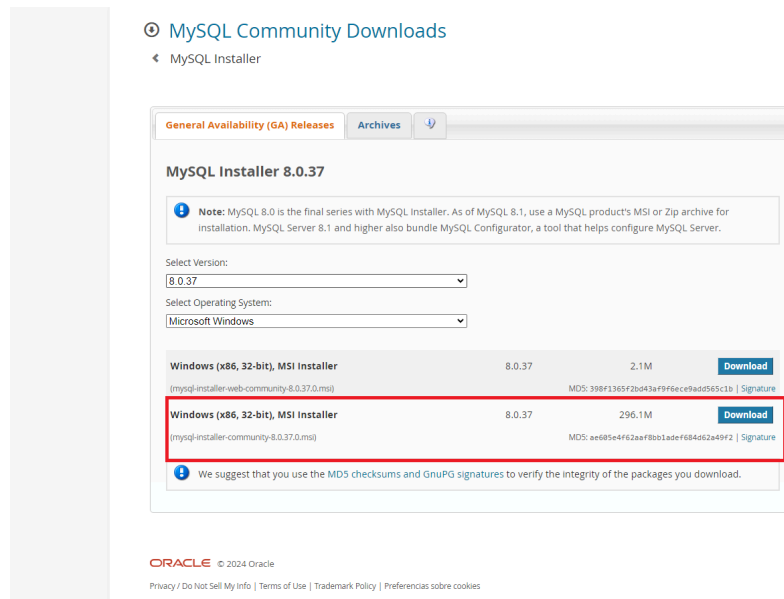
4.22.1 - WINDOWS X86_64



7.3.1.1.4 Instalar MySQL

Para instalar MySQL en Windows:

1. Descargar MySQL Installer desde [MySQL](#).

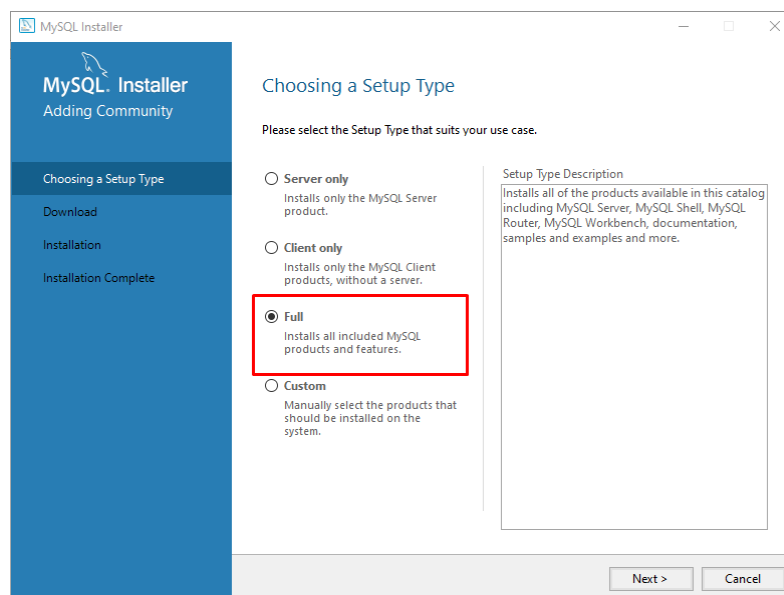


2. Ejecutar el instalador:

- Abre el archivo descargado.

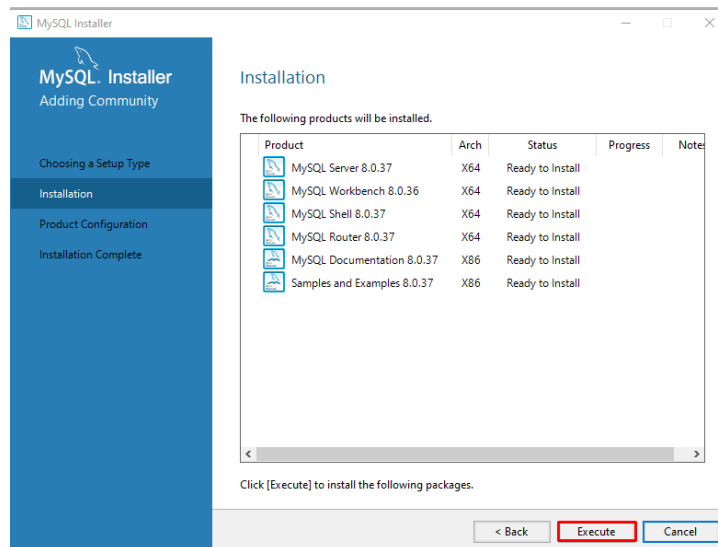
3. Seleccionar el tipo de instalación.

- Elige "Full" para una instalación completa.
- Haz clic en "Next".



4. Instalar las dependencias.

- El instalador verificará y descargará las dependencias necesarias.
- Haz clic en "Execute" para proceder.

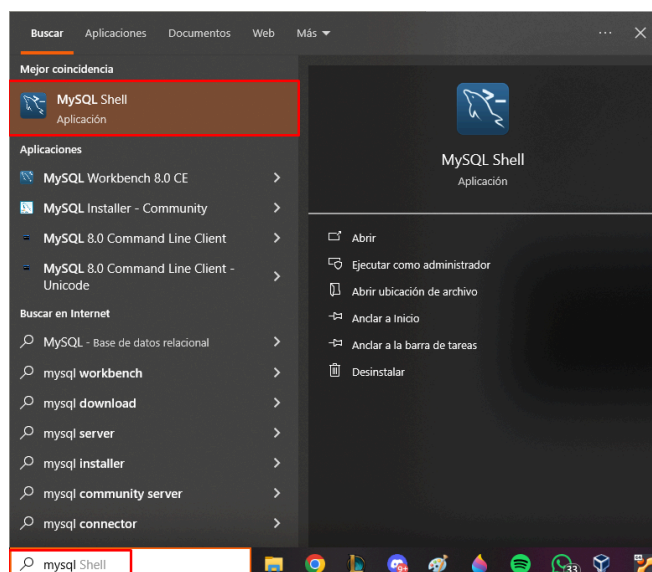


5. Configuración del servidor MySQL.

- Elige el tipo de configuración (General o Server Only).
- Configura el tipo y puerto de red (generalmente, puedes dejar las opciones predeterminadas).
- Establece una contraseña para el usuario `root` y, opcionalmente, crea usuarios adicionales.
- Configura el servicio de Windows si deseas que MySQL se inicie automáticamente con el sistema.

6. Finalizar la instalación.

- Una vez finalizada la instalación, puedes abrir MySQL Workbench o MySQL Shell para empezar a trabajar con MySQL.



```

MySQL Shell
MySQL Shell 8.0.37

Copyright (c) 2016, 2024, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its affiliates.
Other names may be trademarks of their respective owners.

Type '\help' or '\?' for help; '\quit' to exit.
MySQL 35 >

```

Linux

7.3.1.2.1 Instalar JDK

Compruebe la versión de java que tiene

java -version

En caso de que no tenga o tenga una inferior al jdk-17, instale y/o actualice su jdk

sudo apt update

sudo apt install openjdk-19-jdk (última versión disponible)

7.3.1.2.2 Instalar Git

sudo apt install git

7.3.1.2.3 Instalar Spring Tool Suite (STS)

Descarga STS desde [Spring Tool Suite](#).

Extrae el archivo y sigue las instrucciones de instalación correspondientes para tu distribución.

Spring Tools 4 for Eclipse

The all-new Spring Tool Suite 4. Free. Open source.

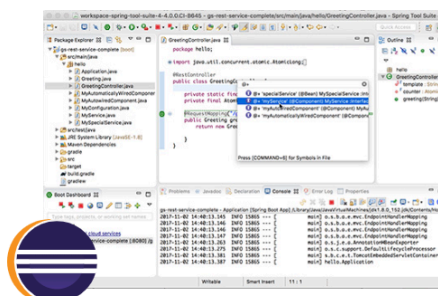
4.22.1 - LINUX X86_64

4.22.1 - LINUX ARM_64

4.22.1 - MACOS X86_64

4.22.1 - MACOS ARM_64

4.22.1 - WINDOWS X86_64

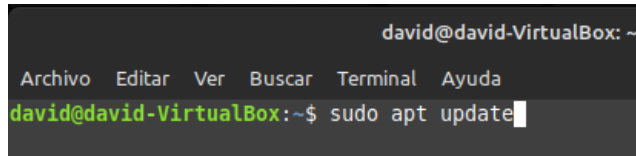


7.3.1.2.4 Instalar MySQL

Para instalar MySQL en Linux (Debian)

1. Actualizar los repositorios:

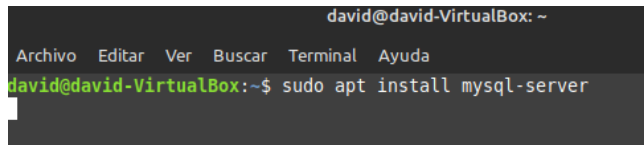
sudo apt update



```
david@david-VirtualBox: ~  
Archivo Editar Ver Buscar Terminal Ayuda  
david@david-VirtualBox:~$ sudo apt update
```

2. Instalar MySQL:

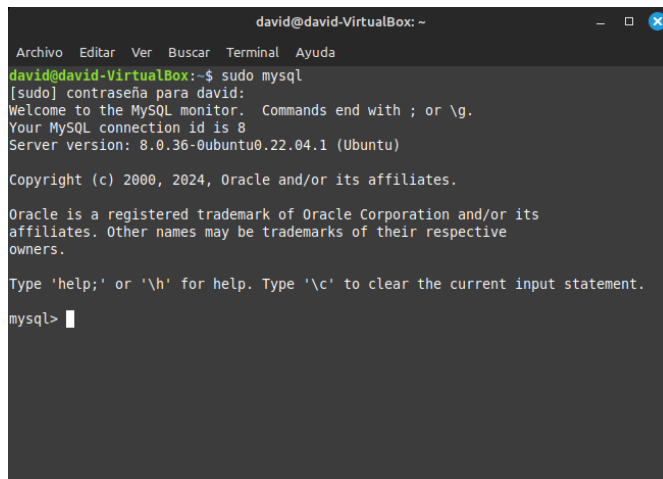
sudo apt install mysql-server



```
david@david-VirtualBox: ~  
Archivo Editar Ver Buscar Terminal Ayuda  
david@david-VirtualBox:~$ sudo apt install mysql-server
```

3. Iniciamos MySQL para comprobar que funciona:

sudo mysql



```
david@david-VirtualBox: ~  
Archivo Editar Ver Buscar Terminal Ayuda  
david@david-VirtualBox:~$ sudo mysql  
[sudo] contraseña para david:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.36-0ubuntu0.22.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2024, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql>
```

Estos pasos deberían permitirte instalar y configurar MySQL en tu sistema Linux.

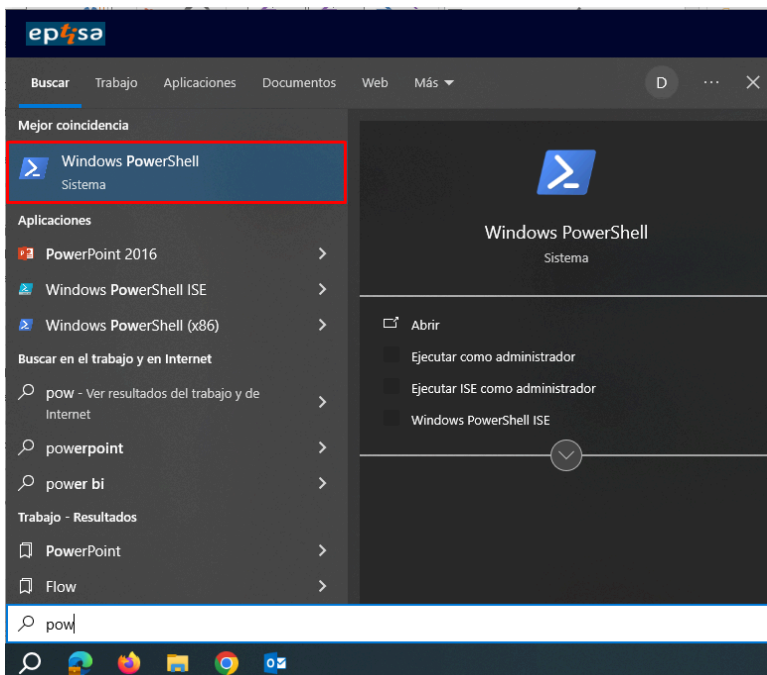
7.3.2 Clonar el Repositorio desde GitHub

Windows

En una terminal de powershell (pulsamos boton de windows y escribimos powershell), nos posicionamos en el directorio donde queremos que se clone el repositorio, podemos movernos con el comando **cd**, una vez estemos situados en nuestro destino, ejecutamos el siguiente comando para clonar el repositorio:

git clone https://github.com/tu-usuario/GymTracker.git

Reemplaza `tu-usuario` con el nombre de tu usuario en GitHub.



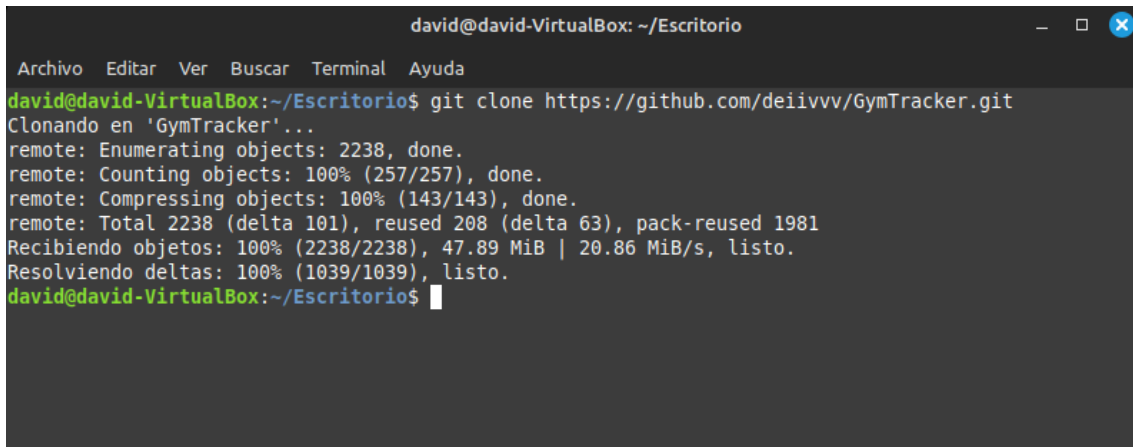
```
PS C:\Users\davidmartin\Desktop> cd TFG
PS C:\Users\davidmartin\Desktop\TFG> git clone https://github.com/tu-usuario/GymTracker.git
>>
```

Linux

Abrimos una terminal, nos posicionamos en el directorio donde queremos que se clone el repositorio, podemos movernos con el comando **cd**, una vez estemos situados en nuestro destino, ejecutamos el siguiente comando para clonar el repositorio:

git clone https://github.com/tu-usuario/GymTracker.git

Reemplaza `tu-usuario` con el nombre de tu usuario en GitHub.

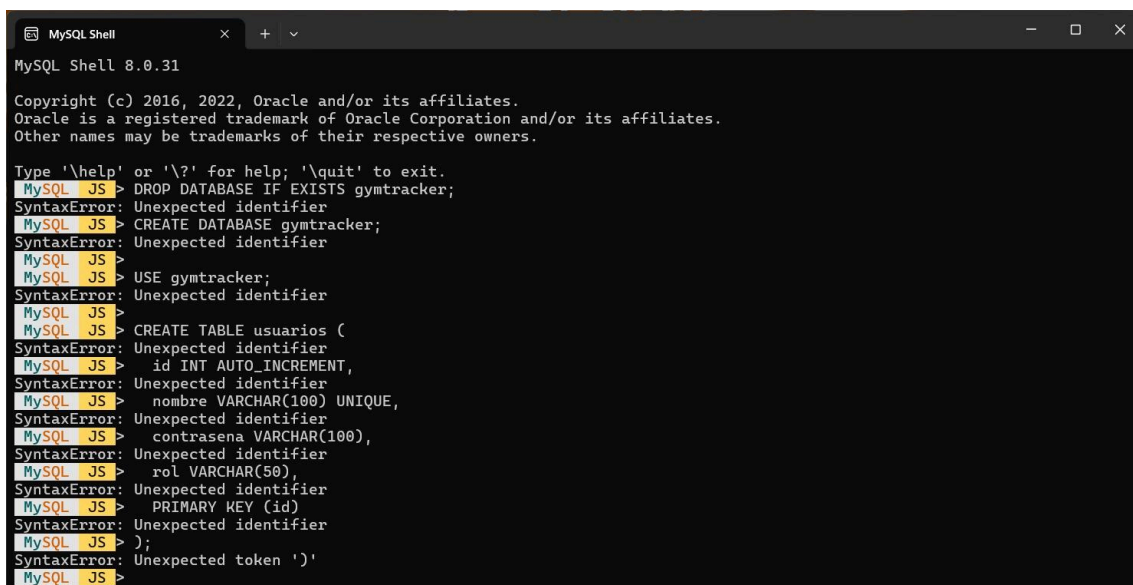


```
david@david-VirtualBox: ~/Escritorio
Archivo Editar Ver Buscar Terminal Ayuda
david@david-VirtualBox:~/Escritorio$ git clone https://github.com/deiivvv/GymTracker.git
Clonando en 'GymTracker'...
remote: Enumerating objects: 2238, done.
remote: Counting objects: 100% (257/257), done.
remote: Compressing objects: 100% (143/143), done.
remote: Total 2238 (delta 101), reused 208 (delta 63), pack-reused 1981
Recibiendo objetos: 100% (2238/2238), 47.89 MiB | 20.86 MiB/s, listo.
Resolviendo deltas: 100% (1039/1039), listo.
david@david-VirtualBox:~/Escritorio$
```

7.3.3 Instalar la base de datos

Windows

Abrimos la carpeta clonada de Github, entramos en la carpeta BBDD ya abrimos el archivo Gymtracker.sql y copiamos todo el contenido. Pegamos el contenido en nuestro MySql Shell.



```
MySQL Shell
MySQL Shell 8.0.31
Copyright (c) 2016, 2022, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its affiliates.
Other names may be trademarks of their respective owners.

Type '\help' or '\?' for help; '\quit' to exit.
MySQL JS > DROP DATABASE IF EXISTS gymtracker;
SyntaxError: Unexpected identifier
MySQL JS > CREATE DATABASE gymtracker;
SyntaxError: Unexpected identifier
MySQL JS > USE gymtracker;
SyntaxError: Unexpected identifier
MySQL JS > CREATE TABLE usuarios (
SyntaxError: Unexpected identifier
MySQL JS >   id INT AUTO_INCREMENT,
SyntaxError: Unexpected identifier
MySQL JS >   nombre VARCHAR(100) UNIQUE,
SyntaxError: Unexpected identifier
MySQL JS >   contrasena VARCHAR(100),
SyntaxError: Unexpected identifier
MySQL JS >   rol VARCHAR(50),
SyntaxError: Unexpected identifier
MySQL JS >   PRIMARY KEY (id)
SyntaxError: Unexpected identifier
MySQL JS > );
SyntaxError: Unexpected token ')'
```

Ya tendríamos nuestra base de datos creada.

Linux

Abrimos la carpeta clonada de Github, entramos en la carpeta BBDD y abrimos el archivo Gymtracker.sql y copiamos todo el contenido.

```
tfg@tfg-VirtualBox: ~  
Archivo Editar Ver Buscar Terminal Ayuda  
tfg@tfg-VirtualBox:~$ sudo mysql;  
[sudo] contraseña para tfg:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.36-0ubuntu0.22.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2024, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> DROP DATABASE IF EXISTS gymtracker;  
rutina, 33, @id_serie_flexiones_diamQuery OK, 6 rows affected (0,34 sec)  
  
mysql> CREATE DATABASE gymtracker;  
ante 3),  
(@id_rutina, 37, @iQuery OK, 1 row affected (0,01 sec)  
  
mysql>  
mysql> USE gymtracker;  
d_serie_press_banDatabase changed  
mysql>  
mysql> CREATE TABLE usuarios (  
-> id INT AUTO_INCREMENT,  
-> nombre VARCHAR(100) UNIQUE,  
-> contrasena VARCHAR(100),
```

Ya tendríamos nuestra base de datos creada.

7.3.4 Ejecutar la Aplicación

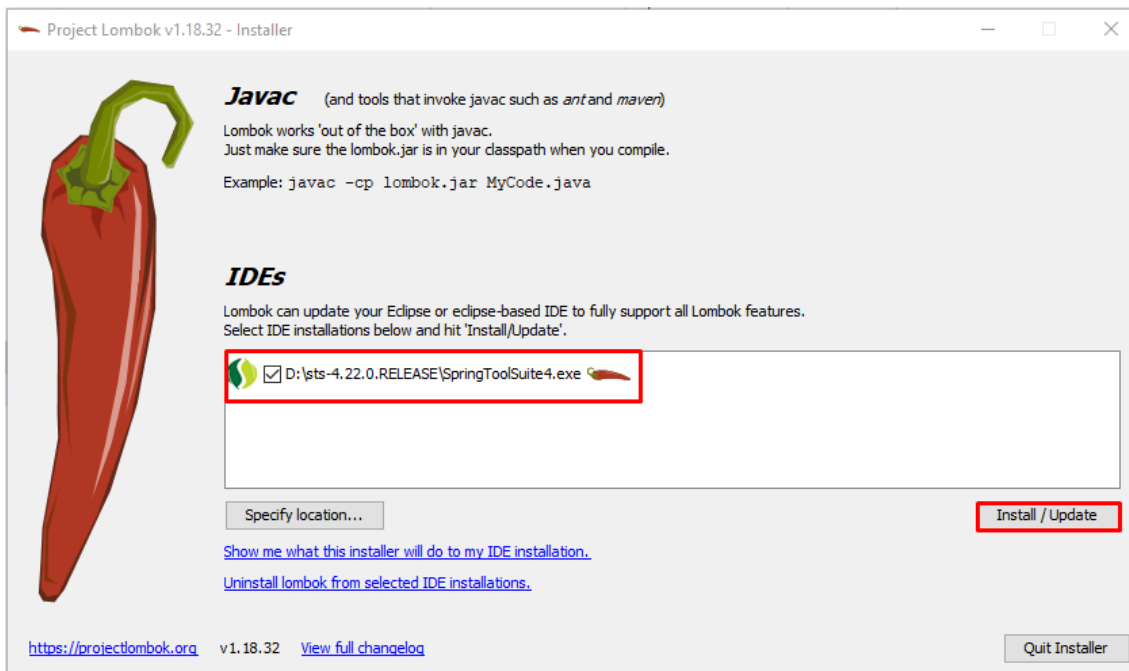
Windows

Una vez que la clonación sea exitosa, primero vamos a instalar **lombok** (Biblioteca Java diseñada para minimizar el código repetitivo que se escribe en las clases Java.). Ahora ya podemos ejecutar la aplicación desde SpringBoot.

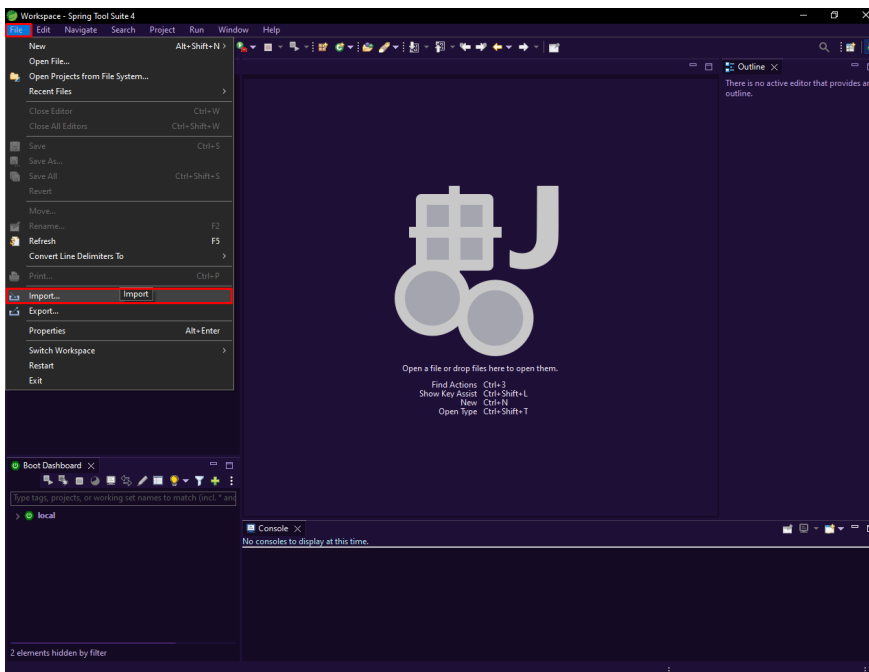
Escritorio > TFG > GymTracker

Nombre	Fecha de modificación	Tipo	Tamaño
Api-Ejercicios	24/04/2024 8:03	Carpeta de archivos	
BBDD	16/05/2024 13:28	Carpeta de archivos	
GymTracker-AppWeb	16/05/2024 13:28	Carpeta de archivos	
Anteproyecto.pdf	24/04/2024 7:53	Documento Adob...	96 KB
README.md	16/05/2024 13:28	Archivo de origen ...	2 KB
lombok.jar	29/04/2024 8:00	Executable Jar File	1.999 KB

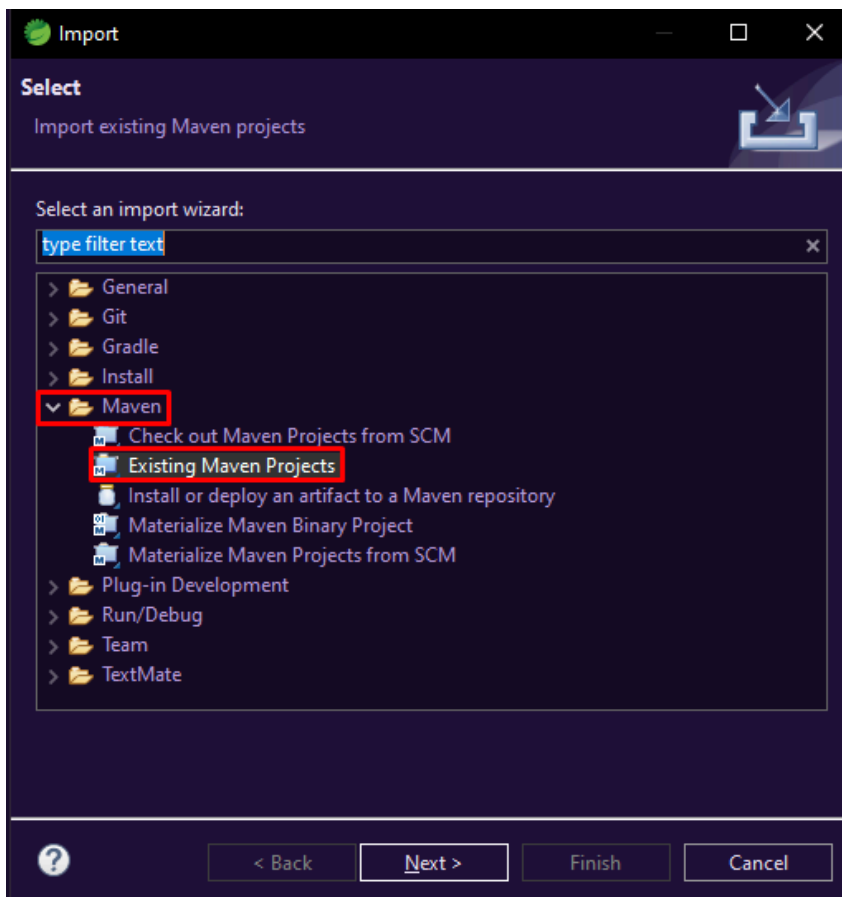
Elegimos la ruta donde se colocar el JAR, lo ideal es dentro de la carpeta sts-4.20.1.RELEASE, seleccionas el archivo SpringToolSuite4.exe (es posible que te salga sin la extension .exe entonces eliges el que no es .ini) y procedemos a instalarlo.



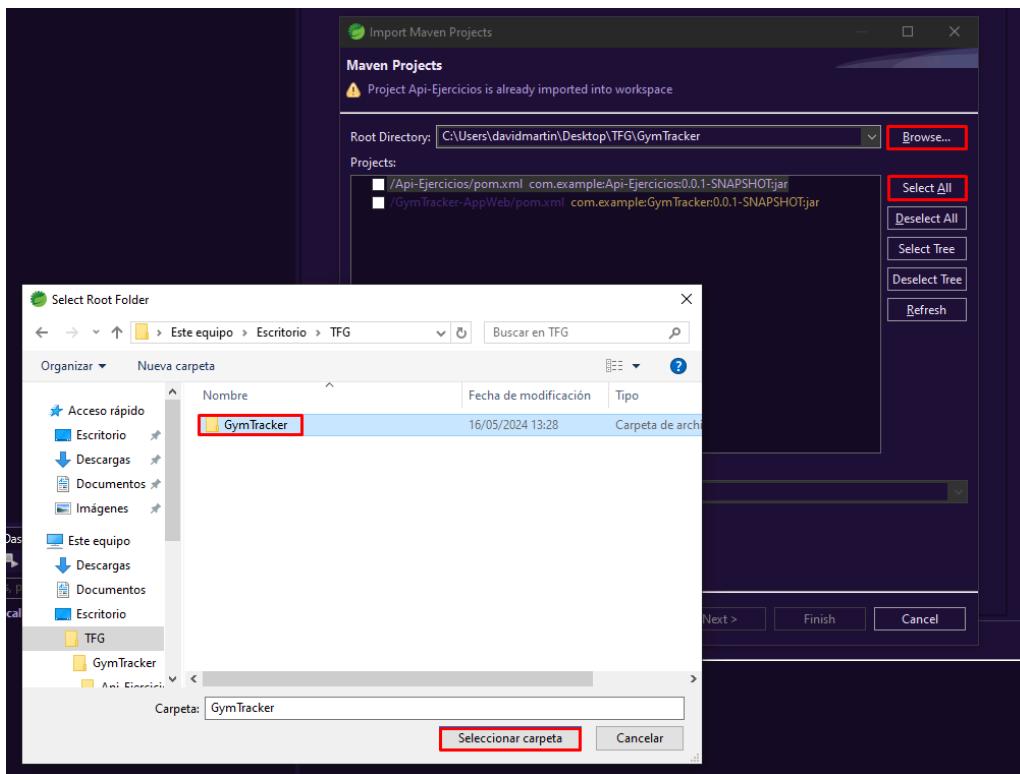
Una vez instalado lombok, iniciamos nuestro STS y lo primero que haremos será hacer click en **file, import**.



Se nos abrirá una nueva pestaña en la que seleccionaremos **maven, Existing Maven Projects**.

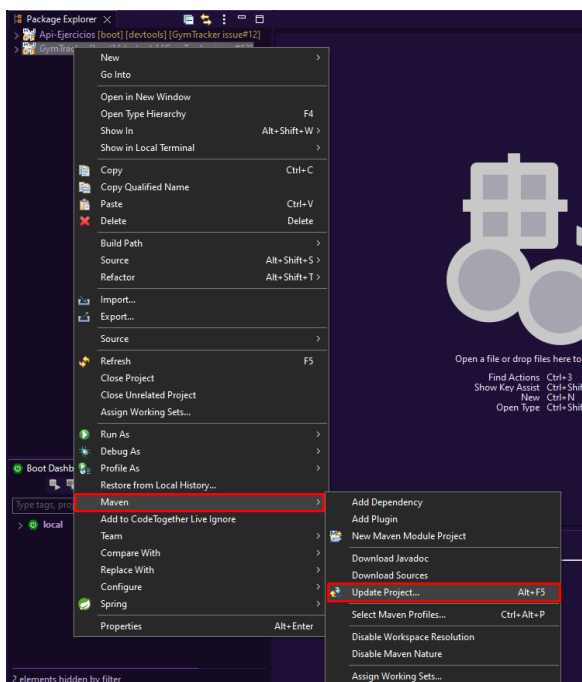


Nos abrirá otra nueva ventana en la que buscaremos nuestro proyecto clonado de GitHub. Para ello pulsamos en **Browse**, nos abrirá una ventana en la que buscaremos nuestra carpeta donde hemos clonado el proyecto. Una vez encontrado pulsamos en **seleccionar carpeta**. En la ventana que nos queda abierta pulsamos **Select All** y **Finish**.



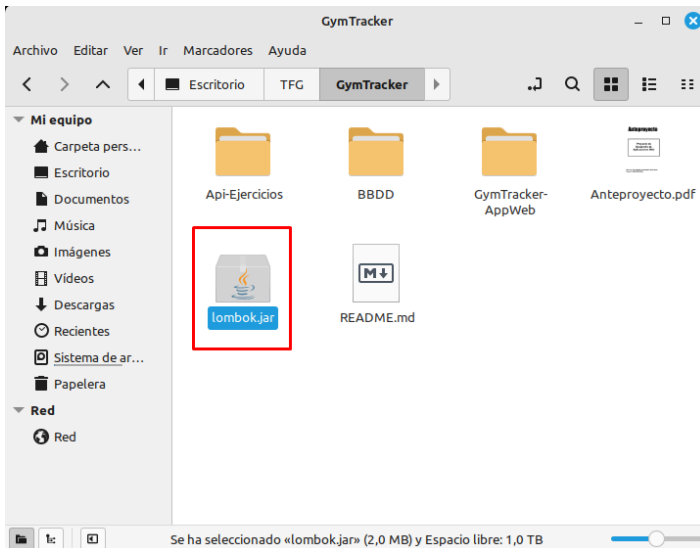
Por último, una vez nos aparezcan los dos proyectos, tenemos que hacer el siguiente paso en los dos.

Hacemos click derecho, vamos a **Maven, Update Project**. Esto debería actualizar el proyecto para que no de problemas con Lombok.

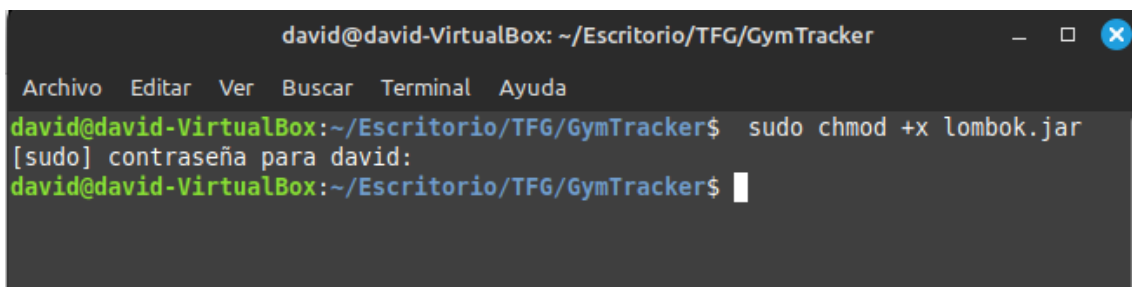


Linux

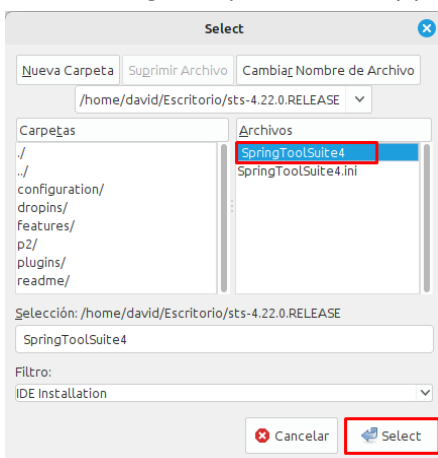
Una vez que la clonación sea exitosa, primero vamos a instalar **lombok** (Biblioteca Java diseñada para minimizar el código repetitivo que se escribe en las clases Java.). Ahora ya podemos ejecutar la aplicación desde SpringBoot.



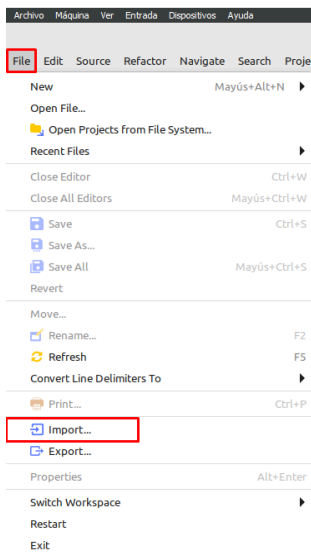
Abrimos una terminal en la carpeta que tenemos el archivo lombok y escribimos el siguiente comando para darle los permisos y pueda ejecutarse. **sudo chmod +x lombok.jar**



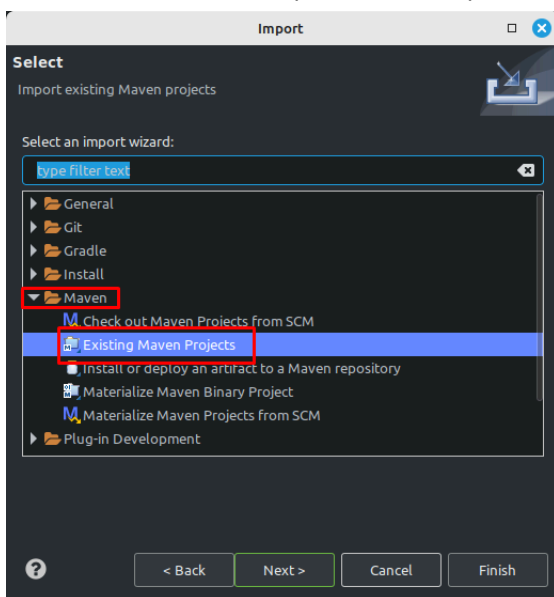
Elegimos la ruta donde se colocar el JAR, lo ideal es dentro de la carpeta sts-4.20.1.RELEASE, seleccionas el archivo SpringToolSuite4.exe (es posible que te salga sin la extensión .exe entonces eliges el que no es .ini) y procedemos a instalarlo.



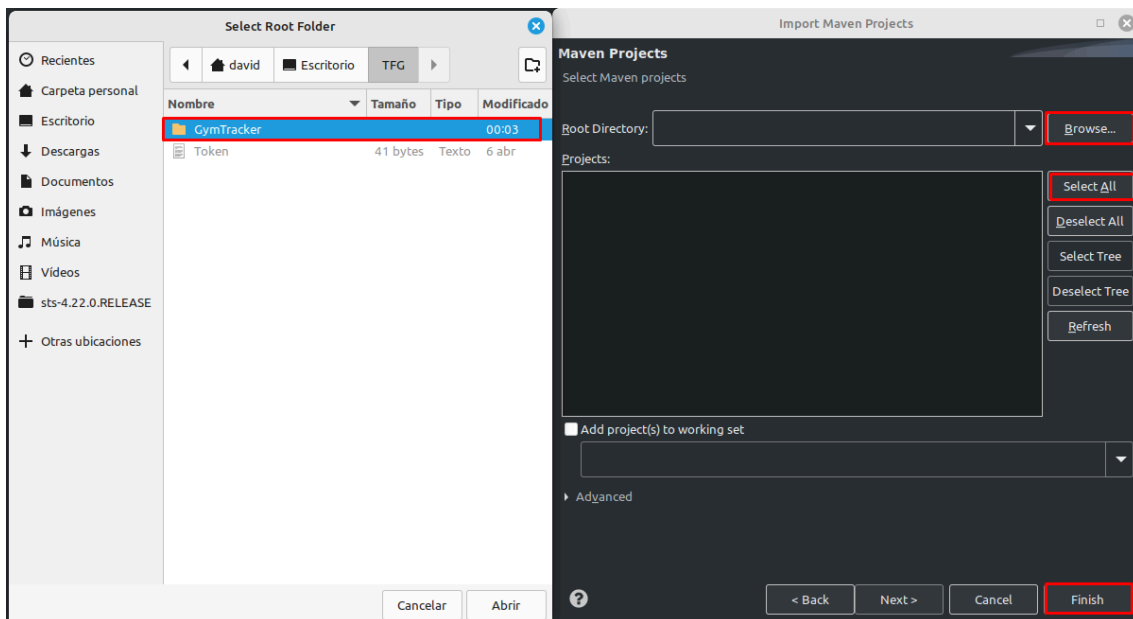
Una vez instalado lombok, iniciamos nuestro STS y lo primero que haremos será hacer click en **file, import.**



Se nos abrirá una nueva pestaña en la que seleccionaremos **maven, Existing Maven Projects.**

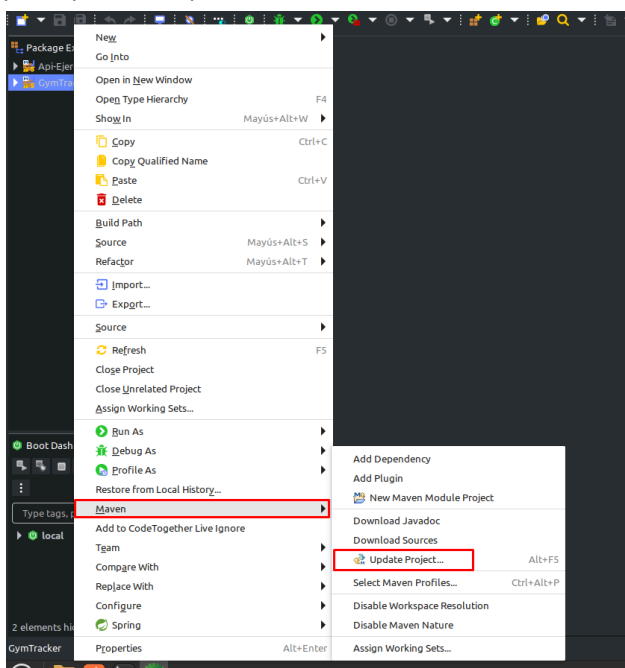


Nos abrirá otra nueva ventana en la buscaremos nuestro proyecto clonado de GitHub. Para ellos pulsamos en **Browse**, nos abrirá una ventana en la que buscaremos nuestra carpeta donde hemos clonado el proyecto. Una vez encontrado pulsamos en **seleccionar carpeta**. En la ventana que nos queda abierta pulsamos **Select All** y **Finish**.



Por último, una vez nos aparezcan los dos proyectos, tenemos que hacer el siguiente paso en los dos.

Hacemos click derecho, vamos a **Maven, Update Project**. Esto debería actualizar el proyecto para que no de problemas con Lombok.

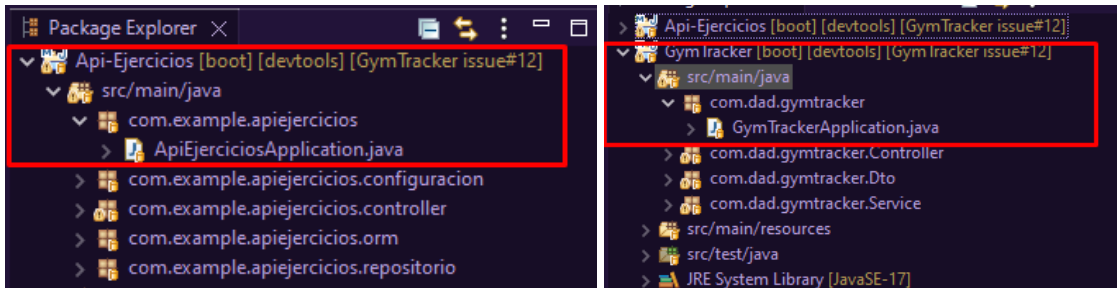


7.3.5 Acceder a la Aplicación

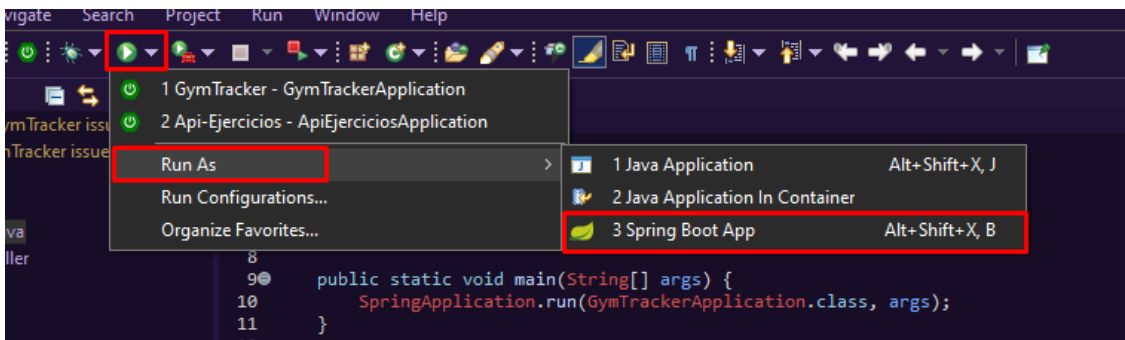
Una vez tengamos todos los pasos anteriores hechos y nuestro proyecto configurado en STS, ya podemos iniciar nuestra aplicación web.

Primero vamos a lanzar los dos proyectos. Debemos hacer el mismo paso en ambos proyectos.

Hacer doble click en **Gymtracker**, **src/main/java**, **com.dad.GymtrackerApplication.java** o **Api-Ejercicios**, **src/main/java**, **com.example.apiejercicios**, **ApiEjercicioApplication.java**.

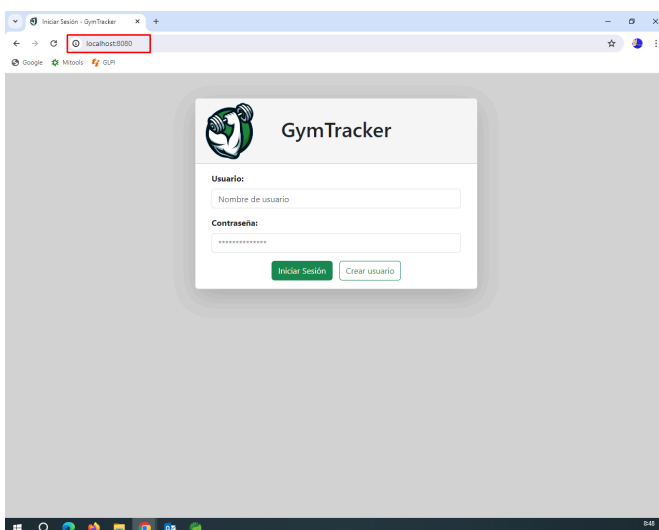


Una vez abierto el archivo, clickaremos en el botón de play verde, y spring app.



Puedes acceder a la aplicación abriendo un navegador web y visitando la siguiente URL:

https://localhost



7.4 Despliegue en local desde Github

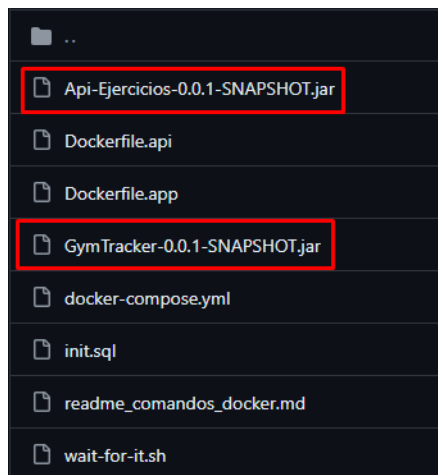
7.4.1 Descarga de los archivos JAR

7.4.1.1 Linux:

Lo primero es descargar los archivos JAR desde el siguiente enlace de GitHub.

<https://github.com/deiivvv/GymTracker/tree/main/docker>

Descargaremos los archivos **Api-Ejercicios-0.0.1-SNAPSHOT.jar** y **GymTracker-0.0.1-SNAPSHOT.jar**

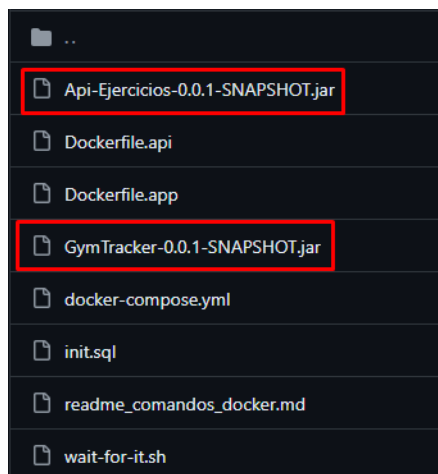


7.4.1.2 Windows:

Lo primero es descargar los archivos JAR desde el siguiente enlace de GitHub.

<https://github.com/deiivvv/GymTracker/tree/main/docker>

Descargaremos los archivos **Api-Ejercicios-0.0.1-SNAPSHOT.jar** y **GymTracker-0.0.1-SNAPSHOT.jar**

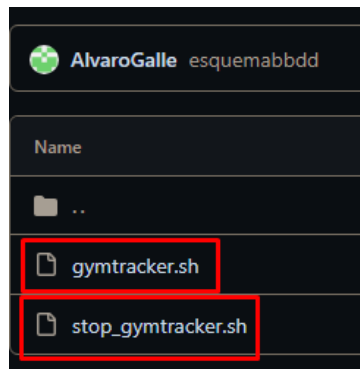


7.4.2 Preparación para el despliegue

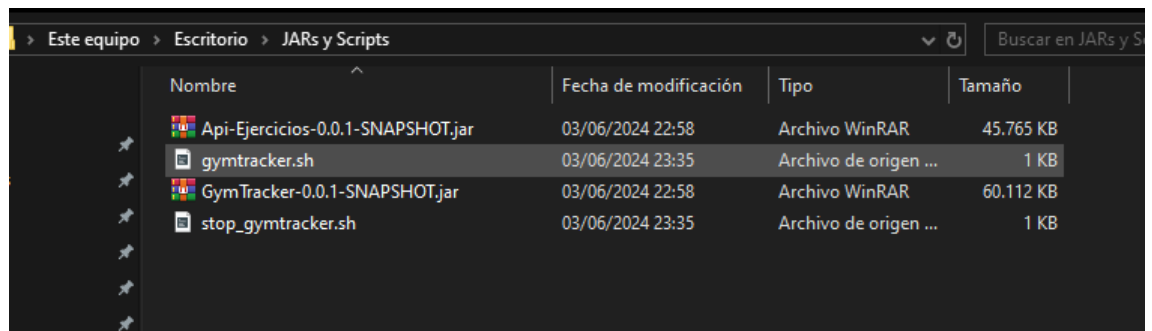
Para preparar el despliegue de nuestra aplicación en local comenzamos por descargar un script que va a ejecutar automáticamente nuestros archivos JAR.

Para ello entraremos al siguiente enlace y nos descargamos los dos archivos que hay: **gymtracker.sh** y **stop_gymtracker.sh**.

<https://github.com/deiivvv/GymTracker/tree/main/aws/Scripts>



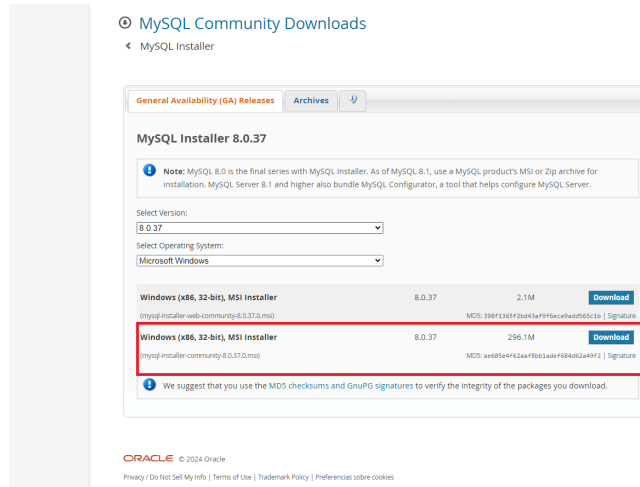
Lo más cómodo será guardar tanto los JAR como los scripts en una carpeta para no perderlos.



En segundo lugar deberemos instalar MySQL para instalar nuestra base de datos.

7.4.2.1 Windows:

1. Descargar MySQL Installer desde [MySQL](#)

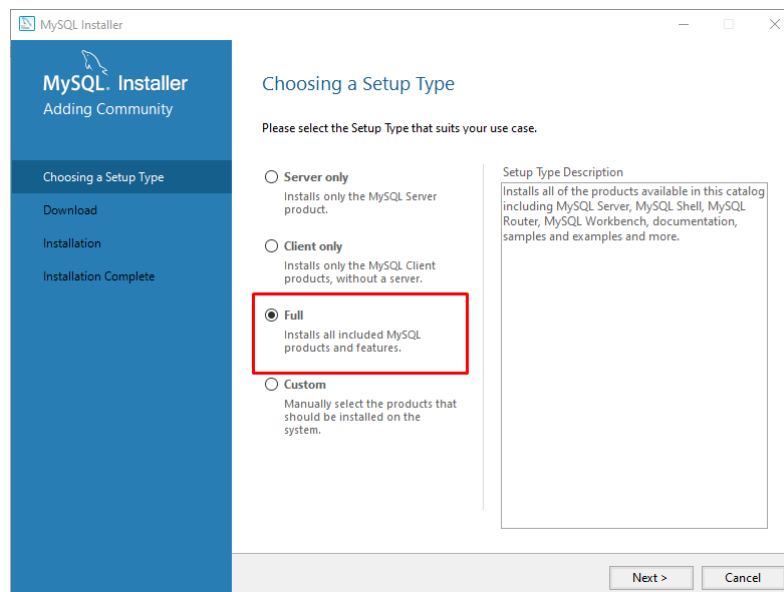


2. Ejecutar el instalador:

- Abre el archivo descargado.

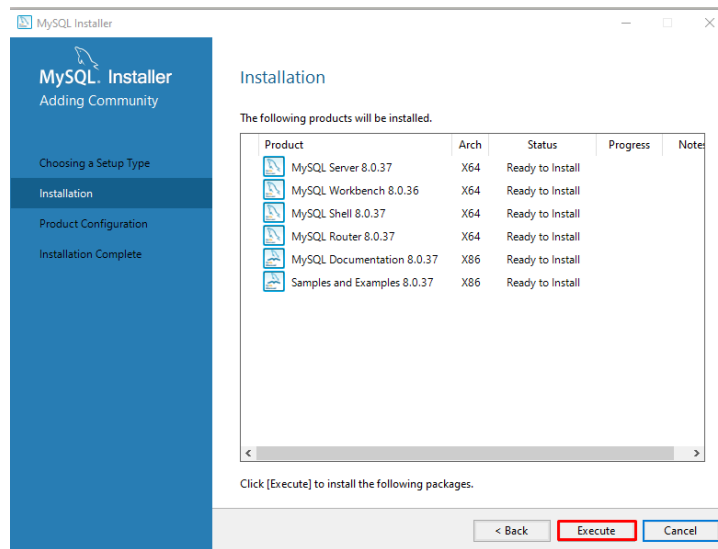
3. Seleccionar el tipo de instalación.

- Elige "Full" para una instalación completa.
- Haz clic en "Next".



4. Instalar las dependencias.

- El instalador verificará y descargará las dependencias necesarias.
- Haz clic en "Execute" para proceder.

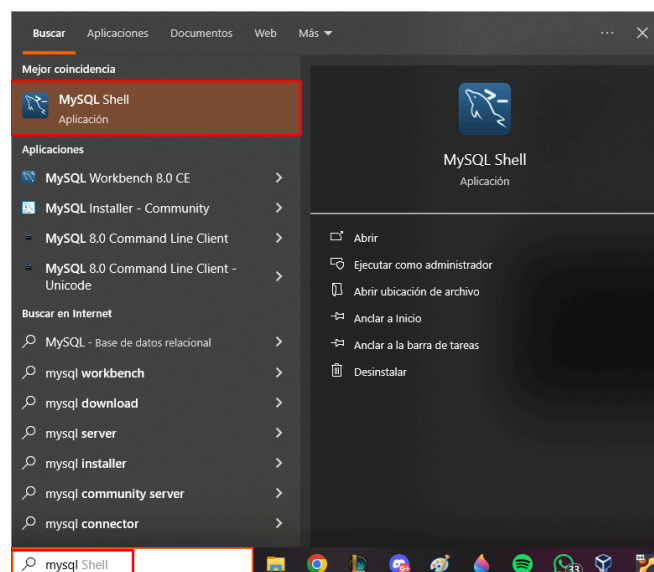


5. Configuración del servidor MySQL.

- Elige el tipo de configuración (General o Server Only).
- Configura el tipo y puerto de red (generalmente, puedes dejar las opciones predeterminadas).
- Establece una contraseña para el usuario `root` y, opcionalmente, crea usuarios adicionales.
- Configura el servicio de Windows si deseas que MySQL se inicie automáticamente con el sistema.

6. Finalizar la instalación.

- Una vez finalizada la instalación, puedes abrir MySQL Workbench o MySQL Shell para empezar a trabajar con MySQL.



```
MySQL Shell
MySQL Shell 8.0.37

Copyright (c) 2016, 2024, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its affiliates.
Other names may be trademarks of their respective owners.

Type '\help' or '? ' for help; '\quit' to exit.
MySQL JS >
```

7.4.2.1 Linux:

Para instalar MySQL en Linux (Debian)

1. Actualizar los repositorios:

sudo apt update

```
david@david-VirtualBox: ~
Archivo Editar Ver Buscar Terminal Ayuda
david@david-VirtualBox:~$ sudo apt update
```

2. Instalar MySQL:

sudo apt install mysql-server

```
david@david-VirtualBox: ~
Archivo Editar Ver Buscar Terminal Ayuda
david@david-VirtualBox:~$ sudo apt install mysql-server
```

3. Iniciamos MySQL para comprobar que funciona:

sudo mysql


```
david@david-VirtualBox: ~  
Archivo Editar Ver Buscar Terminal Ayuda  
david@david-VirtualBox:~$ sudo mysql  
[sudo] contraseña para david:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.36-0ubuntu0.22.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2024, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql>
```

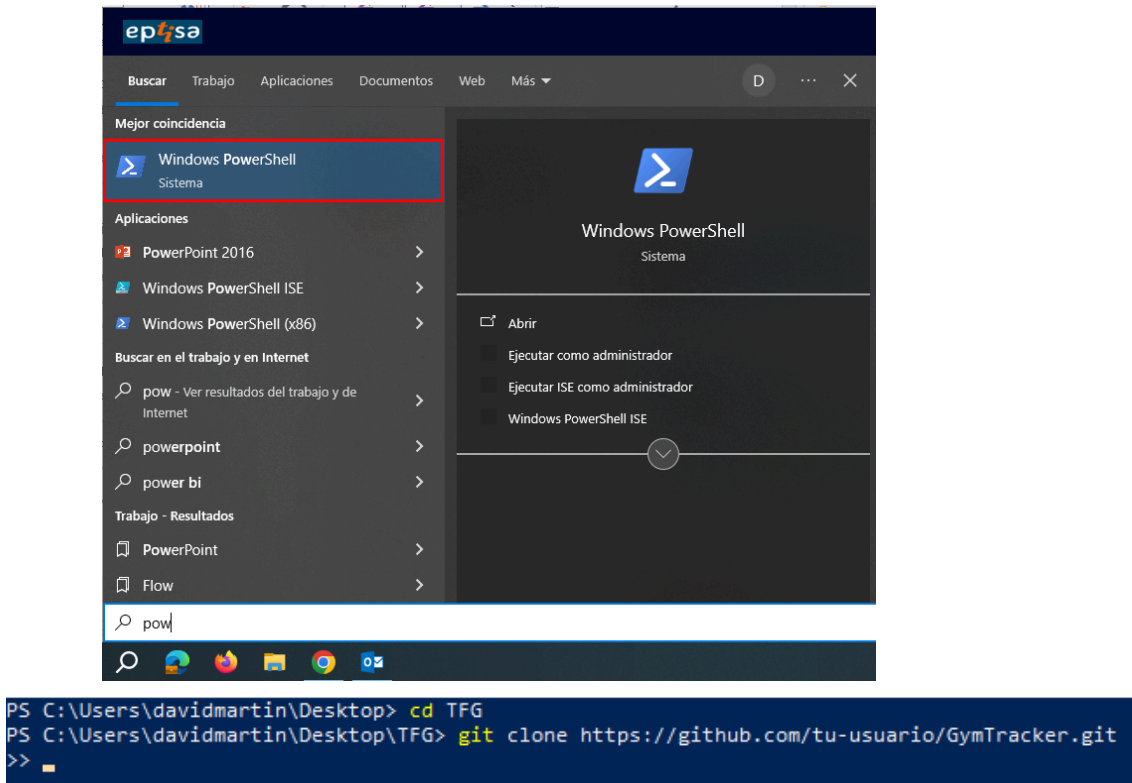
Estos pasos deberían permitirte instalar y configurar MySQL en tu sistema Linux.

7.4.2.1.1 Clonar repositorio de Github para windows

En una terminal de powershell (pulsamos boton de windows y escribimos powershell), nos posicionamos en el directorio donde queremos que se clone el repositorio, podemos movernos con el comando **cd**, una vez estemos situados en nuestro destino, ejecutamos el siguiente comando para clonar el repositorio:

git clone https://github.com/tu-usuario/GymTracker.git

Reemplaza `tu-usuario` con el nombre de tu usuario en GitHub.



7.4.2.1.2 Clonar repositorio de gitHub para linux

Abrimos una terminal, nos posicionamos en el directorio donde queremos que se clone el repositorio, podemos movernos con el comando **cd**, una vez estemos situados en nuestro destino, ejecutamos el siguiente comando para clonar el repositorio:

git clone https://github.com/tu-usuario/GymTracker.git

Reemplaza `tu-usuario` con el nombre de tu usuario en GitHub.

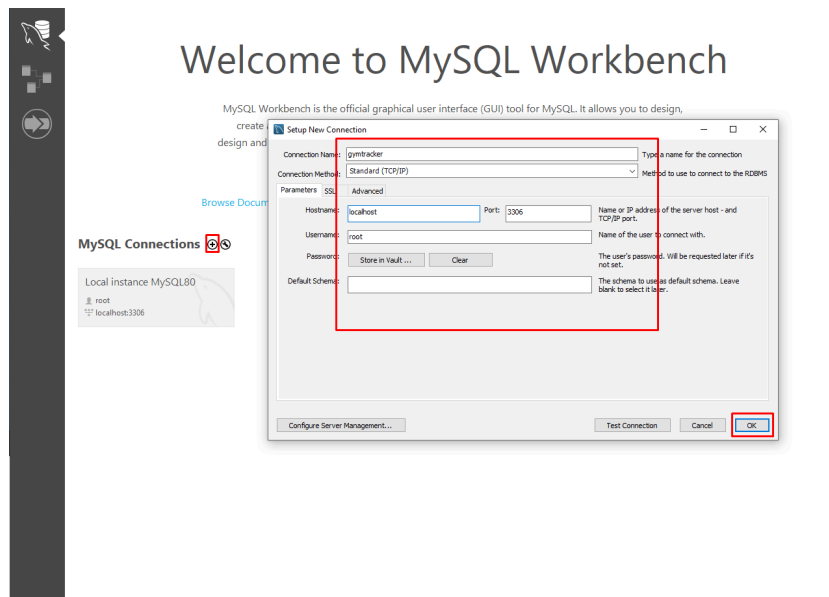
```

david@david-VirtualBox: ~/Escritorio
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
david@david-VirtualBox:~/Escritorio$ git clone https://github.com/deiivvv/GymTracker.git
Clonando en 'GymTracker'...
remote: Enumerating objects: 2238, done.
remote: Counting objects: 100% (257/257), done.
remote: Compressing objects: 100% (143/143), done.
remote: Total 2238 (delta 101), reused 208 (delta 63), pack-reused 1981
Recibiendo objetos: 100% (2238/2238), 47.89 MiB | 20.86 MiB/s, listo.
Resolviendo deltas: 100% (1039/1039), listo.
david@david-VirtualBox:~/Escritorio$
```

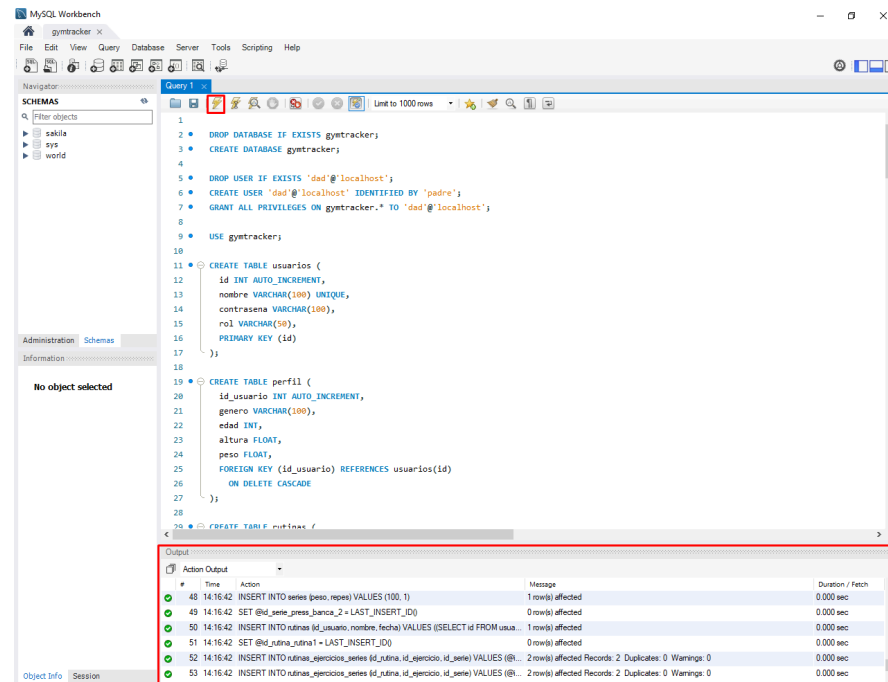
7.4.2.2.1 Volcar base de datos en windows

Abrimos la carpeta clonada de Github, entramos en la carpeta BBDD ya abrimos el archivo Gymtracker.sql y copiamos todo el contenido.

Ahora abrimos nuestro mysql workbench y creamos la conexión.



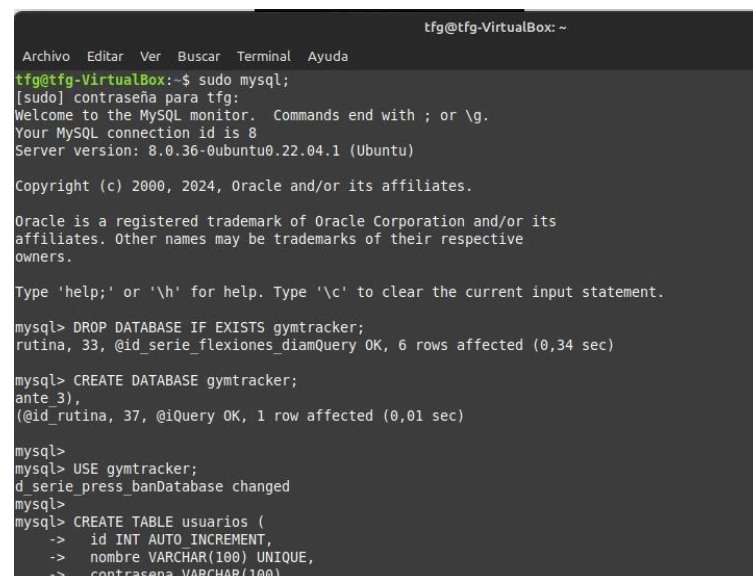
Pegamos el contenido en nuestro script y ya tendríamos nuestra base de datos creada.



7.4.2.2.2 Volcar base de datos en linux

Abrimos la carpeta clonada de Github, entramos en la carpeta BBDD y abrimos el archivo Gymtracker.sql y copiamos todo el contenido.

En una terminal iniciamos mysql y pegamos todo el contenido.



Ya tendríamos nuestra base de datos creada.

7.4.3 Despliegue

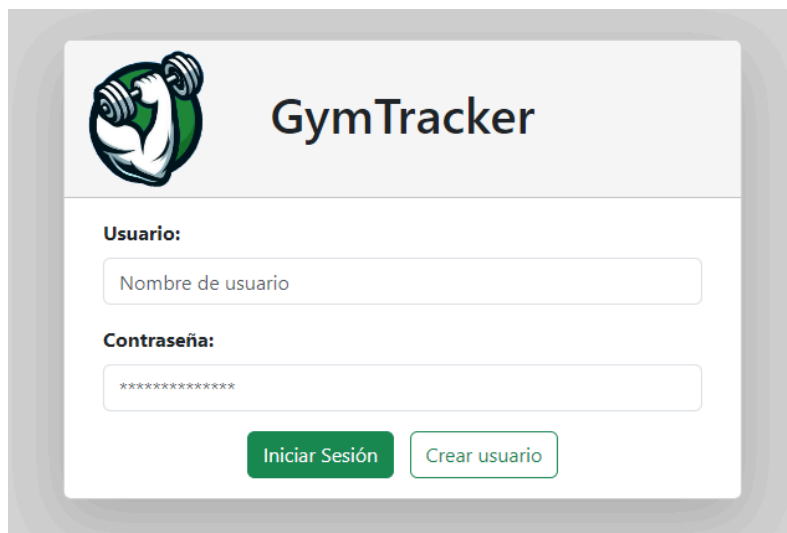
7.4.3.1 Despliegue en windows:

Por último vamos a ejecutar los JAR para que nuestra app funcione, para ello hay que abrir dos terminales de powershell como administrador, haciendo click derecho + ejecutar como administrador. Una vez dentro vamos a navegar con el comando `cd` hasta nuestra carpeta donde se encuentra el JAR.

Por último vamos a ejecutar el comando `java -jar [nombre del jar]` y ya habríamos acabado.

Vamos a comprobar que todo va bien yendo al navegador y escribiendo nuestra url.

`https://localhost`



The screenshot shows the GymTracker web application interface. At the top left is a logo of a muscular arm holding a dumbbell. To its right is the title "GymTracker". Below the logo, there are two input fields: "Usuario:" with a placeholder "Nombre de usuario" and "Contraseña:" with a placeholder "*****". At the bottom, there are two buttons: "Iniciar Sesión" (green) and "Crear usuario" (white with green border).

8. Pruebas

Suceso	Supuesto	Resultado	Solución	Conclusión
Conexión a la base de datos	Queremos consumir la tabla "usuarios" de la base de datos.	Error al conectarse a la base de datos y la aplicación no se levanta correctamente.	Configurar la URL y las credenciales de la base de datos en el archivo application.properties.	Es imprescindible configurar correctamente los detalles de la conexión a la base de datos en el archivo application.properties para que una aplicación de Spring Boot funcione correctamente.
Crear nuevo usuario en la aplicación	El usuario se guarda correctamente en la base de datos y también se crea como usuario de Spring Security con acceso	Los usuarios se guardan en la base de datos correctamente, pero no se crean automáticamente como usuarios de Spring Security con acceso.	Usar InMemoryUserDetailsManager que permita actualizar los usuarios de Spring Security.	Al combinar JPA e InMemoryUserDetailsManager, podemos gestionar tanto la persistencia de los usuarios en la base de datos como su integración en Spring Security.
Implementar un sidebar dinámico en la aplicación web.	El sidebar debe cambiar su contenido o posición dependiendo de la pantalla en la que te encuentres.	El sidebar funciona correctamente.	Uso de javascript para cambiar los estilos del sidebar.	Un sidebar dinámico mejora la usabilidad y la experiencia del usuario al proporcionar contenido relevante según el contexto.
Acceso a la aplicación mediante login	Todos los usuarios podrán acceder a la aplicación a través de un login excepto los que tengan el rol de "bloqueado".	Todos los usuarios acceden sin excepciones.	Comprobar el rol de los usuarios y capturar el rol "bloqueado" para denegar el acceso..	La autenticación debe verificar y bloquear a los usuarios con el rol "bloqueado" para que no puedan acceder a la aplicación.
Consumo de la API para el	Se deben listar y crear los	No se encuentran ejercicios. No hay	Crear un archivo de configuración	Crear un archivo de configuración

listado y creación de ejercicios.	ejercicios consumidos de la API.	conexión con la api	y el método addCorsMappings para regular las direcciones y métodos de acceso.	y el método addCorsMappings para regular las direcciones y métodos de acceso.
Verificar el acceso a la aplicación de usuarios con el mismo nombre de usuario.	Verificar el acceso a la aplicación de usuarios con el mismo nombre de usuario.	Problemas al reconocer la contraseña de cada usuario. .	Exclusividad del nombre de usuario al crear un nuevo usuario.	Asegurar que el nombre de usuario sea único al crear un nuevo usuario para evitar problemas de autenticación.
Guardado de rutinas y ejercicios en la base de datos	Tras crear la rutina se guarda en la base de datos junto con los ejercicios y series de esta	Problemas al guardarlo y luego tratar de recuperarlo	Modificaciones en la estructura de la base de datos. tabla intermedia de relación ternaria	La importancia de una estructura de base de datos bien diseñada y adaptable para el correcto funcionamiento de la aplicación.
Editar perfil	Actualizar los cambios dentro de la base de datos	Los datos se actualizan correctamente.	Consulta la base de datos con la sentencia UPDATE	Las consultas de actualización deben ser precisas para mantener la integridad de los datos en la aplicación.
Hashear contraseñas	Almacenar las contraseñas hasheadas y comprobarlas en el login	Las funciones de JavaScript permiten hashear pero no logramos hacer comprobaciones	Hashear las contraseñas a través del encriptado de Spring Security	Spring Security proporciona mecanismos seguros para hashear y verificar contraseñas.
Acceder a los perfiles como administrador	Poder acceder a los perfiles los usuarios sin saber sus contraseñas	El la sesión no cambia	Recuperar el id de usuario al que queremos acceder	Implementar un mecanismo para que el administrador pueda cambiar temporalmente el contexto de seguridad y actuar como otro usuario.

Crear gráfica de barras	Se creará un diagrama de barras con los datos devueltos por la petición axios	El java script no funciona como se esperaba	Uso de una librería de javaScript, Chart.	El uso de la librería Chart.js permitió la creación exitosa de un diagrama de barras a partir de los datos obtenidos. Esto resalta la utilidad de las librerías de JavaScript para el desarrollo.
Implementar funcionalidad de cierre de sesión.	Tras cerrar sesión, el usuario no debe poder volver a acceder a las pantallas de la aplicación.	Spring Security elimina la sesión y restringe el acceso.	Configurar Spring Security para manejar el cierre de sesión.	El cierre de sesión elimina la sesión del usuario y restringe el acceso a las pantallas de la aplicación, asegurando que solo los usuarios autenticados puedan acceder.

9. Conclusiones

El desarrollo del proyecto GymTracker ha sido un esfuerzo integral que ha abarcado diversas áreas clave de la tecnología y el desarrollo web, proporcionando un valioso aprendizaje y resultados significativos.

9.1 Implementación Exitosa de Tecnologías y Herramientas

- **Spring Boot y Spring Security:** La elección de Spring Boot como el framework principal ha demostrado ser adecuada para la creación de aplicaciones web robustas y escalables. La integración de Spring Security ha añadido una capa crítica de seguridad, garantizando la protección de datos y la gestión segura de usuarios .
- **Uso de Docker y aws:** La utilización de Docker y aws han facilitado el despliegue y la portabilidad de la aplicación, permitiendo una configuración coherente en distintos entornos de desarrollo y producción .
- **Bootstrap y Chart.js:** Estas herramientas han mejorado significativamente la interfaz de usuario, haciéndola más atractiva y funcional. La implementación de gráficos

interactivos mediante Chart.js ha proporcionado a los usuarios una manera visualmente intuitiva de seguir su progreso .

9.2 Desafíos y Soluciones

- **Integración de APIs:** La creación y consumo de una API separada para gestionar ejercicios ha permitido una modularidad y una facilidad de actualización futuras. La correcta configuración de CORS fue crucial para permitir el acceso controlado a la API desde diferentes orígenes.
- **Seguridad y Gestión de Usuarios:** Implementar un sistema de autenticación y autorización con Spring Security presentó desafíos iniciales, especialmente en la gestión de roles y permisos de usuarios. No obstante, estos fueron superados, resultando en una aplicación segura y confiable .

9.3 Lecciones Aprendidas

- **Importancia de la Planificación y la Documentación:** La planificación detallada y la documentación adecuada en cada fase del proyecto han sido vitales para el éxito del desarrollo. Estas prácticas no solo han facilitado el flujo de trabajo sino que también han asegurado una referencia clara para futuros desarrollos o mantenimientos .
- **Adaptabilidad y Resolución de Problemas:** La capacidad para adaptarse y resolver problemas imprevistos ha sido un aprendizaje crucial. Desde errores en la conexión a la base de datos hasta ajustes en la configuración de seguridad, cada desafío ha aportado un valioso aprendizaje sobre cómo abordar y solucionar problemas técnicos .

9.4 Impacto y Futuras Mejoras

- **Facilidad de Uso y Funcionalidad:** La aplicación GymTracker proporciona una plataforma intuitiva y eficiente para la gestión de rutinas de gimnasio, contribuyendo a una mejor experiencia de usuario y a un seguimiento efectivo del progreso en el entrenamiento.
- **Expansión y Escalabilidad:** La arquitectura modular y el uso de tecnologías modernas permiten que GymTracker sea fácilmente expandible. Futuras mejoras pueden incluir la incorporación de nuevas funcionalidades, como análisis avanzados de datos de entrenamiento y la integración con dispositivos de fitness y redes sociales.

GymTracker ha logrado cumplir con los objetivos establecidos, ofreciendo una herramienta valiosa tanto para usuarios individuales como para administradores de gimnasios. El proyecto ha proporcionado un amplio aprendizaje en tecnologías web y ha sentado una base sólida para futuros desarrollos y mejoras.

10. Bibliografía

10.1 Enlaces de descarga

10.1.1 Oracle JDK



<https://www.oracle.com/java/technologies/downloads>

10.1.2 OpenJDK



<https://jdk.java.net>

10.1.3 Git



<https://git-scm.com/download/win>

10.1.4 Spring Tool Suite



<https://spring.io/tools>

10.1.5 MySQL



<https://dev.mysql.com/downloads/installer>

10.1.6 Lombok



<https://projectlombok.org/download>

10.2 Documentación Oficial



10.2.1 Spring Boot

<https://spring.io/projects/spring-boot>



10.2.2 Java

<https://docs.oracle.com/en/java/>



10.2.3 MySQL

<https://dev.mysql.com/doc/>



10.2.4 JavaScript

<https://developer.mozilla.org/en-US/docs/Web/JavaScript>



10.2.5 Axios

<https://axios-http.com/docs/intro>



10.2.6 Bootstrap

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>



10.2.7 HTML

<https://developer.mozilla.org/en-US/docs/Web/HTML>



10.2.8 CSS

<https://developer.mozilla.org/en-US/docs/Web/CSS>



10.2.9 AWS

<https://docs.aws.amazon.com/>



10.2.10 Docker

<https://docs.docker.com/>



10.2.11 Git

<https://www.git-scm.com/doc>



10.2.12 Lombok

<https://projectlombok.org/features/>



10.2.13 Apache Tomcat

<https://tomcat.apache.org/>

10.3 Otras páginas de consulta



10.3.1 Stack Overflow

<https://stackoverflow.com/>



10.3.2 W3Schools

<https://www.w3schools.com/>



10.3.3 ChatGPT

<https://chatgpt.com/>



10.3.4 Microsoft designer

<https://designer.microsoft.com/>

10.4 Páginas importantes en el desarrollo



10.4.1 Duck DNS

<https://www.duckdns.org/>



10.4.2 AWS Academy

<https://awsacademy.instructure.com/>



10.4.3 Github

<https://github.com/>

10.5 Enlaces para el despliegue en Docker

10.5.1 Tutorial para instalar docker

<https://www.youtube.com/watch?v=ZO4KWQfUBBc>

10.5.2 Instalar WSL

<https://learn.microsoft.com/en-us/windows/wsl/install>

10.5.3 Wait-for-it

<https://github.com/vishnubob/wait-for-it>

10.5.4 Instalar docker

<https://docs.docker.com/desktop/install/windows-install/>